

Contents

Base64 関数リファレンス	6
関数一覧	6
使用例	6
エラーハンドリング	7
文字列操作関数リファレンス	7
関数一覧	7
contains	9
starts_with	9
ends_with	9
find	9
substring	9
char_at	10
replace	10
to_upper	10
to_lower	10
trim	11
trim_start	11
trim_end	11
repeat	11
reverse	11
byte_len	11
split	12
join	12
to_int	12
to_float	13
to_str	13
組み込み関数リファレンス	14
関数一覧	14
print	18
Some	19
length	19
has_key	19
add	19
remove	20
range	20
exit	20
arguments	21
sleep	21
env	21
append	22
pop	22
reverse (list)	22
slice	23
take	23
tap	23
filter	23
map	23
sort	24
sort!	24
reverse!	24
appended	24

append!	25
first	25
last	25
get (Map)	25
iter	25
next	26
to_list	26
type_of	26
コレクションリファレンス (タプル・リスト・マップ・セット)	27
タプル	27
リスト	28
Copy-on-Write (CoW) セマンティクス	36
マップ	37
セット	39
イテレータ	41
契約による設計 (Design by Contract)	42
事前条件 (require)	42
事後条件 (ensure)	43
組み合わせ例	43
レコードの不変条件 (invariant)	43
ルール	44
制御構文リファレンス	44
if / else	44
while	45
for	46
async / await	48
@parallel for	49
break	49
continue	49
... (Ellipsis)	49
case	50
対象ありの case (パターンマッチング)	50
スコープルール	54
ディレクティブ	55
構文	55
対象	55
組み込みディレクティブ	55
注意事項	61
エラーリファレンス	61
エラーフォーマット	61
コンパイルエラー一覧	62
ランタイムエラー一覧	63
ファイルシステム関数リファレンス	64
関数一覧	64
使用例	64
注意事項	66
関数リファレンス	67
関数定義の構文	67

引数と戻り値の型	67
ネスト関数	68
再帰	69
オーバーロード	71
デフォルト引数	71
Unit 型関数	72
Task と async 関数	72
ラムダ関数	73
クロージャ	73
関数型	74
高階関数	75
ジェネリック関数	75
UFCS (統一関数呼び出し構文)	77
演算子オーバーロード	78
チェック付き/飽和演算	79
GC リファレンス	80
概要	80
インポート	80
関数	80
仕組み	80
静的解析による最適化	81
使用例	81
HTTP リファレンス	81
型	81
関数 (http パッケージ)	81
使用例	83
動作仕様	85
サポートされるステータスコード	86
HTTP クライアント	87
エラー処理	88
I/O 関数リファレンス	89
関数一覧	89
使用例	89
エラーハンドリング	90
備考	91
JSON 関数リファレンス	91
概要	91
関数一覧	91
Result<JsonValue, Error> のアンラップ	92
使用例	93
注意事項	94
数学関数 (math)	94
概要	94
定数	94
絶対値	94
丸め	94
冪乗・平方根	95
対数	95
指数	96
三角関数	96

逆三角関数	96
2 引数関数	96
判定関数	97
ネットワーク (TCP) リファレンス	97
型	97
関数 (net パッケージ)	97
組み込みオーバーロード関数	98
使用例	98
タイムアウト設定	100
エラー処理	100
バイト変換	101
演算子リファレンス	101
優先順位テーブル	101
算術演算子	101
比較演算子	102
論理演算子	102
ビット演算子	102
エラー伝播演算子 (? / !!)	103
case: 条件式	104
範囲演算子	104
null 合体演算子 (??)	104
複合代入演算子	105
インクリメント・デクリメント演算子	106
演算の型規則	106
演算子オーバーロード	106
パッケージリファレンス	110
概要	110
インポート構文	110
パッケージ解決	111
標準ライブラリ (std)	112
検索パスの優先順位	113
RY_PATH 環境変数	113
制約	114
パッケージファイルの作成	114
Native 関数の命名規約	114
パス関数リファレンス	115
関数一覧	115
使用例	116
プロジェクト管理	117
CLI 概要	117
ry init - プロジェクト初期化	118
ry new - 新規プロジェクト作成	118
ry run - プロジェクトスクリプトの実行	118
ry fmt - コードフォーマッター	119
ry test - テスト実行	119
ry self-update - セルフアップデート	120
package.toml 設定ファイル	121
正規表現リファレンス	122
正規表現リテラル構文	122

関数一覧	122
サポートするパターン構文	123
使用例	124
構造体リファレンス	125
概要	125
定義構文	125
コンストラクタ	126
フィールドアクセス	126
フィールド代入	126
関数引数・戻り値としての使用	127
ネスト構造体	127
比較 (== / !=)	127
レコードのサブタイプ化 (継承)	127
制約とエラー	129
列挙型 (enum)	129
テスト機能	131
実行方法	131
構文	131
出力形式	134
例	134
モック	134
パラメタライズドテスト (@each)	135
プロパティベーステスト (@property)	136
テストカバレッジ	136
テストアウトライン	137
テスト説明のスタイル	137
制限事項	138
スレッドリファレンス	138
型	138
インポート	138
Thread	138
Lock (ミューテックス)	139
RWLock (読み書きロック)	140
Semaphore	140
Barrier	140
AtomicInt	141
AtomicBool	141
async/await との比較	142
型リファレンス	142
型一覧	142
型アノテーション構文	144
使用可能な型名一覧	144
型エイリアス	145
数値リテラル	146
リテラル型	146
範囲型	147
none キーワードと Option 型の省略記法	147
弱参照 (weak T)	148
F-String (文字列補間)	149
型キャスト (as)	149
関連データを持つ enum (ADT)	150

ジェネリック enum	151
Error 型	151
Type	152
union 型	153
any 型	154
型規則（演算時の型変換）	156
型安全性の制約	157

[English](#) | [日本語](#) | [繁體中文](#)

Base64 関数リファレンス

Base64 エンコード・デコード。すべての関数は `base64` からの明示的なインポートが必要です。

```
from base64 import encode, decode, encode_url_safe, decode_url_safe
```

関数一覧

関数	シグネチャ	説明
<code>encode</code>	<code>(str) -> str</code>	文字列を標準 base64 にエンコード
<code>decode</code>	<code>(str) -> Result<str, Error></code>	標準 base64 文字列をデコード
<code>encode_url_safe</code>	<code>(str) -> str</code>	文字列を URL-safe base64 にエンコード（パディングなし）
<code>decode_url_safe</code>	<code>(str) -> Result<str, Error></code>	URL-safe base64 文字列をデコード

使用例

基本的なエンコード・デコード

```
from base64 import encode, decode

encoded = encode("Hello, World!")
print(encoded)  # SGVsbG8sIFdvcmxkIQ==

case decode(encoded):
    Ok(s):
        print(s)  # Hello, World!
    Err(e):
        print(e.message)
```

URL-safe Base64

URL-safe base64 は + と / の代わりに - と _ を使用し、パディング (=) を省略します。URL、ファイル名、トークンに適しています。

```
from base64 import encode_url_safe, decode_url_safe

encoded = encode_url_safe("data with special chars: ?&=")
# 出力に + / = は含まれない
```

```
case decode_url_safe(encoded):
  Ok(s):
    print(s)
  Err(e):
    print(e.message)
```

バイトデータの操作

バイトデータのエンコード・デコードには `io` の `to_bytes` / `bytes_to_str` と組み合わせます。

```
from base64 import encode, decode
from io import to_bytes, bytes_to_str

bytes = to_bytes("binary data")
encoded = encode(bytes_to_str(bytes))
```

エラーハンドリング

`decode` と `decode_url_safe` は `Result<str, Error>` を返します。無効な base64 文字が含まれている場合、デコードは失敗します。

```
case decode("!!!not-valid!!!"):
  Ok(s):
    print(s)
  Err(e):
    print(e.message) # "invalid base64 character at position 0"
```

? 演算子との組み合わせ:

```
function process(input: str) -> Result<str, Error>:
  decoded = decode(input)?
  return Ok(decoded)
```

[English](#) | [日本語](#) | [繁體中文](#)

文字列操作関数リファレンス

文字列 (`str`) に対する操作関数の一覧です。すべての関数で UFCS 記法が使用可能です。

注意: すべての文字列操作は UTF-8 対応です。 `length()`、`char_at()`、`substring()`、`find()`、`reverse()` は Unicode コードポイント単位で動作し、バイト単位ではありません。バイト長が必要な場合は `byte_len()` を使用してください。

関数一覧

検索・判定

関数	シグネチャ	説明
<code>contains</code>	<code>(str, str, bool = false) -> bool</code>	部分文字列が含まれるかを返す
<code>starts_with</code>	<code>(str, str, bool = false) -> bool</code>	接頭辞で始まるかを返す
<code>ends_with</code>	<code>(str, str, bool = false) -> bool</code>	接尾辞で終わるかを返す
<code>find</code>	<code>(str, str) -> Option<int></code>	部分文字列の文字位置を返す (未発見は <code>None</code>)

抽出・変換

関数	シグネチャ	説明
substring	(str, int, int) -> str	部分文字列を取得（文字インデックス）
char_at	(str, int) -> str	指定位置の UTF-8 文字を取得
replace	(str, str, str) -> str	部分文字列を全置換

大文字・小文字

関数	シグネチャ	説明
to_upper	str -> str	ASCII 大文字に変換
to_lower	str -> str	ASCII 小文字に変換

空白除去

関数	シグネチャ	説明
trim	str -> str	前後の空白を除去
trim_start	str -> str	先頭の空白を除去
trim_end	str -> str	末尾の空白を除去

生成・加工

関数	シグネチャ	説明
repeat	(str, int) -> str	文字列を n 回繰り返す
reverse	str -> str	文字列を逆順にする（UTF-8 対応）
byte_len	str -> int	文字列のバイト長を返す

分割・結合

関数	シグネチャ	説明
split	(str, str) -> List<str>	デリミタで分割
join	(List<str>, str) -> str	セパレータで結合

型変換

関数	シグネチャ	説明
to_int	str -> Result<int, Error>	文字列を整数に変換
to_float	str -> Result<float, Error>	文字列を浮動小数点数に変換
to_str	int/float/bool/str/enum/record -> str	値を文字列に変換

contains

シグネチャ: `contains(string: str, substring: str, ignore_case: bool = false) -> bool`

文字列 `string` に部分文字列 `substring` が含まれるかを返します。 `ignore_case` が `true` の場合、比較は大文字小文字を区別しません (ASCII のみ)。

```
print(contains("hello", "ell"))           # true
print("hello".contains("xyz"))           # false (UFCS)
print(contains("Hello World", "hello", true)) # true (大文字小文字区別なし)
```

starts_with

シグネチャ: `starts_with(string: str, prefix: str, ignore_case: bool = false) -> bool`

文字列 `string` が `prefix` で始まるかを返します。 `ignore_case` が `true` の場合、比較は大文字小文字を区別しません (ASCII のみ)。

```
print(starts_with("hello", "hel"))        # true
print("hello".starts_with("world"))       # false (UFCS)
print(starts_with("Hello", "hello", true)) # true (大文字小文字区別なし)
```

ends_with

シグネチャ: `ends_with(string: str, suffix: str, ignore_case: bool = false) -> bool`

文字列 `string` が `suffix` で終わるかを返します。 `ignore_case` が `true` の場合、比較は大文字小文字を区別しません (ASCII のみ)。

```
print(ends_with("hello", "llo"))          # true
print("hello".ends_with("world"))         # false (UFCS)
print(ends_with("Hello World", "WORLD", true)) # true (大文字小文字区別なし)
```

find

シグネチャ: `find(string: str, substring: str) -> Option<int>`

文字列 `string` 中の部分文字列 `substring` の最初の出現位置 (文字位置) を返します。見つからない場合は `None` を返します。

```
print(find("hello world", "world"))       # Some(6)
print(find("hello", "xyz"))               # None
print("abcdef".find("cd"))                # Some(2) (UFCS)
```

substring

シグネチャ: `substring(string: str, start: int, end: int) -> str`

文字列 `string` の `start` から `end` (排他) までの部分文字列を返します。インデックスは文字位置 (UTF-8 対応) です。

範囲外のインデックスは `[0, length]` にクランプされます。クランプ後に `end < start` の場合は空文字列を返します。

```
print(substring("hello world", 0, 5))    # hello
print(substring("hello world", 6, 11))   # world
print("abcdef".substring(1, 4))           # bcd (UFCS)
print(substring("hello", -1, 100))        # hello (クランプされる)
```

char_at

シグネチャ: `char_at(string: str, i: int) -> str`

文字列 `string` の `i` 番目の UTF-8 文字を文字列として返します。インデックスが範囲外の場合はランタイムエラーになります。

負のインデックスは末尾から数えます (Python スタイル) : `-1` は最後の文字、`-2` は最後から 2 番目の文字を指します。

```
print(char_at("hello", 0))    # h
print(char_at("hello", -1))   # o (最後の文字)
print("abc".char_at(2))       # c (UFCS)
```

replace

シグネチャ: `replace(string: str, old: str, new: str) -> str`

文字列 `string` 中の `old` をすべて `new` に置換した新しい文字列を返します。

`old` が空文字列の場合、入力はそのまま (新しいコピーとして) 返されます。

```
print(replace("hello world", "world", "ry"))    # hello ry
print(replace("aaa", "a", "bb"))                # bbbbbb
print("foo bar foo".replace("foo", "baz"))       # baz bar baz (UFCS)
print(replace("hello", "", "X"))                 # hello (空のパターンは何もしない)
```

to_upper

シグネチャ: `to_upper(string: str) -> str`

ASCII 小文字 (a-z) を大文字に変換した新しい文字列を返します。

```
print(to_upper("hello"))    # HELLO
print("Hello World".to_upper()) # HELLO WORLD (UFCS)
```

to_lower

シグネチャ: `to_lower(string: str) -> str`

ASCII 大文字 (A-Z) を小文字に変換した新しい文字列を返します。

```
print(to_lower("HELLO"))    # hello
print("Hello World".to_lower()) # hello world (UFCS)
```

trim

シグネチャ: `trim(string: str) -> str`

文字列の前後の空白文字（スペース、タブ、改行、復帰）を除去した新しい文字列を返します。

```
print(trim(" hello "))    # hello
print(" hi ".trim())      # hi (UFCS)
```

trim_start

シグネチャ: `trim_start(string: str) -> str`

文字列の先頭の空白文字を除去した新しい文字列を返します。

```
print(trim_start(" hello "))  # hello
print(" hi ".trim_start())    # hi (UFCS)
```

trim_end

シグネチャ: `trim_end(string: str) -> str`

文字列の末尾の空白文字を除去した新しい文字列を返します。

```
print(trim_end(" hello "))    # hello
print("hi ".trim_end())       # hi (UFCS)
```

repeat

シグネチャ: `repeat(string: str, count: int) -> str`

文字列 `string` を `count` 回繰り返した新しい文字列を返します。

```
print(repeat("ab", 3))        # ababab
print("ha".repeat(3))         # hahaha (UFCS)
```

reverse

シグネチャ: `reverse(string: str) -> str`

文字列を逆順にした新しい文字列を返します（UTF-8 対応）。

```
print(reverse("hello"))       # olleh
print("abc".reverse())        # cba (UFCS)
```

byte_len

シグネチャ: `byte_len(string: str) -> int`

文字列 `string` のバイト長を返します。UTF-8 文字数を返す `length()` とは異なり、`byte_len()` はバイト数を返します。

```
print(byte_len("hello"))    # 5
print(byte_len("あいう"))   # 9
print(length("あいう"))      # 3 (文字数)
```

split

シグネチャ: `split(string: str, delimiter: str) -> List<str>`

文字列 `string` をデリミタ `delimiter` で分割し、`List<str>` を返します。

デリミタが空文字列 `""` の場合、文字列は個別の文字に分割されます (UTF-8 対応)。

```
parts = split("a,b,c", ",")
print(parts[0])    # a
print(parts[1])    # b
print(parts[2])    # c
```

```
for word in "hello world".split(" "):
    print(word)
# hello
# world
```

文字ごとに分割

```
chars = split("hello", "")
print(chars)    # [h, e, l, l, o]
```

UTF-8 文字

```
chars = split("あいう", "")
print(chars)    # [あ, い, う]
```

Tip: 文字列を 1 文字ずつ反復するなら、`split` を呼ばずに `for` ループを直接使えます。 `for c in s:` は各 UTF-8 コードポイントを 1 文字の `str` として生成します。詳細は [control-flow.md](#) を参照してください。

join

シグネチャ: `join(values: List<str>, sep: str) -> str`

文字列リストの要素をセパレータ `sep` で結合した文字列を返します。

```
parts = ["a", "b", "c"]
print(join(parts, ","))    # a,b,c
print(parts.join("-"))    # a-b-c (UFCS)
print(",".join(parts))    # a,b,c (UFCS, Python スタイル)
```

to_int

シグネチャ: `to_int(string: str) -> Result<int, Error>`

文字列を整数に変換します。先頭の空白は許容されます。文字列が空の場合、無効な文字を含む場合、またはオーバーフローする場合は `Err` を返します。

```
case to_int("42"):
    Ok(v):
```

```

        print(v)                # 42
    Err(e):
        print(e.message)

case "123".to_int():
    Ok(v):
        print(v)                # 123
    Err(e):
        print(e.message)

# 無効な入力 は Err を返す
print(to_int("abc"))            # Err(Error("to_int: invalid character in 'abc'"))
print(to_int(""))              # Err(Error("to_int: empty string"))

```

to_float

シグネチャ: `to_float(string: str) -> Result<float, Error>`

文字列を浮動小数点数に変換します。文字列が空の場合、無効な文字を含む場合、または float の範囲外の場合は Err を返します。

```

case to_float("3.14"):
    Ok(v):
        print(v)                # 3.14
    Err(e):
        print(e.message)

case "2.5".to_float():
    Ok(v):
        print(v)                # 2.5
    Err(e):
        print(e.message)

# 無効な入力 は Err を返す
print(to_float("abc"))          # Err(Error("to_float: invalid character in 'abc'"))
print(to_float(""))            # Err(Error("to_float: empty string"))
print(to_float("1e400"))       # Err(Error("to_float: out of range in '1e400'"))

```

to_str

シグネチャ: `to_str(v: any) -> str`

任意の Ry 値を受け付けます。サポートされる入力型は下の表を参照してください。

値を文字列に変換します。

型	変換形式
int	%ld
float	%g、整数値の場合は末尾に .0 を付加 (例: "3.0", "0.0")
bool	"true" / "false"
str	そのまま返す
enum	バリエーション名 (例: "Red")
record	TypeName(field1: val1, field2: val2)

型	変換形式
List / Map / Set	再帰的にフォーマットされ、ネストしたコンテナ（例: Map<str, List<int>>）もサポート
ユニオン	アクティブなバリエーションとしてフォーマット。List, Map, Set, 関数バリエーションもすべてサポート
関数値（クロージャ / ラムダ）	"<closure>"

整数値の float は末尾に .0 が付加されるため（例: `to_str(3.0) == "3.0"`）、int と視覚的に区別できます。record 型は `to_str` 表現を自動生成します。ユーザー定義の `function to_str(v: MyRecord) -> str` が提供されている場合、自動生成バージョンより優先されます。これは `print()` や f-string 補間でも同様に機能します。

```
print(to_str(42))           # 42
print(to_str(3.14))        # 3.14
print(to_str(3.0))         # 3.0      (整数値の float は .0 を保持)
print(to_str(true))        # true
print(99.to_str())         # 99 (UFCS)
```

```
enum Color:
    Red
    Green
```

```
print(to_str(Color::Red))  # Red
```

```
record Point:
    x: int
    y: int
```

```
p = Point(10, 20)
print(to_str(p))           # Point(x: 10, y: 20)
print(p)                   # Point(x: 10, y: 20)
print(f"pos={p}")          # pos=Point(x: 10, y: 20)
```

[English](#) | [日本語](#) | [繁體中文](#)

組み込み関数リファレンス

関数一覧

コア

関数	説明
<code>print()</code> / <code>print(expr1, expr2, ...)</code> <code>length(value)</code>	値を標準出力に表示（スペース区切り） リスト・マップ・セットの要素数、文字列の UTF-8 文字数を返す
<code>range(n)</code> / <code>range(start, end)</code> / <code>range(start, end, step)</code>	整数のリストを生成
<code>exit(code)</code> <code>arguments()</code>	指定した終了コードでプロセスを終了 コマンドライン引数を List<str> として返す
<code>available_parallelism()</code> <code>sleep(duration_ms)</code>	ランタイムの worker 数を int で返す 指定ミリ秒間、実行を一時停止する
<code>env(key)</code> <code>env(key, default)</code>	環境変数を Option<str> で返す 環境変数を返す。未設定なら default を返す

関数	説明
<code>send(stream, data)</code>	TcpStream または TlsStream を通じて List<u8> を送信し、Result<int, Error> を返す
<code>receive(stream, max)</code>	TcpStream または TlsStream から最大 max バイトを Result<List<u8>, Error> として受信
<code>close(handle)</code>	TcpStream、TlsStream、または TcpListener を閉じる
<code>block_on(task)</code>	現在のスレッドを Task<T> の完了までブロックし、結果を返す
<code>to_str(value)</code>	値を文字列表現に変換する。int、float（整数値は末尾に.0 を付加）、bool、str、record、enum、タプル、List、Map、Set（Map<str, List<int>> のようなネストしたコンテナは再帰的にフォーマット）、Result、Option、ユニオン型（アクティブなバリエーションとしてフォーマット）、関数値（<closure> として出力）をサポート。コレクション内の文字列要素はダブルクォートで囲まれる（例: ["hello", "world"]）
<code>type_of(expr)</code>	expr の型を Type 値として返す。type_of を参照
<code>fail() / fail(message)</code>	現在のテストを失敗としてマークする（ry test モードでのみ使用可能）

Option

関数	説明
<code>Some(expr)</code>	Option 型の値ありバリエーションを構築

Result / Error

関数	説明
<code>Ok(value)</code>	Result<T, Error> の成功バリエーションを構築
<code>Err(error)</code>	Result<T, Error> のエラーバリエーションを構築
<code>Error(message)</code>	メッセージ付きの Error 値を作成
<code>Error(message, code)</code>	メッセージとエラーコード付きの Error 値を作成
<code>result.and_then(closure)</code>	Ok の場合、closure（Result<U, E> を返す）を呼び出す。Err の場合はエラーをそのまま伝播
<code>result.map(closure)</code>	Ok の場合、closure を値に適用し結果を Ok でラップ。Err の場合はエラーをそのまま伝播

チェック付き演算

関数	説明
<code>checked_add(a, b)</code>	オーバーフローなしなら Ok(a + b)、そうでなければ Err(Error("arithmetic overflow"))
<code>checked_sub(a, b)</code>	オーバーフローなしなら Ok(a - b)、そうでなければ Err(Error("arithmetic overflow"))
<code>checked_mul(a, b)</code>	オーバーフローなしなら Ok(a * b)、そうでなければ Err(Error("arithmetic overflow"))
<code>saturating_add(a, b)</code>	a + b を返す。オーバーフロー時は int 範囲にクランプ

関数	説明
<code>saturating_sub(a, b)</code>	$a - b$ を返す。オーバーフロー時は <code>int</code> 範囲にクランプ
<code>saturating_mul(a, b)</code>	$a * b$ を返す。オーバーフロー時は <code>int</code> 範囲にクランプ
<code>wrapping_add(a, b)</code>	オーバーフロー時にラッピングする $a + b$ を返す
<code>wrapping_sub(a, b)</code>	オーバーフロー時にラッピングする $a - b$ を返す
<code>wrapping_mul(a, b)</code>	オーバーフロー時にラッピングする $a * b$ を返す

コレクション操作

関数	説明
<code>has_key(map, key)</code>	マップにキーが存在するかを返す
<code>add(set, value)</code>	セットに要素を追加（重複は無視）
<code>remove(set, value)</code>	セットから要素を削除
<code>append(list, value) / append!(list, value)</code>	リストの末尾に要素を追加（ミューテーション操作）
<code>appended(list, value)</code>	要素を末尾に追加した新しいリストを返す（非破壊）
<code>pop(list)</code>	リストの末尾の要素を削除して <code>Option<T></code> として返す
<code>reverse(list)</code>	逆順の新しいリストを返す（文字列にも対応）
<code>reverse!(list)</code>	リストをその場で逆順にする（ミューテーション操作）
<code>slice(list, start, end)</code>	<code>start</code> から <code>end</code> までの新しい部分リストを返す
<code>take(list, count)</code>	先頭 <code>count</code> 要素の新しいリストを返す
<code>tap(list, function)</code>	各要素に <code>function</code> を呼び出し副作用を実行し、元のリストを返す
<code>filter(list, pred)</code>	述語を満たす要素だけの新しいリストを返す
<code>map(list, function)</code>	各要素を変換した新しいリストを返す
<code>sort(list) / sort(list, comp)</code>	ソート済みの新しいリストを返す（デフォルト昇順）
<code>sort!(list) / sort!(list, comp)</code>	リストをその場でソートする（ミューテーション操作）
<code>insert(list, i, val)</code>	インデックス <code>i</code> に要素を挿入
<code>remove_at(list, i)</code>	インデックス <code>i</code> の要素を削除して返す
<code>items(map)</code>	(キー, 値) タプルのリストを返す
<code>remove(map, key)</code>	指定したキーのエントリを削除
<code>get(map, key)</code>	キーの値を <code>Option<V></code> として返す
<code>get(map, key, default)</code>	キーの値を返す（存在しない場合はデフォルト値）
<code>union(set, set)</code>	2つのセットの和集合を返す
<code>intersection(set, set)</code>	2つのセットの積集合を返す
<code>difference(set, set)</code>	2つのセットの差集合を返す
<code>symmetric_difference(set, set)</code>	2つのセットの対称差を返す
<code>is_subset(set, set)</code>	最初のセットが2番目の部分集合かを返す
<code>is_superset(set, set)</code>	最初のセットが2番目の上位集合かを返す
<code>first(list)</code>	最初の要素を <code>Option<T></code> として返す。空なら <code>None</code>
<code>last(list)</code>	最後の要素を <code>Option<T></code> として返す。空なら <code>None</code>
<code>remove(list, value)</code>	リストから値の最初の出現を削除
<code>is_empty(list / map / set / str)</code>	コレクションまたは文字列が空かを返す
<code>distinct(list)</code>	重複を排除した新しいリストを返す
<code>flatten(list)</code>	ネストされたリストをフラット化した新しいリストを返す
<code>reduce(list, fn)</code>	リデューサ関数を使ってリストを単一の値に畳み込む
<code>fold(list, init, fn)</code>	初期アキュムレータ値を使ってリストを畳み込む
<code>any(list, pred)</code>	述語にマッチする要素が1つでもあれば <code>true</code> を返す
<code>all(list, pred)</code>	すべての要素が述語にマッチすれば <code>true</code> を返す

関数	説明
<code>sum(list)</code>	全要素の合計を返す
<code>min(list)</code>	最小の要素を返す
<code>max(list)</code>	最大の要素を返す
<code>enumerate(list)</code>	(インデックス, 値) タプルのリストを返す。str も受け付け、UTF-8 コードポイントごとに (int, str) を生成する
<code>zip(list1, list2)</code>	2つのリストの要素をペアにした (a, b) タプルのリストを返す。どちらか (あるいは両方) の引数に str を渡すこともできる
<code>keys(map)</code>	すべてのキーを List<K> として返す
<code>values(map)</code>	すべての値を List<V> として返す
<code>merge(map1, map2)</code>	両方のマップのエントリを含む新しいマップを返す

イテレータ

関数	説明
<code>iter(collection)</code>	List、Set、Map から遅延イテレータを作成
<code>next(iter)</code>	次の要素を Option<T> として返す。使い切った場合は None
<code>to_list(iter)</code>	イテレータの残りの要素をすべて List<T> に収集
<code>filter(iter, pred)</code>	述語にマッチする要素のみを返す遅延イテレータを返す
<code>map(iter, function)</code>	各要素を変換する遅延イテレータを返す
<code>take(iter, count)</code>	最大 count 要素を返す遅延イテレータを返す

文字列操作

関数	説明
<code>contains(string, substring)</code>	部分文字列が含まれるか
<code>starts_with(string, prefix)</code>	接頭辞で始まるか
<code>ends_with(string, suffix)</code>	接尾辞で終わるか
<code>find(string, substring)</code>	部分文字列の文字位置 (Option<int>)
<code>byte_len(string)</code>	文字列のバイト長を返す
<code>substring(string, start, end)</code>	部分文字列を取得
<code>char_at(string, i)</code>	指定位置の文字を取得
<code>replace(string, old, new)</code>	部分文字列を全置換
<code>to_upper(string) / to_lower(string)</code>	大文字・小文字変換
<code>trim(string) / trim_start(string) / trim_end(string)</code>	空白除去
<code>repeat(string, count)</code>	文字列を n 回繰り返す
<code>reverse(string)</code>	文字列を逆順にする
<code>split(string, delimiter)</code>	文字列を分割してリストを返す
<code>join(list, sep)</code>	リストの文字列をセパレータで結合
<code>to_int(s) / to_float(s) / to_str(v)</code>	型変換 (to_int と to_float は Result<T, Error> を返す)

-> 詳細は[文字列操作関数リファレンス](#)を参照

print

シグネチャ: `print()` / `print(expr1, expr2, ...)`

1 つ以上の値をスペース区切りで標準出力に表示します。末尾に改行が付きます。引数なしで呼び出すと改行のみを出力します。

型	出力形式
<code>int</code>	<code>%ld</code>
<code>float</code>	<code>%g</code> 、整数値の場合は末尾に <code>.0</code> を付加 (例: <code>3.0</code> , <code>0.0</code>)
<code>bool</code>	<code>true</code> / <code>false</code>
<code>str</code>	<code>%s</code>
<code>Result (Ok)</code>	<code>Ok(value)</code>
<code>Result (Err)</code>	<code>Err(value)</code>
<code>Option (Some)</code>	<code>Some(値)</code>
<code>Option (None)</code>	<code>None</code>
<code>list</code>	<code>[要素 1, 要素 2, ...]</code>
<code>map</code>	<code>{キー 1: 値 1, キー 2: 値 2, ...}</code>
<code>set</code>	<code>{要素 1, 要素 2, ...}</code>
<code>tuple</code>	<code>(要素 1, 要素 2, ...)</code>
<code>enum</code>	バリエーション名 (例: <code>Red</code>)
<code>record</code>	<code>RecordName(field: val, ...)</code>
関数値 (クロージャ / ラムダ)	<code><closure></code>
ユニオン	アクティブなバリエーションの型としてフォーマット

整数値の `float` は常に末尾に `.0` を付けて出力されるため、`int` と視覚的に区別できます。ネストしたコレクション (例: `Map<str, List<int>>`) は内側の要素型のフォーマッタを使って再帰的にフォーマットされます。ユニオンバリエーションの中身が `List`、`Map`、`Set` の場合はそのコレクションとしてフォーマットされ、中身が関数値のバリエーションは `<closure>` として出力されます。

```
print(42)           # 42
print(3.14)         # 3.14
print(3.0)          # 3.0      (整数値の float は .0 を保持)
print(0.0)          # 0.0
print(true)         # true
print("hello")      # hello
print(Ok(42))        # Ok(42)
print(Err(Error("fail"))) # Err(Error: fail (code: 0))
print(Some(1))       # Some(1)
print(None)          # None
print([1, 2, 3])     # [1, 2, 3]
print({"a": 1})       # {"a": 1}
print({1, 2, 3})     # {1, 2, 3}
print((1, "hello"))  # (1, "hello")

# ネストしたコレクション
m: Map<str, List<int>> = {"a": [1, 2, 3]}
print(m)              # {"a": [1, 2, 3]}

# コレクション型のユニオンバリエーション
x: int | List<int> = [1, 2, 3]
print(x)              # [1, 2, 3]

# 関数値
```

```
f = (x: int) => x * 2
print(f)           # <closure>

# 複数引数 (スペース区切り)
print(1, 2, 3)     # 1 2 3
print("hello", "world") # hello world
print(1, "hello", true) # 1 hello true
print()            # (空行)
```

Some

シグネチャ: `Some(expr) -> Option<T>`

`Option` 型の値ありバリエーションを構築します。

```
x: Option<int> = Some(42)
print(x)       # Some(42)
```

length

シグネチャ: `length(x: List<T> | Map<K, V> | Set<T> | str) -> int`

リスト・マップ・セットの要素数、または文字列の UTF-8 文字数を返します。バイト長が必要な場合は `byte_len()` を使用してください。

```
print(length([1, 2, 3])) # 3
print(length({"a": 1, "b": 2})) # 2
print(length({1, 2, 3})) # 3
print(length("hello")) # 5
print(length("あいう")) # 3 (UTF-8 文字数)
```

has_key

シグネチャ: `has_key(m: Map<K, V>, key: K) -> bool`

マップに指定したキーが存在するかを返します。UFCS 記法も使用可能です。

```
m = {"a": 1, "b": 2}
print(has_key(m, "a")) # true
print(m.has_key("z")) # false (UFCS)
```

add

シグネチャ: `add(s: Set<T>, value: T)`

セットに要素を追加します。既に存在する要素を追加した場合は何もしません。UFCS 記法も使用可能です。

```
s = {1, 2, 3}
s.add(4) # UFCS
add(s, 5) # 通常の呼び出し
s.add(1) # 既に存在するため無視
print(length(s)) # 5
```

remove

シグネチャ: `remove(s: Set<T>, value: T)`

セットから要素を削除します。UFCS 記法も使用可能です。

```
s = {1, 2, 3}
s.remove(2)      # UFCS
print(2 in s)    # false
```

range

シグネチャ: `range(n: int) -> List<int>` / `range(start: int, end: int) -> List<int>` / `range(start: int, end: int, step: int) -> List<int>`

整数のリストを生成します。

形式	生成される値
<code>range(n)</code>	<code>[0, 1, ..., n-1]</code>
<code>range(start, end)</code>	<code>[start, start+1, ..., end-1]</code>
<code>range(start, end, step)</code>	<code>[start, start+step, start+2*step, ...]</code> (end は含まない)

- `step > 0` の場合、`start` から昇順に `end` に向かって生成します。
- `step < 0` の場合、`start` から降順に `end` に向かって生成します。
- `step == 0` の場合、ランタイムエラーになります。
- 範囲が空の場合 (例: `range(0, 10, -1)`)、空リストを返します。

```
print(range(3))      # [0, 1, 2]
print(range(2, 5))   # [2, 3, 4]
print(range(0, 10, 2)) # [0, 2, 4, 6, 8]
print(range(10, 0, -3)) # [10, 7, 4, 1]

for i in range(3):
    print(i)
# 0
# 1
# 2
```

exit

シグネチャ: `exit(code: int)`

指定した終了コードでプロセスを即座に終了します。`exit()` 以降の文は到達不能ブロックにコンパイルされ、LLVM の最適化で除去されるため決して実行されません:

```
exit(0)      # 正常終了
exit(1)      # エラー終了

print("a")
exit(0)
print("b")    # 決して出力されない -- exit の後は到達不能
```

同じ扱いは `return`、`break`、`continue` にも適用されます。制御フローを分岐させるこれらの文の後のコードは暗黙的に除去されます。

arguments

シグネチャ: `arguments() -> List<str>`

スクリプトに渡されたコマンドライン引数を文字列のリストとして返します。インタプリタ名やスクリプトファイル名は含まれません。スクリプトパスの後の引数のみです。

```
# 実行: ry script.ry hello world
a = arguments()
print(length(a))    # 2
print(a[0])         # hello
print(a[1])         # world

for x in arguments():
    print(x)
```

sleep

シグネチャ: `sleep(duration_ms: int) -> Unit`

指定されたミリ秒間、現在のスレッドの実行を一時停止します。`duration_ms` が 0 以下の場合は即座に返ります。

```
sleep(1000)    # 1秒待機
sleep(0)       # 即座に戻る
```

env

シグネチャ: `env(key: str) -> Option<str>` / `env(key: str, default: str) -> str`

環境変数の値を返します。1 引数の場合は `Option<str>` (設定済みなら `Some(value)`、未設定なら `None`) を返します。2 引数の場合は値が未設定なら `default` を返します。

プロジェクトルート (`package.toml` が存在するディレクトリ) に `.env` ファイルがある場合、起動時に自動的に読み込まれます。既存の環境変数は `.env` の値で上書きされません。

セキュリティ上の注意: `.env` ファイルには通常シークレット (API キー、データベースパスワード、トークン等) が含まれます。`.env` をバージョン管理にコミットしないでください (`.gitignore` 等に追加してください)。その内容は機密情報として扱ってください。

```
# 1 引数: Option<str> を返す
path = env("PATH")
case path:
  Some(v):
    print(v)
  None:
    print("PATH not set")

# 2 引数: デフォルト値付き
port = env("PORT", "8080")
print(port)    # PORT 未設定なら "8080"
```

.env ファイルの書式

```
# コメントは # で始まる
DATABASE_URL=postgres://localhost/mydb
API_KEY="secret-key-123"
EMPTY_VALUE=
QUOTED='single quoted'
```

環境別 .env ファイル

RY_ENV が設定されている場合、Ry は以下の優先順位で環境別の .env ファイルを読み込みます:

- .env.<環境名> を最初に読み込む (例: RY_ENV=dev なら .env.dev)
- .env を次に読み込む (.env.<環境名> で設定済みの値は上書きされない)
- RY_ENV=prod の場合、.env ファイルは一切読み込まない (セキュリティ)
- RY_ENV が未設定の場合、.env のみ読み込む (後方互換)

環境モードの詳細は [RY_ENV](#) を参照してください。

append

シグネチャ: `append(list: List<T>, value: T)`

リストの末尾に要素を追加します。これはミューテーション操作で、リストがその場で変更されます。UFCS 記法も使用可能です。

```
xs = [1, 2]
xs.append(3)
print(xs)    # [1, 2, 3]
```

pop

シグネチャ: `pop(list: List<T>) -> Option<T>`

リストの末尾の要素を削除して `Option<T>` として返します。リストが空の場合は `None` を返します。UFCS 記法も使用可能です。

```
xs = [1, 2, 3]
v = xs.pop()
print(v)      # Some(3)
print(xs)     # [1, 2]
```

reverse (list)

シグネチャ: `reverse(list: List<T>) -> List<T>`

要素を逆順にした新しいリストを返します。元のリストは変更されません。文字列に対しても使用できます ([文字列操作関数リファレンス](#)を参照)。UFCS 記法も使用可能です。

```
xs = [1, 2, 3]
ys = reverse(xs)
print(ys)     # [3, 2, 1]
print(xs)     # [1, 2, 3] (変更なし)
```

slice

シグネチャ: `slice(list: List<T>, start: int, end: int) -> List<T>`

`start` (含む) から `end` (含まない) までの新しい部分リストを返します。インデックスは有効範囲 (0 から `length(list)` まで) にクランプされます。UFCS 記法も使用可能です。

```
xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))    # [2, 3]
print(slice(xs, 0, 100)) # [1, 2, 3, 4, 5] (クランプされる)
```

take

シグネチャ: `take(list: List<T>, count: int) -> List<T>`

先頭 `count` 要素の新しいリストを返します。`count` がリストの長さを超える場合はリスト全体のコピーを返します。`count <= 0` の場合は空リストを返します。元のリストは変更されません。UFCS 記法も使用可能です。

```
xs = [1, 2, 3, 4, 5]
ys = xs.take(3)
print(ys)    # [1, 2, 3]
print(xs.take(10)) # [1, 2, 3, 4, 5] (クランプされる)
print(xs.take(0))  # []
```

tap

シグネチャ: `tap(list: List<T>, function: function(T) -> R) -> List<T>`

各要素に対して関数を呼び出し (戻り値は無視)、元のリストをそのまま返します。メソッドチェーン中のデバッグや副作用の挿入に有用です。UFCS 記法も使用可能です。

```
xs = [1, 2, 3]
ys = xs.tap((x: int) => print(x)).map((x: int) => x * 2)
# 1, 2, 3 を出力し、ys = [2, 4, 6]
```

filter

シグネチャ: `filter(list: List<T>, pred: function(T) -> bool) -> List<T>`

述語が `true` を返す要素のみを含む新しいリストを返します。元のリストは変更されません。UFCS 記法も使用可能です。

```
xs = [1, 2, 3, 4, 5]
ys = xs.filter((x: int) => x > 3)
print(ys)    # [4, 5]
print(xs)    # [1, 2, 3, 4, 5] (変更なし)
```

map

シグネチャ: `map(list: List<T>, function: function(T) -> U) -> List<U>`

各要素を関数で変換した新しいリストを返します。出力の要素型は入力と異なっても構いません。元のリストは変更されません。UFCS 記法も使用可能です。

```
xs = [1, 2, 3]
ys = xs.map((x: Int) => x * 2)
print(ys)    # [2, 4, 6]
```

sort

シグネチャ: `sort(list: List<T>) -> List<T>` / `sort(list: List<T>, comp: function(T, T) -> bool) -> List<T>`

ソート済みの新しいリストを返します。デフォルトは昇順です。カスタム比較関数を指定できます（第一引数が第二引数の前に来るべき場合に `true` を返す）。元のリストは変更されません。ソートは**安定**です（等しい要素の元の順序が保持されます）。UFCS 記法も使用可能です。

```
xs = [3, 1, 2]
print(xs.sort())    # [1, 2, 3]
```

降順ソート

```
desc = xs.sort((a: Int, b: Int) => a > b)
print(desc)    # [3, 2, 1]
```

sort!

シグネチャ: `sort!(list: List<T>) / sort!(list: List<T>, comp: function(T, T) -> bool)`

リストをその場でソートします。ソートアルゴリズムは `sort()` と同じですが、新しいリストを作成する代わりに元のリストを変更します。UFCS 記法も使用可能です。

```
xs = [3, 1, 2]
xs.sort!()
print(xs)    # [1, 2, 3]
```

reverse!

シグネチャ: `reverse!(list: List<T>)`

リストをその場で逆順にします。UFCS 記法も使用可能です。

```
xs = [1, 2, 3]
xs.reverse!()
print(xs)    # [3, 2, 1]
```

appended

シグネチャ: `appended(list: List<T>, value: T) -> List<T>`

要素を末尾に追加した新しいリストを返します。元のリストは変更されません。UFCS 記法も使用可能です。

```
xs = [1, 2]
ys = xs.appended(3)
print(xs)    # [1, 2] (変更なし)
print(ys)    # [1, 2, 3]
```

append!

シグネチャ: `append!(list: List<T>, value: T)`

`append()` のエイリアスです。リストの末尾に要素をその場で追加します。! 命名規約との一貫性のために提供されています。

first

シグネチャ: `first(list: List<T>) -> Option<T>`

リストの最初の要素を `Option<T>` として返します。リストが空の場合は `None` を返します。

```
print(first([10, 20, 30]))    # Some(10)
```

last

シグネチャ: `last(list: List<T>) -> Option<T>`

リストの最後の要素を `Option<T>` として返します。リストが空の場合は `None` を返します。

```
print(last([10, 20, 30]))    # Some(30)
```

get (Map)

シグネチャ: `get(map: Map<K, V>, key: K) -> Option<V>` / `get(map: Map<K, V>, key: K, default: V) -> V`

2 引数形式はキーの値を `Option<V>` として返します。3 引数形式はキーの値またはデフォルト値を返します。

```
m = {"a": 1, "b": 2}
print(get(m, "a"))           # Some(1)
print(get(m, "z"))           # None
print(get(m, "z", 0))        # 0
```

iter

シグネチャ: `iter(collection: List<T> | Set<T>) -> Iterator<T>` / `iter(collection: Map<K, V>) -> Iterator<(K, V)>`

コレクションから遅延イテレータを作成します。イテレータはデータをコピーせず、元のコレクションを参照します。UFCS 記法も使用可能です。

- `List<T>` と `Set<T>` の場合、要素型は `T`。
- `Map<K, V>` の場合、要素型はタプル `(K, V)`。

```
xs = [1, 2, 3]
it = xs.iter()               # Iterator<int>
ys = it.to_list()            # [1, 2, 3]
```

```
m = {"a": 1, "b": 2}
for k, v in m.iter():        # Iterator<(str, int)>
    print(k)
```

next

シグネチャ: `next(iter: Iterator<T>) -> Option<T>`

イテレータから次の要素を `Option<T>` として返します。イテレータが使い切られた場合は `None` を返します。呼び出しごとにイテレータの内部状態が進みます。UFCS 記法も使用可能です。

```
it = [10, 20].iter()
print(it.next())    # Some(10)
print(it.next())    # Some(20)
print(it.next())    # None
```

to_list

シグネチャ: `to_list(iter: Iterator<T>) -> List<T>`

イテレータの残りの要素をすべて新しいリストに収集します。UFCS 記法も使用可能です。

```
xs = [1, 2, 3, 4, 5]
ys = xs.iter().filter((x: int) => x > 2).to_list()
print(ys)           # [3, 4, 5]
```

type_of

シグネチャ: `type_of(expr: T) -> Type`

式の型を `Type` 値として返します。型定義（プリミティブ、コレクション、record、enum、`Option`、`Result`、関数等）ごとに、コンパイル時に一意な `identity` が与えられるため、`type_of` の結果を `==` で比較して2つの式が同じ型かどうかを判定できます。

- 引数は副作用のために評価されますが、静的型だけが使用されます。
- `Type` 値を `print` や `to_str` で出力すると、人間が読める名前になります（例: `"int"`, `"List"`, `"Point"`）。
- 同じ正規型を持つ2つの式は等しい `Type` 値を返します。異なる record（または、たまたま同名の record と enum）は常に区別可能です。
- 裸の `none` リテラルは `"None"` として報告されます。型が付いた `Option<T>` 値 (`Some(...)`) で構築されたものでも、`none` から代入されたものでも `"Option"` として報告されます。

record Point:

```
  x: int
  y: int
```

enum Color:

```
  Red
  Green
  Blue
```

```
print(to_str(type_of(42)))      # int
print(to_str(type_of(3.14)))    # float
print(to_str(type_of("hello"))) # str
print(to_str(type_of([1, 2, 3]))) # List
print(to_str(type_of({"a": 1}))) # Map
print(to_str(type_of({1, 2})))  # Set
```

```
p = Point(1, 2)
print(to_str(type_of(p)))      # Point
```

```

c = Color::Red
print(to_str(type_of(c)))          # Color

# identity 比較
print(type_of(42) == type_of(100)) # true
print(type_of(42) == type_of(3.14)) # false
print(type_of(p) != type_of(c))    # true

# 低レベルの数値型は `int` とは区別される
x: i32 = 1
print(to_str(type_of(x)))          # i32
print(type_of(x) == type_of(42))  # false

# type_of は反射的: Type 値の型は Type
print(to_str(type_of(type_of(42)))) # Type

```

`type_of` が返す型カテゴリ

入力	<code>to_str(type_of(...))</code>
42	int
3.14	float
true / false	bool
"hello"	str
[1, 2]	List
{"a": 1}	Map
{1, 2}	Set
x: i32 = 1	i32 (u8, i16, ..., f32 も同様)
record 値	record 名 (例: Point)
enum 値	enum 名 (例: Color)
none リテラル	None
Some(1)	Option
x: Option<int> = none	Option
Ok(1) / Err(e)	Result
ラムダ / クロージャ	function
<code>type_of(x)</code>	Type

裸の `none` リテラルは型が付いた `Option` 値と区別するため "None" として報告されます。`Option<T>` コンテナ (`Some(...)` で構築されたものでも、`Option<T>` 型の変数に `none` を代入したものでも) は "Option" として報告されます。

[English](#) | [日本語](#) | [繁體中文](#)

コレクションリファレンス (タプル・リスト・マップ・セット)

タプル

概要

固定長・異種型の値の組み合わせ。LLVM literal StructType として実装されたスタック上の値型です。

構文

```

t = (42,)          # 単一要素タプル (末尾カンマ必須)
t = (1, 3.14)

```

```
t: (int, float) = (1, 3.14)
```

型アノテーション

```
single: (int,) = (42,) # 単一要素には末尾カンマ必須
pair: (int, str) = (42, "hello")
triple: (int, float, bool) = (1, 2.0, true)
```

比較

タプルは要素ごとの比較による `==` と `!=` をサポートします。

```
print((1, 2) == (1, 2)) # true
print((1, 2) != (3, 4)) # true
print(("a", 1) == ("b", 1)) # false
```

要素アクセス

`.0`, `.1`, ... の数値インデックスでアクセスします。

```
t = (10, 3.14)
print(t.0) # 10
print(t.1) # 3.14
```

関数戻り値

```
function swap(a: int, b: int) -> (int, int):
    return (b, a)
```

```
result = swap(1, 2)
print(result.0) # 2
print(result.1) # 1
```

制約とエラー

制約	詳細
範囲外インデックス 比較演算子	コンパイルエラー <code>==</code> と <code>!=</code> のみサポート。 <code><</code> 、 <code><=</code> 、 <code>></code> 、 <code>>=</code> は未対応

リスト

概要

同一型の可変長シーケンス。ヒープ上に確保されます。

構文

```
xs = [1, 2, 3]
xs: List<int> = [1, 2, 3]
```

空リスト

空リストは要素型を決定するために型アノテーションが必要です:

```
xs: List<int> = []
xs: List<str> = []
```

連結

リストは + と += で連結できます:

```
a = [1, 2, 3]
b = [4, 5, 6]
c = a + b      # [1, 2, 3, 4, 5, 6]
a += [7, 8]    # a は [1, 2, 3, 7, 8] になる
```

両方のオペランドは同じ要素型でなければなりません。

対応する要素型

int, float, bool, str

等価性

リストは == と != をサポートします。2 つのリストは、同じ長さで、対応するすべての要素が等しい場合に等しくなります。

```
[1, 2, 3] == [1, 2, 3]    # true
[1, 2, 3] != [1, 2, 4]   # true
[1, 2]    != [1, 2, 3]   # true (長さが違う)
```

インデックスアクセス

```
xs = [1, 2, 3]
print(xs[0])    # 1
print(xs[2])    # 3
```

負のインデックスは末尾から数えます (Python スタイル) :

```
xs = [10, 20, 30]
print(xs[-1])   # 30 (最後の要素)
print(xs[-2])   # 20
print(xs[-3])   # 10
```

範囲外アクセス (リストの長さを超える負のインデックスを含む) はランタイムエラーになります。

インデックス代入

```
xs = [1, 2, 3]
xs[0] = 99
print(xs[0])    # 99
xs[-1] = 42     # 最後の要素に代入
print(xs[2])    # 42
```

インデックスとフィールド代入のチェーン

インデックス代入とフィールド代入は合成可能です。= / += / -= / *= / /= / //= / %= / **= / &= / |= / ^= / <<= / >>= の左辺は、可変変数を根とする任意の後置チェーンを使えます。

record Point:

```
    x: int
    y: int
```

```
pts = [Point(1, 2), Point(3, 4)]
pts[0].x = 99          # リスト中の record フィールド更新
pts[0].x += 1
print(pts[0].x)        # 100
```

```
grid = [[1, 2], [3, 4]]
grid[0][1] = 99      # ネストしたリスト要素
print(grid[0])       # [1, 99]
```

```
m: Map<str, Map<str, int>> = {"a": {"x": 1}}
m["a"]["x"] = 42      # ネストしたマップ要素
```

複合代入は各インデックスをちょうど 1 回だけ評価するため、`xs[f()] += 1` は `f()` を 1 回しか呼びません。存在しないマップキーに対する複合代入はランタイムエラーになります。累積したい場合は事前にキーを挿入してください:

```
m = {"a": 1}
m["a"] += 10          # OK → {"a": 11}
m["b"] += 10          # ランタイムエラー: 存在しないマップキーへの複合代入
```

ネストした書き込みは隔離される: `grid[i][j] = v` や `r.items[i] = v` (list を持つ record フィールド経由) のようなチェーン書き込みでは、ミューテーションが適用される前に LHS パス上の参照カウント > 1 の各レベルが私有化されるため、外側コンテナ-あるいはパス上のいずれかの内側コンテナ-のエイリアスは書き込み前の状態を観測します。詳細は下の [Path Copy-on-Write](#) を参照してください。

length

```
xs = [1, 2, 3]
print(length(xs))    # 3
```

print

```
xs = [1, 2, 3]
print(xs)            # [1, 2, 3]
```

for 走査

```
xs = [10, 20, 30]
for x in xs:
    print(x)
# 10
# 20
# 30
```

append

リストの末尾に要素を追加します。これはミューテーション操作です。

```
xs = [1, 2]
xs.append(3)
print(xs)            # [1, 2, 3]
```

pop

リストの末尾の要素を削除して返します。空のリストの場合は `None` を返します。

```
xs = [1, 2, 3]
v = xs.pop()
print(v)              # Some(3)
print(xs)              # [1, 2]
```

reverse

要素を逆順にした新しいリストを返します。元のリストは変更されません。文字列に対しても使用できます。

```
xs = [1, 2, 3]
print(reverse(xs))    # [3, 2, 1]
print(xs)             # [1, 2, 3] (変更なし)
```

slice

start (含む) から end (含まない) までの新しい部分リストを返します。インデックスは有効範囲にクランプされます。

```
xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))    # [2, 3]
print(slice(xs, 0, 100))  # [1, 2, 3, 4, 5] (クランプされる)
```

take

先頭 count 要素の新しいリストを返します。count がリストの長さを超える場合はリスト全体のコピーを返します。count <= 0 の場合は空リストを返します。元のリストは変更されません。

```
xs = [1, 2, 3, 4, 5]
ys = xs.take(3)
print(ys)    # [1, 2, 3]
print(xs.take(10))    # [1, 2, 3, 4, 5] (クランプされる)
print(xs.take(0))     # []
```

tap

各要素に対して関数を呼び出し (戻り値は無視)、元のリストをそのまま返します。メソッドチェーン中のデバッグや副作用の挿入に有用です。

```
xs = [1, 2, 3]
ys = xs.tap((x: int) => print(x)).map((x: int) => x * 2)
# 1, 2, 3 を出力し、ys = [2, 4, 6]
```

filter

述語を満たす要素だけを含む新しいリストを返します。元のリストは変更されません。

```
xs = [1, 2, 3, 4, 5]
ys = xs.filter((x: int) => x > 3)
print(ys)    # [4, 5]
```

map

各要素を関数で変換した新しいリストを返します。出力の要素型は入力と異なっても構いません。元のリストは変更されません。

```
xs = [1, 2, 3]
ys = xs.map((x: int) => x * 2)
print(ys)    # [2, 4, 6]
```

sort

ソート済みの新しいリストを返します。デフォルトは昇順です。カスタム比較関数を指定できます。元のリストは変更されません。ソートは安定です (等しい要素の元の順序が保持されます)。内部的に TimSort を使用しています。

```
xs = [3, 1, 2]
print(xs.sort())    # [1, 2, 3]
```

降順ソート

```
desc = xs.sort((a: int, b: int) => a > b)
print(desc)        # [3, 2, 1]
```

filter, map, sort のチェーン

これらの関数は新しいリストを返すため、UFCS で連鎖できます。

```
xs = [5, 3, 1, 4, 2]
result = xs.filter((x: int) => x > 1).map((x: int) => x * 10).sort()
print(result)      # [20, 30, 40, 50]
```

reduce

アキュムレータ関数を使ってリストを単一の値に畳み込みます。最初の要素を初期値として使用します。

```
xs = [1, 2, 3, 4, 5]
total = reduce(xs, (a: int, b: int) => a + b)
print(total)       # 15
```

fold

明示的な初期値とアキュムレータ関数を使ってリストを単一の値に畳み込みます。

```
xs = [1, 2, 3, 4, 5]
total = fold(xs, 0, (a: int, b: int) => a + b)
print(total)       # 15
```

any

述語を満たす要素が1つ以上あれば true を返します。

```
xs = [1, 2, 3, 4, 5]
print(any(xs, (x: int) => x > 4))    # true
print(any(xs, (x: int) => x > 9))    # false
```

all

すべての要素が述語を満たす場合に true を返します。

```
xs = [2, 4, 6]
print(all(xs, (x: int) => x > 0))    # true
print(all(xs, (x: int) => x > 3))    # false
```

sum

全要素の合計を返します。

```
xs = [1, 2, 3, 4, 5]
print(sum(xs))    # 15
```

min

最小の要素を返します。

```
xs = [3, 1, 4, 1, 5]
print(min(xs))    # 1
```


max

最大の要素を返します。

```
xs = [3, 1, 4, 1, 5]
print(max(xs))    # 5
```

first

最初の要素を返します。空のリストの場合は `None` を返します。

```
xs = [10, 20, 30]
print(first(xs))   # Some(10)
```

last

最後の要素を返します。空のリストの場合は `None` を返します。

```
xs = [10, 20, 30]
print(last(xs))    # Some(30)
```

is_empty

コンテナが空であれば `true` を返します。リスト・マップ・セット・文字列を受け付けます。

```
xs = [1, 2, 3]
print(is_empty(xs))           # false
print(is_empty([]))           # true (実際には型注釈が必要)
print(is_empty(""))           # true (str サポート, #831)
print(is_empty("hello"))      # false
```

enumerate

(インデックス, 要素) のタプルのリストを返します。

```
xs = [10, 20, 30]
pairs = enumerate(xs)
# pairs = [(0, 10), (1, 20), (2, 30)]

# for ループでのタプル分解
for i, x in enumerate(xs):
    print(f"{i}: {x}")    # 0: 10, 1: 20, 2: 30
```

zip

2つのリストを (要素1, 要素2) のタプルのリストに結合します。結果の長さは短い方のリストと同じになります。

```
xs = [1, 2, 3]
ys = ["a", "b", "c"]
pairs = zip(xs, ys)
# pairs = [(1, "a"), (2, "b"), (3, "c")]

# for ループでのタプル分解
for a, b in zip(xs, ys):
    print(f"{a}: {b}")    # 1: a, 2: b, 3: c
```

insert

指定したインデックスに要素を挿入します。そのインデックス以降の要素は右にシフトされます。

```
xs = [1, 2, 3]
insert(xs, 1, 99)
print(xs)    # [1, 99, 2, 3]
```

remove_at

指定したインデックスの要素を削除して返します。そのインデックス以降の要素は左にシフトされます。

```
xs = [1, 2, 3, 4]
v = remove_at(xs, 1)
print(v)      # 2
print(xs)     # [1, 3, 4]
```

remove

リストから指定した値の最初の出現を削除します。値が見つからない場合は何もしません。破壊的操作です。

```
xs = [1, 2, 3, 2, 4]
remove(xs, 2)
print(xs)    # [1, 3, 2, 4]
```

distinct

重複を排除した新しいリストを返します。元の順序は保持されます（最初の出現を残します）。元のリストは変更されません。

```
xs = [1, 2, 3, 2, 1, 4]
print(distinct(xs))    # [1, 2, 3, 4]
print(xs)              # [1, 2, 3, 2, 1, 4] (変更なし)
```

flatten

ネストされたリスト（リストのリスト）を1段階フラット化します。新しいリストを返します。元のリストは変更されません。

```
xs = [[1, 2], [3, 4]]
print(flatten(xs))    # [1, 2, 3, 4]
print(xs)             # [[1, 2], [3, 4]] (変更なし)
```

その場変更版

一部のリスト操作には2つの形式があります。新しいリストを返す非変更バージョンと、名前が! で終わるその場変更バージョンです。レシーバを変更する意図を呼び出し側で明示したいときは! 形式を、元を保存したいときは非変更形式を使ってください。

その場 (!)	非変更の対応	差異
<code>append!(xs, v) / append(xs, v)</code>	<code>appended(xs, v)</code>	<code>append!</code> / <code>append</code> はその場で変更し <code>Unit</code> を返す。 <code>appended</code> は新しいリストを返す
<code>sort!(xs) / sort!(xs, cmp)</code>	<code>sort(xs) / sort(xs, cmp)</code>	<code>sort!</code> は <code>xs</code> をその場で変更する。 <code>sort</code> は新しいソート済みリストを返す

その場 (!)	非変更の対応	差異
<code>reverse!(xs)</code>	<code>reverse(xs)</code>	<code>reverse!</code> は <code>xs</code> をその場で変更する。 <code>reverse</code> は新しい反転リストを返す

すべての ! バージョンは Copy-on-Write に参加します。xs が共有されている場合 (参照カウント > 1)、外側バッファはその場での変更前に 1 度だけクローンされるため、エイリアスは影響を受けません (Copy-on-Write (CoW) セマンティクス を参照)。

```
xs = [3, 1, 2]
sort!(xs)
print(xs)          # [1, 2, 3]
```

```
ys = [1, 2, 3]
reverse!(ys)
print(ys)          # [3, 2, 1]
```

```
zs = [1, 2]
append!(zs, 3)
print(zs)          # [1, 2, 3]
```

非変更版: `appended` は新しいリストを返す

```
a = [1, 2]
b = appended(a, 3)
print(a)           # [1, 2] (変更なし)
print(b)           # [1, 2, 3]
```

どちらを使うべきか:

- 元が保存されることを読者が理解できたほうが分かりやすい場合 (例: 関数型パイプライン、あるいは元がその後も必要なとき) は `sort`, `reverse`, `appended` を優先してください。
- 元を本当に置き換える場合は、中間コピーの割り当てを避け、呼び出し側で変更を明示するために `sort!`, `reverse!`, `append!` を優先してください。

操作の計算量

操作	計算量
<code>xs[i]</code> インデックスアクセス	$O(1)$
<code>append</code> / <code>append!</code>	$O(1)$ (均償)
<code>pop</code>	$O(1)$
<code>first</code> , <code>last</code>	$O(1)$
<code>insert</code> , <code>remove_at</code>	$O(n)$
<code>sort</code> / <code>sort!</code>	$O(n \log n)$
<code>take</code>	$O(n)$
<code>tap</code>	$O(n)$
<code>filter</code> , <code>map</code> , <code>reduce</code> , <code>fold</code>	$O(n)$
<code>reverse</code> / <code>reverse!</code>	$O(n)$
<code>distinct</code>	$O(n)$
<code>length</code>	$O(1)$

制約とエラー

制約	詳細
全要素は同一型	異なる型が混在するとコンパイルエラー
空リスト []	型アノテーションが必要 (例: <code>xs: List<int> = []</code>)
範囲外アクセス	ランタイムエラー (<code>exit(1)</code>)

Copy-on-Write (CoW) セマンティクス

すべてのコレクション型 (List, Map, Set) は ARC 管理下で **Copy-on-Write** セマンティクスを使用します。これは以下を意味します:

- **代入はデータを共有する:** `b = a` はコレクションをコピーしません。両方の変数が同じデータを参照します。参照カウントがインクリメントされます。
- **変更時にパスをたどってコピーが発生する:** 共有コレクションが変更される場合、LHS パス上で参照カウント > 1 のすべてのレベルが、ミューテーション適用前にクローンされます。トップレベルの書き込み (`b.append(...)`)、`b` が共有された状態の `b[i] = v` は、最外コンテナだけをクローンします。チェーン書き込み (`b[i][j] = v`, `r.items[i] = v`, `m[k1][k2] = v`) は根から下へたどり、共有状態のままの各中間コンテナをクローンするため、外側コンテナあるいはパス上のいずれかの内側コンテナのエイリアスは変更から隔離されます。
- **単一所有者はその場で変更する:** コレクション (またはパス上のどこかのホップ) の参照が 1 つだけの場合 (`strong_count == 1`)、そのレベルの CoW チェックはクローンをスキップし、コピーなしでその場で変更します。

```
a = [1, 2, 3]      # strong_count = 1
b = a             # strong_count = 2 (共有)
append(b, 4)      # strong_count > 1 → 外側バッファをコピーしてから変更
                  # a = [1, 2, 3] (strong_count = 1)
                  # b = [1, 2, 3, 4] (strong_count = 1, 新しい割り当て)

c = [10, 20]      # strong_count = 1
append(c, 30)     # strong_count == 1 → その場で変更 (コピーなし)
```

Path Copy-on-Write

Path CoW はネストしたコレクションを通じたチェーン書き込みを扱います。書き込み側は LHS の根となる変数 (または record フィールド) から葉の書き込み地点まで下り、参照カウントが 1 より大きい各レベルを私有化します。これにより、外側コンテナ、あるいは書き込みパス上のいずれかの内側コンテナを共有するエイリアス間で厳密なエイリアス隔離が得られます。

```
a = [[1, 2], [3, 4]]
b = a             # 外側リストは共有: strong_count = 2
b[0][0] = 99      # たどる手順: 外側をクローン (a と b はそれぞれ
                  # 自分の外側を持つ)、b の inner[0] をクローン
                  # (a の inner[0] は [1, 2] のまま)、クローンに 99 を書き込む

# a = [[1, 2], [3, 4]]
# b = [[99, 2], [3, 4]]
```

ARC 管理のコレクション値を持つ record フィールドも、インデックスホップと同じように path CoW に参加します。record 間の代入 (`r2 = r1`) は各 ARC フィールドをリテインするため、その後の `r2.items[i] = v` はフィールドスロットで参照カウント > 1 を観測し、変更前にクローンします:

```
record Box:
  items: List<int>

r1 = Box([1, 2, 3])
```

```

r2 = r1
r2.items[0] = 99
# r1.items[0] == 1
# r2.items[0] == 99

```

path-CoW チェーンの根としてサポートされないもの: メソッド呼び出しの lvalue (`f().x[i] = v`)、および record フィールドをたどる中でインデックスホップが混在するチェーン (`rec.arr[0].items[i] = v`) です。これらの形はコンパイル時エラーになります。中間値をまずローカル変数に代入してください。

CoW が発生する操作

型	変更操作
List	<code>append()</code> 、 <code>pop()</code> 、 <code>insert()</code> 、 <code>remove()</code> 、 <code>remove_at()</code> 、 <code>sort!()</code> 、 <code>reverse!()</code> 、インデックス代入 (<code>xs[i] = val</code>)
Map	<code>remove()</code> 、インデックス代入 (<code>m[key] = val</code>)
Set	<code>add()</code> 、 <code>remove()</code>

非変更操作 (`appended`、`slice`、`take`、`filter`、`map`、`sort`、`reverse`、`get`、`items` 等) は新しいコレクションを作成し、CoW は発生しません。

マップ

概要

キーと値の対応表。ヒープ上に確保されます。

構文

```

m = {"a": 1, "b": 2}
m: Map<str, int> = {"a": 1, "b": 2}

```

等価性

マップは `==` と `!=` をサポートします。2 つのマップは、エントリ数が同じで、片方のマップに含まれるすべてのキーと値のペアが、もう片方にも同じ値で存在する場合に等しくなります。

```

{"a": 1, "b": 2} == {"a": 1, "b": 2}    # true
{"a": 1}          != {"a": 2}           # true (値が違う)
{"a": 1}          != {"b": 1}           # true (キーが違う)

```

キーアクセス

```

m = {"a": 1, "b": 2}
print(m["a"])    # 1

```

挿入・更新

```

m = {"a": 1}
m["b"] = 2    # 新規追加
m["a"] = 99   # 更新

```

length

```

m = {"a": 1, "b": 2, "c": 3}
print(length(m))    # 3

```

print

```
m = {"a": 1, "b": 2}
print(m)    # {"a": 1, "b": 2}
```

has_key

```
m = {"a": 1, "b": 2}
print(m.has_key("a"))    # true
print(m.has_key("z"))    # false
```

keys

マップの全キーのリストを返します。

```
m = {"a": 1, "b": 2, "c": 3}
print(keys(m))    # ["a", "b", "c"]
```

values

マップの全値のリストを返します。

```
m = {"a": 1, "b": 2, "c": 3}
print(values(m))    # [1, 2, 3]
```

items

マップの全エントリの（キー， 値）タプルのリストを返します。

```
m = {"a": 1, "b": 2}
pairs = items(m)
# pairs = [("a", 1), ("b", 2)]
```

remove (マップ)

指定したキーのエントリをマップから削除します。キーが存在しない場合は何もしません。

```
m = {"a": 1, "b": 2}
remove(m, "a")
print(m)    # {"b": 2}
```

get

指定したキーの値を返します。キーが存在しない場合はデフォルト値を返します。

```
m = {"a": 1, "b": 2}
print(get(m, "a", 0))    # 1
print(get(m, "z", 0))    # 0
```

merge

2つのマップを結合した新しいマップを返します。キーが重複する場合は第2マップの値が優先されます。元のマップは変更されません。

```
m1 = {"a": 1, "b": 2}
m2 = {"b": 99, "c": 3}
m3 = merge(m1, m2)
print(m3["a"])    # 1
print(m3["b"])    # 99
print(m3["c"])    # 3
```

制約とエラー

制約	詳細
全キーは同一型	異なる型のキーが混在するとコンパイルエラー
全値は同一型	異なる型の値が混在するとコンパイルエラー
空マップ	型注釈が必要 (<code>m: Map<str, int> = {"a": 1}</code> など)
存在しないキーアクセス	ランタイムエラー (<code>exit(1)</code>)
キー検索	ハッシュテーブル (平均 $O(1)$)
容量超過時	自動で 2 倍に拡張

セット

概要

同一型の要素を重複なしで保持するコレクション。ヒープ上に確保されます。

構文

```
s = {1, 2, 3}
s: Set<int> = {1, 2, 3}
```

対応する要素型

int, float, bool, str

等価性

セットは `==` と `!=` をサポートします。等価性は順序に依存しません。2 つのセットはまったく同じ要素を含むときに等しくなります。

```
{1, 2, 3} == {3, 2, 1}    # true (順序は関係ない)
{1, 2}    != {1, 2, 3}   # true (サイズが違う)
```

in 演算子 (所属チェック)

```
s = {1, 2, 3}
print(2 in s)    # true
print(5 in s)    # false
```

length

```
s = {1, 2, 3}
print(length(s)) # 3
```

print

```
s = {1, 2, 3}
print(s)    # {1, 2, 3}
```

add (要素追加)

重複する要素を追加した場合は無視されます。

```
s = {1, 2, 3}
s.add(4)          # 追加
s.add(1)          # 既に存在するため無視
print(length(s))  # 4
```

remove (要素削除)

```
s = {1, 2, 3}
s.remove(2)
print(2 in s)    # false
```

for 走査

```
s = {10, 20, 30}
for x in s:
    print(x)
```

空セット

空セットは型注釈が必要です。

```
s: Set<int> = {}
```

関数引数

```
function has_value(s: Set<int>, v: int) -> bool:
    return v in s
```

union

両方のセットの全要素を含む新しいセットを返します。

```
a = {1, 2, 3}
b = {3, 4, 5}
print(union(a, b))    # {1, 2, 3, 4, 5}
```

intersection

両方のセットに存在する要素のみを含む新しいセットを返します。

```
a = {1, 2, 3}
b = {2, 3, 4}
print(intersection(a, b))    # {2, 3}
```

difference

最初のセットにはあるが、2 番目のセットにはない要素を含む新しいセットを返します。

```
a = {1, 2, 3}
b = {2, 3, 4}
print(difference(a, b))    # {1}
```

symmetric_difference

いずれかのセットにあるが、両方にはない要素を含む新しいセットを返します。

```
a = {1, 2, 3}
b = {2, 3, 4}
print(symmetric_difference(a, b))    # {1, 4}
```


is_subset

最初のセットの全要素が2番目のセットに含まれている場合に true を返します。

```
a = {1, 2}
b = {1, 2, 3}
print(is_subset(a, b))    # true
print(is_subset(b, a))    # false
```

is_superset

最初のセットが2番目のセットの全要素を含んでいる場合に true を返します。

```
a = {1, 2, 3}
b = {1, 2}
print(is_superset(a, b))  # true
print(is_superset(b, a))  # false
```

制約とエラー

制約	詳細
全要素は同一型 空セット {}	異なる型が混在するとコンパイルエラー 型注釈が必要
要素検索	ハッシュテーブル (平均 $O(1)$)
容量超過時	自動で2倍に拡張

イテレータ

概要

効率的なデータ変換パイプラインを実現する遅延イテレータの抽象化です。イテレータは中間結果のコピーや実体化を行わず、各要素をオンデマンドで処理します。

イテレータの作成

`iter()` を使って任意のコレクションからイテレータを作成します:

```
xs = [1, 2, 3, 4, 5]
it = xs.iter()           # Iterator<int>

s = {10, 20, 30}
sit = s.iter()           # Iterator<int>

m = {"a": 1, "b": 2}
mit = m.iter()           # Iterator<(str, int)>
```

遅延メソッドチェーン

イテレータのメソッドは新しいイテレータを返し、消費されるまで評価されないパイプラインを形成します:

メソッド	説明
<code>.filter(function)</code>	述語が true を返す要素のみを生成
<code>.map(function)</code>	各要素を関数で変換
<code>.take(count)</code>	最大 count 要素を生成

メソッド	説明
------	----

```
result = [1, 2, 3, 4, 5]
    .iter()
    .filter((x: int) => x > 2)
    .map((x: int) => x * 2)
    .take(2)
    .to_list()    # [6, 8]
```

イテレータの消費

メソッド	説明
<code>.to_list()</code>	全要素を <code>List<T></code> に収集
<code>.next()</code>	次の要素を <code>Option<T></code> として返す

```
it = [10, 20].iter()
print(it.next())    # Some(10)
print(it.next())    # Some(20)
print(it.next())    # None
```

for ループのサポート

イテレータは for ループで直接使用できます:

```
for x in [1, 2, 3].iter().filter((x: int) => x > 1):
    print(x)
# 2
# 3
```

```
for k, v in {"a": 1, "b": 2}.iter():
    print(k)
```

[English](#) | [日本語](#) | [繁體中文](#)

契約による設計 (Design by Contract)

Ry は Eiffel スタイルの契約による設計をサポートしています。事前条件 (require)、事後条件 (ensure)、レコードの不変条件 (invariant) を使用できます。契約違反時はプロセスが `exit(1)` で終了します。

事前条件 (require)

事前条件は関数の入口でチェックされます。関数が正しく呼び出されるために満たすべき条件を指定します。

```
function deposit(amount: int, balance: int) -> int:
    require:
        amount > 0
        balance >= 0
    new_balance: int = balance + amount
    return new_balance
```

事前条件が満たされない場合、以下のメッセージとともにプログラムが終了します:

```
Contract violation: require failed in deposit()
```

事後条件 (ensure)

事後条件はすべての `return` の直前にチェックされます。関数の戻り値について保証する条件を指定します。

変数バインド

`ensure` には戻り値をバインドする変数名が必要です。この変数を事後条件の式で使用できます。

```
function abs(x: int) -> int:
  ensure v:
    v >= 0
  if x < 0:
    return -x
  return x
```

Ry の関数引数はイミュータブルなので、`ensure` ブロック内で引数を直接参照して入口時の値と比較できます:

```
function increment(x: int) -> int:
  ensure v:
    v == x + 1
  return x + 1
```

タプル展開

タプルを返す関数では、カンマ区切りで複数の変数名を指定できます:

```
function divide(a: int, b: int) -> (int, int):
  ensure q, r:
    q >= 0
    r >= 0
  return (a // b, a % b)
```

バインド変数の数はタプルの要素数と一致する必要があります。

組み合わせ例

```
function deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  ensure v:
    v >= 0
    v == balance + amount
  new_balance: int = balance + amount
  return new_balance
```

レコードの不変条件 (invariant)

不変条件はレコードのインスタンスが常に満たすべき条件です。以下のタイミングでチェックされます: - 構築時 - フィールド代入後

```
record BankAccount:
  balance: int
```

```

    min_balance: int
    invariant:
        balance >= min_balance

a = BankAccount(100, 0)    # OK: 100 >= 0
a.balance = -1             # Contract violation: invariant failed

```

ルール

- `require` と `ensure` ブロックはオプションで、関数本体の前に記述します。
- 両方を使う場合、`require` は `ensure` の前に記述する必要があります。
- `ensure` には戻り値をバインドする変数名が必要です (例: `ensure v:`)。
- タプル戻り値の場合、複数のバインド変数を指定できます (例: `ensure q, r:`)。
- `invariant` は `record` 定義の末尾、全フィールド宣言の後に記述します。
- すべての契約違反は `exit(1)` でプログラムを終了し、診断メッセージを出力します。

[English](#) | [日本語](#) | [繁體中文](#)

制御構文リファレンス

if / else

文の構文

```

if condition:
    # then ブロック
else:
    # else ブロック (省略可)

```

式の形

`if` は値を生成する式としても使えます。2 つの形式がサポートされています:

単一式形式 (`=>`) :

```
x = if condition => true_value else false_value
```

例:

```

abs_val = if x > 0 => x else -x
label = if score >= 90 => "A" else "B"

```

単一式形式の `else` 分岐は値を直接取ります (`=>` は不要)。両方の分岐は同じ型を返す必要があり、`else` は必須です。

ブロック形式 (`:`) :

```

x = if condition:
    compute_something()
else:
    compute_other()

```

ブロック形式では、各ブロックは式文で終わる必要があります (tail-expression セマンティクス)。`else` 分岐は必須で、両方の分岐は同じ型を返す必要があります。

値を返す多分岐の条件式が必要な場合は、代わりに `case:` を使ってください (下記参照)。

条件式の型

型	false になる値	true になる値
bool	false	true
int	0	非 0
float	0.0	非 0

bool、整数、float のみが条件式に使用できます。str、List、Map、Set、イテレータ、クロージャ、record、Option、Result は条件式に直接使用できません。コレクションや文字列に対しては、長さチェックを明示的に書いてください:

```
xs = [1, 2, 3]
# ☒ エラー: この型の値はブール条件として使えない
# if xs:
#     print("non-empty")
# ✓ 明示的な長さチェック
if length(xs) > 0:
    print("non-empty")
# ✓ is_empty を使った等価なコード
if not is_empty(xs):
    print("non-empty")
```

Option と Result については、条件式として使うのではなく、case で明示的にバリエントをパターンマッチしてください。これらのルールは while、case アーム、単項 not 演算子にも等しく適用されます。

例

```
x = 10

if x > 5:
    print("big")
else:
    print("small or equal")
```

スコープルール

- if / else の各ブロックはそれぞれ独立したブロックスコープを持つ。
- ブロック内で宣言した変数はブロック外からアクセスできない。

```
if true:
    y = 42
# y はここではアクセス不可
```

while

構文

```
while condition:
    # ループ本体
```

条件式が true の間、ループ本体を繰り返す。

例

```
i = 0
while i < 5:
    print(i)
    i += 1
```

break / continue との組み合わせ

```
i = 0
while true:
    if i >= 3:
        break
    i += 1
```

for

構文

```
# リスト / セット走査
for x in iterable_expr:
    # x に各要素が代入される

# range (0 始まり)
for i in range(n):
    # i = 0, 1, ..., n-1

# range (開始・終了指定)
for i in range(start, end):
    # i = start, start+1, ..., end-1

# range (ステップ指定)
for i in range(start, end, step):
    # i = start, start+step, start+2*step, ...
```

文字列の反復

str に対する for ループは、各 **Unicode コードポイント** を 1 文字の str として生成します。マルチバイトの UTF-8 シーケンス (CJK 文字や絵文字を含む) は正しくデコードされ、マルチバイト文字の途中で分割されることはありません。

これは **コードポイント** 単位の反復であり、**grapheme クラス**単位ではありません。複数のコードポイントにまたがるユーザーが認識する文字-結合マーク列 (例: 基底文字 + U+0301) や ZWJ 絵文字シーケンス (例: 家族や肌の色の合成) - は、コードポイントごとに 1 回ずつ複数の反復として生成されます。grapheme クラスを意識した反復が必要な場合は、for c in s: に頼らず、将来のセグメンテーションヘルパーで文字列を分解してください。

```
for c in "hello":
    print(c)          # h, e, l, l, o

for c in "こんにちは":
    print(c)          # こ, ん, に, ち, は (バイトごとではない)

for c in "a b":
    print(c)          # a,  , b
```

ループ変数は str 型なので、他の文字列関数に渡すことができます:

```
for c in "abc":
    print(to_upper(c))      # A, B, C
```

空文字列を反復するとループ本体は 0 回実行されます。enumerate と zip も str 引数を受け付け、同じコードポイント単位を生成します:

```
for i, c in enumerate("abc"):
    print(i, c)

for a, b in zip("abc", "xyz"):
    print(a + b)           # ax, by, cz
```

マップのキー・値走査

```
for k, v in map_expr:
    # k はキー、v は各エントリの値
```

タプル分解代入

タプルのリストを走査する際、タプルの要素数に合わせて N 個の変数に分解できます。_ で値を破棄できます。

```
xs = [10, 20, 30]
```

```
for i, x in enumerate(xs):
    print(f"{i}: {x}")      # 0: 10, 1: 20, 2: 30
```

```
for a, b in zip([1, 2], [10, 20]):
    print(a + b)           # 11, 22
```

```
for _, x in enumerate(xs):
    print(x)               # インデックスを破棄
```

N要素の分解代入 (3個以上の変数)

```
triples = [(1, 2, 3), (4, 5, 6)]
```

```
for a, b, c in triples:
    print(a + b + c)       # 6, 15
```

```
for a, _, c in triples:
    print(a + c)           # 4, 10 (中間の要素を破棄)
```

範囲演算子 (..)

.. 演算子は両端を含む整数の範囲を生成します。1 .. 5 は [1, 2, 3, 4, 5] を生成します。

```
for i in 1 .. 5:
    print(i)              # 1 2 3 4 5
```

例

```
xs = [10, 20, 30]
```

```
for x in xs:
    print(x)
```

```
s = {1, 2, 3}
```

```
for x in s:
    print(x)
```

```

for i in range(5):
    print(i)      # 0 1 2 3 4

for i in range(2, 6):
    print(i)      # 2 3 4 5

for i in range(0, 10, 2):
    print(i)      # 0 2 4 6 8

for i in range(10, 0, -3):
    print(i)      # 10 7 4 1

# マップの走査
m = {"a": 1, "b": 2}
for k, v in m:
    print(k)
    print(v)

# 範囲演算子
for i in 1 .. 3:
    print(i)      # 1 2 3

```

async / await

async function は並行実行される関数を宣言します。async function を呼び出すと Task<T> が返ります。別の async function 内では await を使い、同期コンテキストからは block_on() を使って結果を待ちます。

```

async function add(a: int, b: int) -> int:
    return a + b

# 同期コンテキストからは block_on() を使用
t: Task<int> = add(20, 22)
print(block_on(t))          # 42
print(block_on(add(1, 2)))  # 3

# async function 内では await を使用
async function double_add(a: int, b: int) -> int:
    result = await add(a, b)
    return result * 2

```

ルール

- async function name(...) -> T: の T は await 後の値型です。
 - async function の呼び出し結果は常に Task<T> です。
 - await expr は Task<T> にのみ使用でき、結果は T です。
 - await は async function 内でのみ使用可能。同期コンテキストからは block_on(task) を使用します。
 - block_on(task) は現在のスレッドをタスク完了までブロックし、結果を返します。
 - async function ... -> Unit をサポートします。値を返さない task の待機には block_on(task) を使うのが基本です。
 - task はランタイムの worker pool 上で実行され、task ごとに OS スレッドを作る実装ではありません。
 - async ラムダと async @native function は v1 では未対応です。
-

@parallel for

@parallel は range(...) または整数 .. を使う counted for ループにだけ付与できます。ループ本体はランタイムのワーカプール上でチャンク単位に並列実行されます。

```
@parallel
for i in range(8):
    print(i)
```

制約

- 対応するのは range(...) と整数 .. のみです。
- 分解代入付きの反復は未対応です。
- 外側スコープのミュータブル変数への代入は拒否されます。
- break と continue は使えません。
- v1 ではループ本体内のインデックス代入とフィールド代入も拒否されます。

ランタイムの worker 数は available_parallelism() で取得できます。

break

- 最も内側のループ (while または for) を即座に脱出する。
- ループの外で使用するとコンパイルエラー。

```
for i in range(10):
    if i == 5:
        break      # i == 5 で脱出
    print(i)        # 0 1 2 3 4
```

エラー例

```
# ループ外での break はコンパイルエラー
break      # Error: break outside loop
```

continue

- 最も内側のループの現在のイテレーションを終了し、次のイテレーションへスキップする。
- ループの外で使用するとコンパイルエラー。

```
for i in range(5):
    if i == 2:
        continue    # i == 2 をスキップ
    print(i)         # 0 1 3 4
```

... (Ellipsis)

- 何もしない文 (no-op)。空ブロックのプレースホルダーとして使用する。
- 関数ボディ、if/else、while、for、case アームなど任意のブロック内で使用可能。

```
function not_yet():
    ...

if true:
    ...
```

```
else:
    ...
```

case

case は、対象値なしの多分岐条件フロー（以前の when）と、パターン マッチング（以前の match）を 1 つの構文に統合したものです。2 つの 形式がサポートされています:

- case: - 対象なし、各アームは条件式（when: の置き換え）
- case <expr>: -対象あり、各アームはパターン（match の置き換え）

どちらの形式もブロック本体 (:) と式本体 (=>) の両方をサポートします。

注意: when と match キーワードは統合された case 構文に置き換え られ、削除されました。when / match を使う従来の Ry コードは移行が 必要です。

対象なしの case

対象値のない多分岐条件フローには case: を使います。

構文

```
case:
    condition:
        # 本体
    condition:
        # 本体
    -:
        # フォールバック
```

例

```
x = 0

case:
    x > 0:
        print("positive")
    x < 0:
        print("negative")
    -:
        print("zero")
```

アームは上から順に評価され、最初に条件が真になったアームだけが実行されます。文の場合、ワイルドカードアーム _: は省略可能です。

式形式の case: については、下の「式の形」セクションを参照してください。

対象ありの case（パターンマッチング）

構文

```
case expression:
    pattern:
        # 本体
    pattern if guard_condition:
```

```

    # ガード付き本体
-:
    # ワイルドカード（何にでもマッチ）

```

パターンの種類

パターン	例	説明
ワイルドカード	<code>_</code>	何にでもマッチ
リテラル	<code>0, "hello", true</code>	値の等値比較
変数束縛	<code>n</code>	何にでもマッチし、変数に束縛
enum バリエーション	<code>Color::Red</code>	enum タグの比較（単純な enum）
ADT enum バリエーション	<code>Shape::Circle(r)</code>	関連データを持つ enum バリエーションにマッチし、束縛する
<code>Some(x)</code>	<code>Some(v)</code>	<code>Option</code> が値ありの場合、中身を束縛
<code>None</code>	<code>None</code>	<code>Option</code> が値なしの場合
<code>Ok(x)</code>	<code>Ok(v)</code>	<code>Result</code> が <code>Ok</code> の場合、中身を束縛
<code>Err(x)</code>	<code>Err(e)</code>	<code>Result</code> が <code>Err</code> の場合、エラー値を束縛
OR パターン	<code>1 \ 2 \ 3</code>	いずれかにマッチ

guard 節

`pattern if condition:` の形式でガード条件を指定できる。パターンがマッチし、かつガード条件が真の場合にのみアームが実行される。

OR パターン

複数のパターンを `|` で結合し、いずれかにマッチさせることができます。変数束縛 (`n`、`Some(x)`、`Ok(v)`、`Err(e)`) は OR パターン内では使用できません。

```

case x:
  1 | 2 | 3:
    print("small")
  -:
    print("other")

```

enum の OR パターン

```

case color:
  Color::Red | Color::Blue:
    print("warm or cool")
  Color::Green:
    print("green")

```

網羅性チェック

- enum 型: すべてのバリエーションをカバーするか `_` が必要。OR パターンの各選択肢は個別にカウントされる。
- Option 型: `Some` と `None` の両方をカバーするか `_` が必要。
- bool 型: `true` と `false` の両方をカバーするか `_` が必要。
- int / float / str リテラル: `_` が必須。
- ガード付きアームは網羅性にカウントされない。

例

```
# enum のパターンマッチ
enum Color:
    Red
    Green
    Blue

case color:
    Color::Red:
        print("red")
    Color::Green:
        print("green")
    Color::Blue:
        print("blue")

# Option のパターンマッチ
x: Option<int> = Some(42)
case x:
    Some(v):
        print(v)
    None:
        print("nothing")

# Result のパターンマッチ
function divide(a: int, b: int) -> Result<int, Error>:
    if b == 0:
        return Err(Error("division by zero"))
    return Ok(a // b)

case divide(10, 2):
    Ok(v):
        print(v)          # 5
    Err(e):
        print(e.message)

# リテラルのパターンマッチ
case x:
    0:
        print("zero")
    1:
        print("one")
    -:
        print("other")

# guard 節
case x:
    n if n > 0:
        print("positive")
    n if n < 0:
        print("negative")
    -:
        print("zero")
```

ADT enum のパターンマッチ

enum バリエントが関連データを持つ場合、バインディングパターンを使って値を取り出します。

```
enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point

s = Shape::Circle(3.14)
case s:
    Shape::Circle(r):
        print(r)          # 3.14
    Shape::Rectangle(w, h):
        print(w)
        print(h)
    Shape::Point:
        print("point")
```

複数フィールドを持つバリエントは、宣言順に各フィールドを別々の名前に束縛します。

式の形

case: と case <expr>: のいずれも、各アームの : を => に置き換えることで式として使えます。各アームは単一の式を提供し、その値が結果になります。

```
# 対象なしの case 式
label = case:
    x > 100 => "huge"
    x > 10  => "big"
    x > 0   => "small"
    -      => "non-positive"
```

パターンマッチング式の形式:

構文

```
result = case expression:
    pattern => value_expression
    pattern if guard => value_expression
    _ => default_value
```

case 文でサポートされるすべてのパターンは case 式でもサポートされます: リテラル、変数束縛、enum、ADT enum、Some/None、Ok/Err、OR パターン、guard、ワイルドカード。

case 式は網羅的でなければなりません (case 文と同じルール)。

例

```
# Option
value = case opt:
    Some(v) => v
    None    => 0

# Enum
label = case direction:
    Direction::North => "N"
    Direction::South => "S"
    Direction::East  => "E"
```

```

Direction::West => "W"

# Guard
grade = case score:
  n if n >= 90 => "A"
  n if n >= 80 => "B"
  -           => "F"

# OR パターン
kind = case x:
  1 | 2 | 3 => "small"
  -       => "large"

# ADT enum
area = case shape:
  Shape::Circle(r)  => 3.14 * r * r
  Shape::Rectangle(w, h) => w * h
  Shape::Point      => 0.0

```

スコープルール

- 各 case アームはブロックスコープを持つ。
 - 変数束縛パターン (n)、Some(x)、Ok(v)、Err(e) で束縛された変数はそのアーム内でのみ有効。
-

スコープルール

ブロックスコープ

- if / else / while / for / case の各ブロックはブロックスコープを持つ。
- ブロック内で宣言した変数はブロックの終了と同時にスコープから外れる。

```

for i in range(3):
    tmp = i * 2
# tmp はここではアクセス不可

if true:
    a = 1
# a はここではアクセス不可

```

内側スコープでの再代入

- 内側のスコープで変数に代入すると、外側の変数が変更される (Python スタイルのスコーピング)。
- シャドーイングは行われず、内側の代入は同じ変数を変更する。

```

x = 10
if true:
    x = 99 # 外側の x を変更
    print(x) # 99
print(x) # 99

```

[English](#) | [日本語](#) | [繁體中文](#)

ディレクティブ

ディレクティブは宣言に付与できるコンパイル時メタデータアノテーションです。Java のアノテーションと同様の `@name` 構文を使用します。

構文

```
@name
@name(key=value, ...)
```

ディレクティブは対象の宣言の前に配置します。複数のディレクティブを重ねることもできます。

対象

ディレクティブは以下の宣言に適用できます:

- `function` - 関数定義 (`@it` / `@describe` で装飾された名前付きテスト関数を含む)
- `record` - レコード定義
- 変数宣言 (`@const` 付きまたは通常代入)
- `record` 定義内のフィールド
- `for` - カウント付きループのみ (`@parallel` 用)
- `it` / `describe` 呼び出し (従来のラムダ形式) - `@each` と `@property` 用のテストケースとテストグループ

組み込みディレクティブ

`@deprecated`

宣言を非推奨としてマークします。非推奨のエンティティが使用（呼び出し、参照、アクセス）されると、コンパイル時に警告が出力されます。

関数に対して:

```
@deprecated
function old_function() -> int:
    return 42

print(old_function())    # warning: 'old_function' is deprecated
```

型に対して:

```
@deprecated
record OldPoint:
    x: int
    y: int

@const
p = OldPoint(1, 2)    # warning: 'OldPoint' is deprecated
```

変数に対して:

```
@deprecated
@const
old_value = 99

print(old_value)      # warning: 'old_value' is deprecated
```

フィールドに対して:

```
record Config:
    @deprecated
```

```
old_setting: int
new_setting: int
```

```
@const
c = Config(1, 2)
print(c.old_setting)      # warning: 'Config.old_setting' is deprecated
print(c.new_setting)      # 警告なし
```

@const

変数を不変としてマークします。`@const` で宣言された変数は初期化後に再代入できません。`@const` なしの場合、変数はデフォルトで可変です。

```
@const
x = 42
# x = 10    # エラー: @const 変数への再代入はできません
```

型アノテーション付き:

```
@const
name: str = "hello"
```

タプル分割代入:

```
@const
a, b = (1, 2)
```

トップレベルの `@const` と関数について: トップレベルの `@const` 宣言は、同じソースファイル内で以降に定義されるすべてのトップレベル関数から参照可能です。不変性はすべての参照に対して強制され、トップレベル `@const` の record フィールドへのミューテーションも含まれます。詳細は [functions.md](#) の「トップレベル変数と関数ボディ内での `@const`」セクションを参照してください。

@native

ランタイムによって実装が提供される関数を宣言します。関数本体を持つことはできません。

オプションの文字列引数で共有ライブラリのモジュール名を指定できます。`@native("libname")` 関数が呼ばれると、JIT は対応する共有ライブラリ (macOS では `libry_<libname>.dylib`、Linux では `libry_<libname>.so`) を動的にロードし、そこからランタイムシンボルを解決します:

```
@native          # 組み込み (プロセスに静的リンクされる)
@native("base64") # libry_base64.dylib/.so から動的ロード
```

基本構文:

```
@native
function contains(string: str, substring: str) -> bool

print(contains("hello world", "world")) # true
```

演算子オーバーロード:

```
@native
function operator+(a: str, b: str) -> str

print("hello" + " world") # hello world
```

UFCS との組み合わせ:

```
@native
function to_upper(string: str) -> str
```



```
print("hello".to_upper()) # HELLO
```

引数数の検証:

@native 宣言に型シグネチャが含まれている場合、コンパイラは呼び出し時に引数の数を検証します。オーバーロードされた関数（例: 1, 2, 3 引数の range）もサポートされ、いずれかのオーバーロードにマッチすれば検証を通過します。

```
@native
function range(n: int) -> List<int>
@native
function range(start: int, end: int) -> List<int>

print(length(range(5)))      # OK: 1 引数のオーバーロードにマッチ
print(length(range(1, 10)))  # OK: 2 引数のオーバーロードにマッチ
print(length(range()))       # Error: expects 1 or 2 argument(s), but got 0
```

標準ライブラリ宣言 (core/):

core/ ディレクトリにはすべての組み込み関数の @native 宣言がカテゴリ別に格納されています:

ファイル	内容
core/builtins.ry	print, length, range, enumerate, zip, exit, args, available_parallelism, sleep
core/str.ry	contains, starts_with, ends_with, find, substring, char_at, replace, to_upper, to_lower, trim, trim_start, trim_end, repeat, reverse, split, join
core/convert.ry	to_int, to_float, to_str
core/list.ry	append, pop, insert, remove_at, slice, distinct, flatten, sort, first, last, is_empty
core/map.ry	keys, values, items, has_key, get, merge
core/set.ry	add, remove, union, intersection, difference, symmetric_difference, is_subset, is_superset
core/higher_order.ry	filter, map, reduce, fold, any, all, sum, min, max

これらのファイルは ry 実行バイナリの近くに core/ ディレクトリが存在する場合、プレリユードとして自動的にロードされます。プレリユードにより、組み込み関数呼び出し時の引数数検証が有効になります。

制約事項: - @native 関数は本体を持ってません（シグネチャの後に : を付けるとエラー）。- 本体を付けるとパースエラー: @native function must not have a body。- 引数なしの@native の場合、宣言した関数は既存の組み込み関数に対応している必要があります。対応していない場合はコンパイル時にエラーになります。@native("libname") の場合、関数は宣言されたシグネチャに基づいてコンパイルされ、ロードされたライブラリからシンボルを解決できなかった場合は JIT リンク時に失敗します。

ライブラリ指定: - @native("libname") は、native 関数が libry_<libname>.dylib (macOS) または libry_<libname>.so (Linux) という共有ライブラリに存在することを指定します。JIT 起動時、必要な共有ライブラリは以下の検索パス（順序通り）からロードされます: 1. exe/./lib/ - インストールレイアウト 2. exe/lib/ - 開発 / ビルドレイアウト 3. \$RY_HOME/lib/ - ユーザーインストール環境 - @native（静的）と@native("libname")（動的）のどちらの宣言も、引数数検証と呼び出し解決のために登録されます。違いは JIT にランタイムシンボルをどう提供するかだけです。- ランタイム関数名は _ry_<libname>_<fn_name> という規約に従います（例: @native("base64") fn encode(...) → _ry_base64_encode）。これは stdlib パッケージでも、ユーザー定義の native ライブラリでも同じように動作します。

@parallel

カウント付き for ループを並列実行対象としてマークします。

```
@parallel
for i in range(8):
    work(i)
```

対応対象:

- for 文のみ

制約事項:

- for 文には 1 つの @parallel だけ指定できます。
- 反復対象は range(...) または整数 .. レンジに限られます。
- 分解代入付きの反復は未対応です。
- 外側の可変変数への代入は拒否されます。
- v1 ではループ本体内の break、continue、インデックス代入、フィールド代入は拒否されます。

@each

パラメタライズドテストを有効にし、テストを異なるパラメータで複数回実行します。

構文 (名前付き関数に対して、推奨) :

```
@each([(arg1, arg2, ...), ...])
@it("should handle {0} and {1}")
function test_handle(param1: type, param2: type):
    # テスト本体
```

構文 (従来の it ラムダに対して) :

```
@each([(arg1, arg2, ...), ...])
it("should handle {0} and {1}", (param1: type, param2: type):
    # テスト本体
)
```

引数はタプルのリストを返す任意の式を使えます。関数呼び出しも可能です:

```
@each(make_inputs())
@it("should handle {0}")
function test_handle(x: int):
    # テスト本体
```

対応対象: @it で装飾された関数、または従来の it 呼び出し。

制約事項: - 引数はタプルのリストを返す式である必要がある - タプルのアリティは関数のパラメータ数と一致する必要がある - 説明文の {0}, {1}, ... はパラメータ値の文字列表現で置換される

@property

プロパティベーステストを有効にし、テストにランダム入力を生成します。

構文 (名前付き関数に対して、推奨) :

```
@property(count=100)
@it("should verify property name")
function test_property(a: int, b: int):
    # ランダム値によるテスト本体
```

構文 (従来の it ラムダに対して) :

```
@property(count=100)
it("should verify property name", (a: int, b: int):
    # ランダム値によるテスト本体
)
```

対応対象: `@it` で装飾された関数、または従来の `it` 呼び出し。

パラメータ:

パラメータ	型	デフォルト	説明
count	int	100	ランダム試行回数

対応するパラメータ型:

型	範囲
int	-1000 ~ 1000
float	-1000.0 ~ 1000.0
bool	true または false
str	ランダム ASCII、0-20 文字

失敗時は反例（失敗を引き起こしたパラメータ値）が表示されます。

@it

名前付き関数に装飾することでテストケースを宣言します。関数ボディがテスト本体となり、`ry test` で実行されます。完全な仕様は [テストリファレンス](#) を参照してください。

構文:

```
@it("description")
function test_name():
    # アサーション
```

基本例:

```
@it("should add 1 + 2 = 3")
function test_add():
    expect(1 + 2).to_eq(3)
```

@each / @property との組み合わせ:

```
@each([(1, 2, 3), (0, 0, 0), (-1, 1, 0)])
@it("should add {0} + {1} = {2}")
function test_add_each(a: int, b: int, expected: int):
    expect(a + b).to_eq(expected)
```

```
@property(count=100)
@it("should verify addition is commutative")
function test_commutative(a: int, b: int):
    expect(a + b).to_eq(b + a)
```

対応対象: `function` 宣言のみ。関数は戻り値型の注釈を持つてはいけません。

制約事項: - `ry test` で実行される `*.test.ry` ファイル内でのみ有効 - `@each` と組み合わせる場合、関数のパラメータリストはタプルのアリティと一致する必要がある - `@property` と組み合わせる場合、各パラメータ型はサポートされているジェネレータ型 (`int`、`float`、`bool`、`str`) のいずれかでなければならない

@describe

名前付き関数に装飾することで、関連するテストをグループ化します。ボディ内で宣言された内側の `@it` 関数はそのグループに属し、ボディに直接宣言された変数はすべての内側 `@it` にキャプチャされる共有セットアップ

プとして機能します。従来のラムダ形式と異なり、@describe グループは**ネストが可能**です。出力はネストの深さに比例してインデントされます。

構文:

```
@describe("group name")
function group_name():
    @it("nested test")
    function test_nested():
        # アサーション
```

基本例:

```
@describe("arithmetic")
function arithmetic_tests():
    @it("should subtract")
    function test_sub():
        expect(10 - 3).to_eq(7)

    @it("should multiply")
    function test_mul():
        expect(4 * 5).to_eq(20)
```

共有セットアップ:

外側の @describe ボディで宣言された変数は、内側のすべての@it 関数に自動的にキャプチャされます。

```
@describe("shared setup")
function shared_setup_tests():
    base = 100
    offset = 5

    @it("should use base")
    function test_base():
        expect(base).to_eq(100)

    @it("should use base and offset")
    function test_combined():
        expect(base + offset).to_eq(105)
```

ネストしたグループ:

```
@describe("outer")
function outer():
    @describe("inner")
    function inner():
        @it("should pass deeply nested test")
        function test_deep():
            expect(1 + 1).to_eq(2)
```

対応対象: function 宣言のみ。関数はパラメータも戻り値型の注釈も持つてはいけません。

@inline

LLVM オプティマイザにインライン化のヒントを与えます。デフォルトでは、関数を積極的にインライン化するように指示します。

基本的な使い方（常にインライン化）:

```
@inline
```

```
function add(a: int, b: int) -> int:
    return a + b
```

mode パラメータ付き:

```
@inline(mode="always")
function hot_path(x: int) -> int:
    return x * 2 + 1
```

```
@inline(mode="hint")
function medium_path(x: int) -> int:
    return x + 1
```

```
@inline(mode="never")
function cold_error_handler(msg: str):
    print("ERROR: " + msg)
```

モード:

モード	LLVM 属性	説明
always (デフォルト)	AlwaysInline	常にインライン化する
hint	InlineHint	オブティマイザにインライン化を提案する
never	NoInline	インライン化を禁止する

制約: - `@inline` は `@native` と併用できません (native 関数にはインライン化するボディがありません)。- 不明な mode 値はコンパイルエラーになります。

パラメータ (将来拡張)

ディレクティブは将来の拡張に備え、パラメータ構文をサポートしています:

```
@deprecated(reason="use new_api instead")
function old_api() -> int:
    return 0
```

現時点では、パラメータはパースされますが `@deprecated` ディレクティブでは使用されません。

注意事項

- 非推奨のエンティティは正常に動作します。警告が出力されるだけです。
- 警告は使用箇所では出力され、定義箇所では出力されません。
- 非推奨のエンティティを定義しても、使用しなければ警告は出力されません。
- 未知のディレクティブ名はパースエラーになります。
- サポートされない対象 (`if`、`while` 等) にディレクティブを付与するとパースエラーになります。 `for` に使えるのは `@parallel` のみです。

[English](#) | [日本語](#) | [繁體中文](#)

エラーリファレンス

エラーフォーマット

`ry` はコンパイルエラーを Rust 風のリッチフォーマットで表示します。エラーの正確な位置がソースコードのコンテキストとともに表示されます:

```

error: cannot reassign @const variable: x
--> main.ry:5:1
|
5 | x = 10
|   ^ cannot reassign @const variable: x

```

各エラーメッセージには以下が含まれます: - エラーレベルとメッセージ (error: ...) - ファイル位置 (--> ファイル: 行: 列) - ソース行 (該当するコード) - キャレットインジケータ (^) 正確な列を指し示す

コンパイルエラー一覧

エラー内容	原因	例
未宣言変数の使用	宣言されていない変数を参照した	print(x) (x が未宣言)
@const 変数への再代入	@const で宣言した変数に再代入した	@const x = 1 → x = 2
同名変数の再宣言	同スコープで同名の変数を再宣言した	x = 1 → 同名の再宣言
変数の型変更再代入	変数に異なる型の値を代入した	x = 1 → x = 3.14
型アノテーション不一致	宣言時の型と代入値の型が異なる	x: int = 3.14
オーバーロード戻り値型の重複	引数型が同一で戻り値型のみ異なるオーバーロードを定義した	引数 (int, int) で戻り値が int と float の2つの関数
オーバーロード解決失敗	引数型に一致するオーバーロードが存在しない	function add(a: int, b: int) に add(1.0, 2.0)
ビット演算に float を渡す	&, , ^, ~, <<, >> に float 型を渡した	3.14 & 1
空リスト	空リストリテラル [] から型を推論できない	xs = []
空マップ	空マップリテラル {} から型を推論できない	m = {}
タプル範囲外インデックス	タプルの存在しないインデックスにアクセスした	t = (1, 2) → t.2
ループ外での break/continue	for/while ループの外で break または continue を使用した	関数トップレベルで break
ブロック内でのモジュールインポート	from 文を関数・条件ブロック内で使用した	関数内で from math
循環インポート	モジュールが相互にインポートし合っている	a.ry が b.ry を、b.ry が a.ry をインポート
同一フィールド名の重複	構造体内で同じフィールド名を2回定義した	type T: x: int の中に x を2つ定義
case の網羅性不足	case がすべてのパターンを網羅していない	enum の一部バリエーションが未カバー、Option で None が欠落、Result で Ok/Err が欠落、リテラルに _ なし
? の非 Result 型への適用	Result 型でない式に? を適用した	x = 42 → x?

? の非 Result 返却関数での使用

Result を返さない関数内で
? を使用した

function foo() ->
int: -> bar()?

コンパイルエラーの例

```
# @const 変数への再代入
@const
x = 10
x = 20    # エラー

# 型変更再代入
n = 1
n = "hello"    # エラー: int 型に str 型を代入

# 空リスト
xs = []    # エラー: 型推論不可

# ループ外での break
break    # エラー: ループの外

# ブロック内でのインポート
function foo():
    from math    # エラー: トップレベルのみ

# 同一フィールド名の重複
record Bad:
    x: int
    x: float    # エラー: x が重複
```

ランタイムエラー一覧

エラー内容	原因	例
リスト範囲外アクセス	リストのインデックスが範囲を超えている	xs = [1, 2, 3] -> xs[5]
マップ存在しないキーアクセス	マップに存在しないキーを参照した	m = {"a": 1} -> m["b"]
契約違反	require・ensure・invariant の条件が false と評価された	契約による設計 を参照

すべてのランタイムエラーはプロセスを `exit(1)` で終了します。

ランタイムエラーの例

```
# リスト範囲外アクセス
xs = [1, 2, 3]
print(xs[10])    # ランタイムエラー: exit(1)

# マップ存在しないキーアクセス
m = {"a": 1}
print(m["z"])    # ランタイムエラー: exit(1)
```

ファイルシステム関数リファレンス

ファイルおよびディレクトリの操作を行います。すべての関数は `filesystem` からの明示的なインポートが必要です。

`filesystem` パッケージはファイルやディレクトリ自体に対する操作（コピー、移動、削除など）を扱います。一方、`io` パッケージはファイルの内容の読み書きを扱います。

```
from filesystem import list_dir, walk, glob_files, copy, move, remove, remove_all
from filesystem import make_dir, make_dir_all, file_size, is_file, is_dir, is_symlink
from filesystem import chmod, symlink, read_link
```

関数一覧

関数	シグネチャ	説明
<code>list_dir</code>	<code>(str) -> Result<List<str>, Error></code>	ディレクトリ内のエントリを一覧表示（非再帰）
<code>walk</code>	<code>(str) -> Result<List<str>, Error></code>	すべてのファイルとディレクトリを再帰的に一覧表示
<code>glob_files</code>	<code>(str) -> Result<List<str>, Error></code>	<code>glob</code> パターンに一致するファイルを検索
<code>copy</code>	<code>(str, str) -> Result<Unit, Error></code>	ファイルをコピー
<code>move</code>	<code>(str, str) -> Result<Unit, Error></code>	ファイルまたはディレクトリを移動・リネーム
<code>remove</code>	<code>(str) -> Result<Unit, Error></code>	ファイルまたは空のディレクトリを削除
<code>remove_all</code>	<code>(str) -> Result<Unit, Error></code>	ファイルまたはディレクトリツリーを再帰的に削除
<code>make_dir</code>	<code>(str) -> Result<Unit, Error></code>	単一のディレクトリを作成
<code>make_dir_all</code>	<code>(str) -> Result<Unit, Error></code>	ディレクトリと不足している親ディレクトリをすべて作成
<code>file_size</code>	<code>(str) -> Result<int, Error></code>	ファイルサイズをバイト単位で返す
<code>is_file</code>	<code>(str) -> bool</code>	パスが通常ファイルかどうかを確認
<code>is_dir</code>	<code>(str) -> bool</code>	パスがディレクトリかどうかを確認
<code>is_symlink</code>	<code>(str) -> bool</code>	パスがシンボリックリンクかどうかを確認
<code>chmod</code>	<code>(str, int) -> Result<Unit, Error></code>	ファイルのパーミッションを変更（POSIX モード）
<code>symlink</code>	<code>(str, str) -> Result<Unit, Error></code>	シンボリックリンクを作成
<code>read_link</code>	<code>(str) -> Result<str, Error></code>	シンボリックリンクのターゲットを読み取る

使用例

ディレクトリ操作

```
from filesystem import make_dir, make_dir_all, list_dir, remove_all
```



```

# 単一のディレクトリを作成
case make_dir("/tmp/myapp"):
    Ok(_):
        print("created")
    Err(e):
        print("error: " + e.message)

# ネストされたディレクトリを作成 (mkdir -p と同等)
make_dir_all("/tmp/myapp/data/logs")

# ディレクトリの内容を一覧表示
case list_dir("/tmp/myapp"):
    Ok(entries):
        for entry in entries:
            print(entry)
    Err(e):
        print("error: " + e.message)

# ディレクトリツリーを削除 (rm -rf と同等)
remove_all("/tmp/myapp")

```

ファイル操作

```

from filesystem import copy, move, remove, file_size
from io import write_text

write_text("/tmp/hello.txt", "Hello, World!")

# ファイルをコピー
copy("/tmp/hello.txt", "/tmp/hello_copy.txt")

# ファイルサイズを取得
case file_size("/tmp/hello.txt"):
    Ok(sz):
        print("size: " + to_str(sz))
    Err(e):
        print("error: " + e.message)

# ファイルを移動・リネーム
move("/tmp/hello_copy.txt", "/tmp/renamed.txt")

# ファイルを削除
remove("/tmp/renamed.txt")

```

再帰的な走査

```

from filesystem import walk, glob_files

# ディレクトリツリーを走査 (find と同等)
case walk("/var/log"):
    Ok(files):
        for f in files:
            print(f)
    Err(e):
        print("error: " + e.message)

```

```
# glob パターンマッチング
case glob_files("/var/log/*.log"):
    Ok(matches):
        for m in matches:
            print(m)
    Err(e):
        print("error: " + e.message)
```

パスの種類チェック

```
from filesystem import is_file, is_dir, is_symlink

if is_file("/etc/hosts"):
    print("regular file")

if is_dir("/tmp"):
    print("directory")

if is_symlink("/usr/local/bin/python"):
    print("symbolic link")
```

シンボリックリンク

```
from filesystem import symlink, read_link, is_symlink

# シンボリックリンクを作成
symlink("/usr/local/bin/ry", "/tmp/ry_link")

# シンボリックリンクを確認・読み取り
if is_symlink("/tmp/ry_link"):
    case read_link("/tmp/ry_link"):
        Ok(target):
            print("points to: " + target)
        Err(e):
            print("error: " + e.message)
```

パーミッション

```
from filesystem import chmod

# chmod 755 (rwxr-xr-x) — 10進数値を使用: 0o755 = 493
chmod("/tmp/script.sh", 493)

# chmod 644 (rw-r--r--) — 0o644 = 420
chmod("/tmp/data.txt", 420)
```

注意事項

- is_file、is_dir、is_symlink はエラー時（例: パスが存在しない）に false を返す
- is_file と is_dir はシンボリックリンクをたどる。is_symlink はリンクを検出するために lstat を使用する
- list_dir はエントリ名のみを返す（フルパスではない）
- walk はすべてのエントリ（ファイルとディレクトリの両方）のフルパスを返す
- glob_files はパターンに一致するファイルがない場合、空のリストを返す（エラーではない）

- `remove` は空でないディレクトリでは失敗する。再帰的な削除には `remove_all` を使用する

[English](#) | [日本語](#) | [繁體中文](#)

関数リファレンス

関数定義の構文

```
function function_name(param_name: type, ...) -> return_type:
    # 本体
    return value
```

- 引数の型は省略可能。省略時は `any` 型として扱われる。
- 戻り値型は省略可能。省略時は**ボディから推論**される（名前付き関数とラムダの両方）。`return` 文がなければ `Unit` に推論される。明示的に任意の戻り値型を許容するには `-> any` を指定する。
- 本体はインデントされたブロック。
- 明示的な戻り値型（`Unit` と `any` を除く）を持つ関数は、すべての制御フローパスで `return` 文が必要です。不足している場合はコンパイルエラーになります。
- 関数には `require`（事前条件）と `ensure`（事後条件）を定義できます。[契約による設計](#)を参照。

命名規則: 関数名と引数名は `snake_case`（例: `add`, `get_value`, `map_list`）を使用する必要があります。コンパイラがこの規則を強制します。

```
function add(a: int, b: int) -> int:
    return a + b
```

```
function greet(name: str) -> Unit:
    print("Hello, " + name)    # 戻り値型は Unit (明示的)
```

引数と戻り値の型

項目	説明
引数型	省略可能。: 型 を省略すると <code>any</code> になる
戻り値型	省略可能。省略時はボディから推論される（ <code>return</code> 文がなければ <code>Unit</code> ）
<code>Unit</code>	値を返さない関数の戻り値型

注意: 関数の引数は**不変**です。関数ボディ内で引数を再代入することはできません。これにより、事後条件チェックで引数のエントリ時の値を常に利用できます（[契約による設計](#)を参照）。

```
function no_return(x: int) -> Unit:    # 戻り値型 Unit (明示的)
    print(x)
```

```
function get_value() -> int:          # 戻り値型 int
    return 42
```

```
function identity(x) -> any:          # 引数型 any (省略)
    return x
```

型省略と `any`

引数の型アノテーションを省略すると、その引数は `any` として扱われます。`any` は実行時に任意のプリミティブ値を受け入れる動的型です。Python の型なし引数と同様の仕組みです。

```
# すべての引数が any
function add(a, b):
    return a + b

add(1, 2)           # 3 (int + int)
add("hello", " world") # "hello world" (str + str)
add(1, 2.0)         # 3.0 (int + float)
```

型アノテーションに `any` を明示的に書くこともできます:

```
function identity(x: any) -> any:
    return x
```

戻り値型の推論

戻り値型を省略すると、ボディ内の `return` 文から推論されます:

```
function double(x: int):      # 戻り値型は int に推論
    return x * 2

function greet(name: str):    # 戻り値型は Unit に推論 (return なし)
    print("Hello, " + name)
```

明示的に任意の戻り値型を許容するには `-> any` を指定します:

```
function flexible(x: any) -> any:
    return x    # int、float、str 等を返せる
```

ネスト関数

関数は他の関数の内部に定義できます。ネスト関数は囲む関数のスコープ内からのみ可視で、外部からは呼び出せません。

```
function outer() -> int:
    function helper() -> int:
        return 42
    return helper()

outer()    # 42
# helper() # エラー: undefined function
```

兄弟スコープにある同名のネスト関数は衝突しません:

```
function foo() -> int:
    function helper() -> int:
        return 1
    return helper()

function bar() -> int:
    function helper() -> int:
        return 2
    return helper()

foo()    # 1
bar()    # 2
```

ネスト関数は値として使用でき、高階関数に渡すこともできます。同じスコープ内のネスト関数間の相互再帰も動作します (コンパイラが前方宣言します)。

クロージャキャプチャ

ネストされた名前付き関数は、ラムダと同様に囲むスコープから変数をキャプチャできます。ネスト関数が外側の変数を参照すると、それはクロージャになります:

```
function make_adder(base: int) -> function(int) -> int:
  function add(x: int) -> int:
    return x + base
  return add
```

```
add10 = make_adder(10)
add10(5)    # 15
```

キャプチャルール:

- キャプチャは**値渡し**（ラムダと同じ）です。値はクロージャ生成時点でコピーされます。
- キャプチャされた変数はネスト関数のボディ内で**再代入できません**。
- ARC 管理の値（文字列、リスト等）は適切にリテイン／リリースされます。
- ネスト関数がキャプチャを持たない場合、単なる関数ポインタのままです（オーバーヘッドなし）。
- 多段階のキャプチャも動作します。深くネストされた関数は、囲むスコープの任意の変数を参照できます。

再帰

関数は自身を呼び出せる。

```
function factorial(n: int) -> int:
  if n <= 1:
    return 1
  return n * factorial(n - 1)
```

相互再帰

関数は定義順に関係なく相互に呼び出すことができます。コンパイラは明示的な戻り値型を持つ関数をボディ処理前に前方宣言します-これはトップレベルおよび別の関数ボディ内に定義されたネスト関数のいずれにも適用されます-ただし参照される型がすべて既知（プリミティブ型は常に利用可能、record/enum 型はファイル内で先に定義されている必要がある）であることが条件です。

```
function is_even(n: int) -> bool:
  if n == 0:
    return true
  return is_odd(n - 1)      # 下で定義された is_odd を呼び出す

function is_odd(n: int) -> bool:
  if n == 0:
    return false
  return is_even(n - 1)     # 上で定義された is_even を呼び出す
```

前方参照の要件:

- 関数は**明示的な戻り値型アノテーション**（-> type）を持つ必要があります。推論された戻り値型の関数は前方参照できません。
- 関数は**トップレベル**または別の関数ボディ内に定義できます。前方参照は同じスコープレベル内で動作します。
- すべてのパラメータ型と戻り値型は前方宣言の時点で解決可能でなければなりません（例: record 型はそれを使用する関数の前に定義されている必要があります）。

トップレベル変数と関数ボディ内での@const

トップレベルの `let` 束縛と `@const` 宣言は、同じソースファイル内で宣言が参照する関数のテキスト上に現れている限り、任意のトップレベル関数- およびそれらの関数内のネスト関数やラムダ- から可視です。

```
@const
PI: float = 3.14

@const
MAX_RETRIES: int = 5

counter: int = 0

function area(radius: float) -> float:
    return PI * radius * radius           # トップレベル @const を読む

function clamp_retries(n: int) -> int:
    if n > MAX_RETRIES:
        return MAX_RETRIES
    return n

function bump():
    counter = counter + 1                 # トップレベル可変 `let` に書く
```

ルール:

- ソース順に厳密。関数ボディは同じファイル内でそれより後に宣言されたトップレベル束縛を参照できません。束縛を関数の上に移動するか、遅延的に呼び出されるヘルパー関数でラップしてください。
- `@const` は読み取り専用。再代入やフィールドミュートーション（トップレベル `@const P: Point` に対する `P.x = 99`）はコンパイル時に拒否されます。
- 可変 `let` への書き込みはスルー。関数内部からトップレベル可変変数に代入すると、実際にトップレベル束縛がミュートーションされます。同名のローカルが作成されるわけではありません。
- ネストしたブロックはモジュールレベルではない。トップレベルの `if`、`while`、`for` ブロック内の `let` はそのブロックにローカルで、関数からは可視ではありません。

制限事項 (v0.0.8):

- 並列 `for` ブロックはトップレベル可変変数に代入できません（データ競合の回避）。
- トップレベル `weak` 参照とリソース型の束縛（`file / regex` ハンドル）は、まだ関数ボディからアクセスできません。必要ならフォローアップ issue でこれらのユースケースを追跡してください。

末尾呼び出し最適化

コンパイラは、関数の最後の処理が自分自身の呼び出しである自己再帰末尾呼び出しを自動的に検出し、LLVM の `musttail` 最適化を適用します。これにより、末尾再帰関数は一定のスタック空間を使用することが保証され、深い再帰でのスタックオーバーフローを防ぎます。

```
function sum_to(n: int, acc: int) -> int:
    if n <= 0:
        return acc
    return sum_to(n - 1, acc + n)         # 末尾呼び出し → 最適化される

sum_to(1000000, 0)                       # スタックオーバーフローなしで動作
```

TCO の条件:

- `return` 文で自分自身を直接呼び出している（`return f(args)`）
- 呼び出しの結果がそのまま返される（`return n * f(n-1)` は末尾呼び出しではない）
- 関数に `ensure`（事後条件）がない

相互再帰（A が B を呼び、B が A を呼ぶ）は現在最適化されません。

オーバーロード

引数の数や型が異なる同名の関数を複数定義できる。

ルール

- 引数の数または型が異なれば同名の関数を定義可能。
- 呼び出し時は引数の型と数に基づいて適切な関数が選択される。
- 戻り値型だけが異なるオーバーロードはできない。

```
function area(side: int) -> int:
    return side * side

function area(w: int, h: int) -> int:
    return w * h

a = area(5)           # 25
b = area(3, 4)        # 12
```

解決優先順位

複数のオーバーロードが呼び出しに一致する場合、コンパイラは以下の優先順位（高い順）で最も具体的なものを選択します：

1. 完全型一致 – 引数の型がパラメータの型に完全に一致
2. 暗黙の拡大変換 – 安全な拡大変換（`u8 → int`、`u8 → float`、`int → float`）
3. union 型一致 – 引数の型が union パラメータ型のメンバー
4. any 型一致 – パラメータの型が any（任意の値を受け入れる）

完全一致が最も多いオーバーロードが選ばれます。2 つ以上のオーバーロードが同じ具体性を持つ場合、コンパイラは曖昧性エラーを報告します。

低レベル数値型（`i8`、`i16`、`i32`、`i64`、`u8`～`u64`、`f32`）は暗黙の拡大変換に参加しません– 明示的な `as` キャストが必要です。

```
function process(x: int) -> str:
    return "int"

function process(x) -> str:           # x: any
    return "any"

process(42)           # "int" – 完全一致 (int) が any に勝つ
process("hello")      # "any" – str の完全一致がないため any にフォールバック

function double(x: float) -> float:
    return x * 2.0

double(5)             # OK – int が暗黙的に float に拡大変換され、10.0 を返す
```

デフォルト引数

パラメータにデフォルト値を指定でき、呼び出し時に末尾の引数を省略できる。

構文

```
function connect(host: str, port: int = 8080, timeout: int = 30):
    # ...

connect("localhost")           # port=8080, timeout=30
connect("localhost", 3000)      # port=3000, timeout=30
connect("localhost", 3000, 10000) # port=3000, timeout=10000
```

ルール

- デフォルトパラメータは非デフォルトパラメータの後に配置する。
- デフォルト値を持つパラメータには明示的な型注釈が**必須**（例: `x: int = 10`; `x = 10` はコンパイルエラー）。
- デフォルト値はコンパイル時定数式（リテラルおよび `@const` 変数）に限定。
- デフォルト引数により曖昧なオーバーロード（arity 範囲の重複 + 型の一致）が生じる場合、コンパイルエラーとなる。

```
# エラー: 曖昧なオーバーロード
function calc(x: int, y: int = 0) -> int:
    return x + y
function calc(x: int) -> int:           # 上の calc(int) と衝突
    return x * 2
```

制限事項

- ジェネリック関数およびラムダ式ではデフォルト引数は未サポート。

Unit 型関数

戻り値のない関数は `Unit` を返す。戻り値型は省略可能（`Unit` に推論される）、または `-> Unit` で明示的に指定可能。

```
function log(msg: str) -> Unit:
    print(msg)
```

Task と async 関数

`Task<T>` は並行実行の組み込みハンドル型です。 `async function` は `Task<T>` を返し、`await` は別の `async function` 内で `T` を取り出します。 `block_on(task)` は同期コンテキストからタスクの完了までブロックします。

```
async function add(a: int, b: int) -> int:
    return a + b

# 同期コンテキストからは block_on() を使用
t: Task<int> = add(20, 22)
print(block_on(t))           # 42
block_on(add(1, 2))          # 待機して結果を捨てる

# async function 内では await を使用
async function double_add(a: int, b: int) -> int:
    return (await add(a, b)) * 2
```


ルール

- `async function name(...) -> T:` の `T` は `await` 後の値型です。
 - `async function` の呼び出し結果は常に `Task<T>` です。
 - `await expr` は `Task<T>` にのみ使用でき、結果は `T` です。
 - `await` は `async function` 内でのみ使用可能。同期コンテキストからは `block_on(task)` を使用します。
 - `block_on(task)` は現在のスレッドをタスク完了までブロックし、結果を返します。
 - `async function ... -> Unit` をサポートします。値を返さない `task` の待機には `block_on(task)` を使うのが基本です。
 - `task` はランタイムの worker pool 上で実行され、`task` ごとに OS スレッドを作る実装ではありません。
 - `async` ラムダと `async @native function` は v1 では未対応です。
-

ラムダ関数

無名関数をその場で定義できる。

構文

```
# 単一式（式の値が返る。戻り値型は推論）
(param_name: type, ...) => expression

# 引数型の省略（any がデフォルト）
(param_name, ...) => expression

# 複数行ブロック
(param_name: type, ...):
    # 複数の文
    return value

# 戻り値型の明示（省略可能）
(param_name: type, ...) -> return_type => expression
```

例

```
double = (x: int) => x * 2
result = double(5)    # 10

add = (a: int, b: int) => a + b
sum = add(3, 4)       # 7

# 複数行ラムダ
abs = (x: int):
    if x < 0:
        return -x
    return x
```

クロージャ

ラムダ関数は定義された時点の外側スコープの変数を値でキャプチャします。クロージャはキャプチャ時点の独立したコピーを受け取り、キャプチャされた変数はクロージャ内で再代入できません。

外側の変更はクロージャに影響しない

クロージャはコピーを保持するため、クロージャ定義後に元の変数を再代入しても、キャプチャされた値には影響しません:

```
base = 10
add_base = (x: int) => x + base    # base を値でキャプチャ (10 のコピー)

base = 99                        # キャプチャ済みの値には影響しない
r = add_base(5)                  # 15 (キャプチャ時の base = 10 を使用)
```

キャプチャされた変数は実質的に final

キャプチャされた変数はクロージャ内で再代入できません。再代入しようとするとコンパイルエラーになります:

```
counter = 0
inc = ():
    counter += 1    # コンパイルエラー: cannot modify captured variable 'counter' inside closure

inc()
```

キャプチャされた record へのフィールド代入は許可されています。変数自体の再代入ではなく、コピーの内部状態を変更するためです:

```
record Point:
    x: int
    y: int

p = Point(0, 0)
move = ():
    p.x = p.x + 1    # OK — キャプチャされたコピーのフィールドを変更
```

注意: フィールドの変更はクロージャのコピーにのみ適用され、外側の変数には影響しません。

キャプチャルール

項目	内容
キャプチャ方式	値キャプチャ (コピー)
キャプチャタイミング	ラムダ定義時
キャプチャ変数の再代入	不可 (コンパイルエラー)
キャプチャされた record へのフィールド代入	可 (コピーのみを変更)
外側変数の変更の影響	ない (クロージャは独自のコピーを保持)

Python/JavaScript ユーザーへの注意: JavaScript ではクロージャは変数を参照でキャプチャするため、キャプチャされた変数への変更はクロージャの外側にも反映されます。Python ではクロージャは外側の変数にアクセスでき、外側の名前の再バインド (例: `counter += x`) には `nonlocal` 宣言が必要です。Ry では、クロージャは常に値でキャプチャし、キャプチャされた変数は実質的に final です - クロージャ内で再代入できません。これは意図的なもので、特に並行処理や高階関数のコンテキストにおいて安全性と予測可能性を確保します。

関数型

関数を値として扱うための型。

構文

```
function(param_type1, param_type2, ...) -> return_type
```

例

```
f: function(int) -> int = (x: int) => x * 2
g: function(int, int) -> int = (a: int, b: int) => a + b

function apply(func: function(int) -> int, x: int) -> int:
    return func(x)

result = apply(f, 5)    # 10
```

文字列表現

print()、to_str()、f-string の補間はいずれも関数値に対して"<closure>" を生成します:

```
f = (x: int) => x + 1
print(f)                # <closure>
s = to_str(f)           # "<closure>"
msg = f"fn={f}"         # "fn=<closure>"
```

注意: クローージャ間の等価比較 (== / !=) はサポートされておらず、コンパイル時エラーになります。

高階関数

関数を引数として受け取ったり、戻り値として返したりできる。

```
function map_list(xs: List<int>, f: function(int) -> int) -> List<int>:
    result: List<int> = []
    for x in xs:
        result += [f(x)]
    return result

doubled = map_list([1, 2, 3], (x: int) => x * 2)
# [2, 4, 6]
```

ジェネリック関数

関数は型パラメータを持つことができ、コード重複なしに型安全な再利用を実現します。

構文

```
function name<T, U>(param1: T, param2: U) -> T:
    # T、U を型として使用するボディ
```

例

```
function identity<T>(x: T) -> T:
    return x

# 明示的な型引数
result = identity[int](42)    # 42
```

```
result = identity[str]("hello") # "hello"
```

型推論 (実引数から型引数を推論)

```
result = identity(42)           # T = int, result = 42
result = identity("hello")      # T = str, result = "hello"
```

複数の型パラメータ

```
function pick_first<T, U>(a: T, b: U) -> T:
    return a
```

```
result = pick_first(1, "x")      # T = int, U = str, result = 1
result = pick_first("hello", 42) # T = str, U = int, result = "hello"
```

コンテナ型内の型パラメータ

型パラメータはジェネリックなコンテナ型 (List<T>, Map<K, V>, Set<T>), タプル (T, T), 関数型 function(T) -> T の内部にも現れることができます。推論は宣言されたパラメータ型を実際の引数と構造的に突き合わせるため、形が曖昧でない限り明示的な型注釈は必要ありません。

```
function first_of<T>(xs: List<T>) -> T:
    return xs[0]
```

```
first_of([1, 2, 3])           # T = int  -> 1
first_of(["hello", "world"])  # T = str  -> "hello"
first_of([[1, 2], [3, 4]])    # T = List<int> -> [1, 2]
```

```
function map_lookup<K, V>(m: Map<K, V>, k: K) -> V:
    return m[k]
```

```
map_lookup({1: "a", 2: "b"}, 1)      # K = int, V = str -> "a"
map_lookup({"x": 10, "y": 20}, "y") # K = str, V = int -> 20
```

```
function pair_first<T>(p: (T, T)) -> T:
    return p.0
```

```
pair_first((42, 7))      # T = int -> 42
pair_first(("a", "b"))   # T = str -> "a"
```

複数のパラメータ位置から参照される型パラメータは統一されます -両方の出現が同じ具体型に解決される必要があります:

```
function apply_list<T>(xs: List<T>, f: function(T) -> T) -> T:
    return f(xs[0])
```

```
apply_list([10, 20, 30], (x: int) => x + 1) # T = int -> 11
```

推論が型パラメータを決定できない場合 (例: 空のコンテナリテラル)、明示的な name[Type](args) 構文を使ってください:

```
first_of[int]([]) # 空リスト: コンパイラに T = int を明示的に伝える
```

引数間で矛盾する推論が発生した場合、曖昧な型不一致ではなく、型パラメータと関数名を明示したわかりやすいコンパイルエラーになります:

```
function same<T>(a: T, b: T) -> T:
    return a
```

```
same(1, "x") # error: conflicting type inference for 'T' in call to 'same'
```

型制約 (バウンド)

型パラメータには `: RecordName` 構文でレコード型の制約を付けることができます。具体型はバウンド型自体またはそのサブタイプでなければなりません。

```
record Animal:
```

```
  name: str
```

```
  legs: int
```

```
record Dog < Animal:
```

```
  breed: str
```

```
function get_name<T: Animal>(a: T) -> str:
```

```
  return a.name
```

```
get_name(Dog("Rex", 4, "Lab")) # OK — Dog は Animal のサブタイプ
```

```
get_name(Animal("Cat", 4))     # OK — 完全な型一致
```

バウンド付きとバウンドなしの型パラメータを混在させることができます:

```
function pair_name<T: Animal, U>(a: T, x: U) -> str:
```

```
  return a.name
```

動作の仕組み

ジェネリック関数は**単相化**を使用します: 型引数の一意な組み合わせごとに特殊化されたバージョンの関数が生成されます。同じインスタンス化はキャッシュされ、複数の呼び出しで再利用されます。型制約がある場合、インスタンス化時に検証されます。

UFCS (統一関数呼び出し構文)

`a.f(b)` の形式で `f(a, b)` を呼び出せる。メソッドチェーンに使いやすい。

構文

```
# 通常呼び出し
```

```
f(a, b)
```

```
# UFCS 呼び出し (等価)
```

```
a.f(b)
```

チェーン

```
function double(x: int) -> int:
```

```
  return x * 2
```

```
function add_one(x: int) -> int:
```

```
  return x + 1
```

```
result = 5.double().add_one() # double(5) -> 10, add_one(10) -> 11
```

フィールドアクセスとの混在

フィールドアクセス (`.field`) と UFCS (`.method()`) は同じドット記法で書けるが、引数の有無で区別される。

```
p = Point(3, 4)
length = p.x.to_float()    # フィールドアクセス + UFCS
```

演算子オーバーロード

ユーザー定義型に対して演算子の振る舞いを定義できる。

構文

```
# 二項演算子 (引数 2 個)
function operator<op>(a: type, b: type) -> return_type:
    ...

# 単項演算子 (引数 1 個)
function operator<op>(a: type) -> return_type:
    ...
```

オーバーロード可能な演算子

種別	演算子
算術 (二項)	+ - * / % ** //
比較 (二項)	== != < <= > >=
ビット (二項)	& \ ^ << >>
論理 (二項)	and or
所属	in
添字	[] (読み取り)、[] = (書き込み)
呼び出し	()
キャスト	as
単項	- ~ not

戻り値型の制約

比較演算子、論理演算子、所属演算子は `bool` を返す必要がある:

種別	演算子	必須戻り値型
比較	== != < <= > >=	<code>bool</code>
論理	and or not	<code>bool</code>
所属	in	<code>bool</code>
キャスト	as	必須 (ターゲット型)

```
# OK
function operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y

# エラー: comparison operator '==' must return 'bool', but returns 'int'
function operator==(a: Vec2, b: Vec2) -> int:
    return 42
```

算術演算子・ビット演算子には戻り値型の制約はない。

二項 / 単項の区別

引数の個数で区別する。

```
record Vec2:
  x: float
  y: float

# 二項 +
function operator+(a: Vec2, b: Vec2) -> Vec2:
  return Vec2(a.x + b.x, a.y + b.y)

# 単項 -
function operator-(v: Vec2) -> Vec2:
  return Vec2(-v.x, -v.y)

# 比較
function operator==(a: Vec2, b: Vec2) -> bool:
  return a.x == b.x and a.y == b.y

v1 = Vec2(1.0, 2.0)
v2 = Vec2(3.0, 4.0)
v3 = v1 + v2      # Vec2(4.0, 6.0)
v4 = -v1          # Vec2(-1.0, -2.0)
```

チェック付き/飽和演算

低レベル整数型 (i8, i16, i32, i64, u8, u16, u32, u64) 向けの明示的なオーバーフロー制御用組み込み関数です。両方の引数は同じ型でなければなりません。

関数	戻り値	動作
checked_add(a, b)	Result<T, Error>	オーバーフロー時に Err を返す
checked_sub(a, b)	Result<T, Error>	アンダーフロー時に Err を返す
checked_mul(a, b)	Result<T, Error>	オーバーフロー時に Err を返す
saturating_add(a, b)	T	オーバーフロー時に型の最小/最大値にクランプ
saturating_sub(a, b)	T	アンダーフロー時に型の最小/最大値にクランプ
saturating_mul(a, b)	T	オーバーフロー時に型の最小/最大値にクランプ
wrapping_add(a, b)	T	明示的なラッピング (+ と同じ)
wrapping_sub(a, b)	T	明示的なラッピング (- と同じ)
wrapping_mul(a, b)	T	明示的なラッピング (* と同じ)

```
# チェック付き: Result を返す。case や ? でハンドリング
r = checked_add(2147483647i32, 1i32)
case r:
  Ok(v):
    print(v)
  Err(e):
    print("overflow!")    # "overflow!" が出力される

# 飽和: 境界値にクランプ
```

```
v = saturating_add(2147483647i32, 100i32)
print(v as int)    # 2147483647
```

ラッピング: 自己文書化的なラッピング動作

```
v = wrapping_add(2147483647i32, 1i32)
print(v as int)    # -2147483648
```

注意: これらの関数は浮動小数点型 (f32) や高レベルの int 型をサポートしていません。低レベル整数のデフォルトの +、-、* 演算子はラッピング動作 (符号付きは 2 の補数、符号なしはモジュラー) を使用します。

[English](#) | [日本語](#) | [繁體中文](#)

GC リファレンス

gc パッケージは、循環参照チェーンを検出して回収するサイクルコレクタの制御を提供します。サイクルコレクタは ARC (自動参照カウント) と併用されるセーフティネットです。

概要

Ry はメモリ管理に ARC を使用しており、ほとんどの解放を即座に決定論的に処理します。ただし、ARC 単体では循環参照 (例: A -> B -> A) を回収できません。サイクルコレクタは CPython スタイルの試行削除アルゴリズムを使用して、定期的にこのような到達不能なサイクルを検出し解放します。

循環を回避するには weak 参照を使用することが推奨されますが、サイクルコレクタはユーザーが見落としたケースを捕捉します。

インポート

```
from gc import collect, enable, disable, set_threshold
```

関数

関数	シグネチャ	説明
collect	() -> int	完全なコレクションサイクルを実行する。回収されたオブジェクト数を返す。
enable	() -> Unit	自動コレクションを有効にする (デフォルトで有効)。
disable	() -> Unit	自動コレクションを無効にする。パフォーマンスが重要なセクションで有用。
set_threshold	(n: int) -> Unit	自動コレクションの候補数しきい値を設定する (デフォルト: 700)。

仕組み

- 候補の追跡:** arc_release がオブジェクトの参照カウントをデクリメントしてもゼロより大きい場合、そのオブジェクトは候補セットに追加される (サイクルを形成する可能性のある型のみ)。
- 試行削除:** コレクタは候補が参照するすべてのオブジェクトの参照カウントを暫定的にデクリメントする。試行削除だけでオブジェクトのカウントがゼロになった場合、そのオブジェクトは候補セット内からのみ到達可能であり、サイクルの一部である。
- 復活チェック:** 候補セットの外部からまだ到達可能なオブジェクトのカウントは復元される。

4. 回収: 残りの到達不能なオブジェクトが解放される。

静的解析による最適化

コンパイラはコンパイル時に静的解析を行い、参照サイクルを形成する可能性のある型（例: 再帰的な ADT enum）を特定します。これらの型のみが GC 候補追跡に参加します。非循環型は GC オーバーヘッドがゼロです。

使用例

```
from gc import collect, disable, enable

# パフォーマンスが重要なセクションで自動コレクションを無効にする
disable()

# ... パフォーマンスが重要なコード ...

# 再度有効にしてコレクションを強制実行
enable()
collected = collect()
print(f"Collected {collected} cyclic objects")
```

[English](#)

HTTP リファレンス

型

型	説明
HttpRequest	受信 HTTP リクエストの不透明ハンドル
HttpResponse	送信 HTTP レスポンスの不透明ハンドル
HttpClientResponse	HTTP クライアントレスポンスの不透明ハンドル

HttpRequest はサーバーフレームワークから提供されます。HttpResponse は `response()` で作成します。HttpClientResponse はクライアント関数 (`http_get`、`http_post`、`http_request`) から返されます。

関数 (http パッケージ)

これらの関数は明示的なインポートが必要です:

```
from http import listen, method, path, header, body, body_bytes, query, query_all, cookie, cookies, form
```

サーバー

関数	シグネチャ	説明
<code>listen</code>	<code>(host: str, port: int, handler: function(HttpRequest) -> HttpResponse) -> Unit</code>	指定アドレスで HTTP サーバーを起動します。accept ループでブロックし、リクエストごとに <code>handler</code> を呼び出します。

関数	シグネチャ	説明
listen	(host: str, port: int, handler: function(HttpRequest) -> HttpResponse, max_requests: int) -> Unit	max_requests 件のリクエストを処理した後に停止する HTTP サーバーを起動します。async function + block_on() によるライフサイクル管理が可能になります。
listen	(host: str, port: int, handler: function(HttpRequest) -> HttpResponse, max_requests: int, port_callback: function(int) -> Unit) -> Unit	上記と同じですが、bind + listen 成功後に実際にバインドされたポートで port_callback を呼び出します。OS が割り当てるエフェメラルポートを使用するにはポート 0 を指定します。

リクエストアクセサ

関数	シグネチャ	説明
method	(req: HttpRequest) -> str	HTTP メソッドを返します (例: "GET"、"POST")。
path	(req: HttpRequest) -> str	クエリ文字列を除いたリクエストパスを返します (例: "/search?q=hello" の場合 "/search")。
header	(req: HttpRequest, key: str) -> Option<str>	リクエストヘッダーの値を返します (大文字小文字を区別しない検索)。見つからない場合は None を返します。
body	(req: HttpRequest) -> str	リクエストボディを文字列として返します。最初の NUL バイトで切り詰められます。バイナリデータには body_bytes を使用してください。
body_bytes	(req: HttpRequest) -> List<u8>	リクエストボディをバイトリストとして返します。バイナリセーフで、NUL を含むすべてのバイトを保持します。
query	(req: HttpRequest, key: str) -> Option<str>	クエリパラメータの値を返します。見つからない場合は None を返します。値は自動的に URL デコードされます。
query_all	(req: HttpRequest) -> Map<str, str>	すべてのクエリパラメータをマップとして返します。キーと値は自動的に URL デコードされます。
cookie	(req: HttpRequest, name: str) -> Option<str>	名前で指定した Cookie の値を返します。見つからない場合は None を返します。
cookies	(req: HttpRequest) -> Map<str, str>	すべての Cookie をマップとして返します。
form_field	(req: HttpRequest, name: str) -> Option<str>	マルチパートフォームのテキストフィールドの値を返します。見つからない場合は None を返します。
form_file	(req: HttpRequest, name: str) -> Option<Map<str, str>>	ファイルアップロード情報を Option として返します。Some(map) には "filename"、"content_type"、"data" のキーが含まれます。見つからない場合は None を返します。

関数	シグネチャ	説明
form_fields	(req: HttpRequest) -> Map<str, str>	すべてのマルチパートフォームのテキストフィールドをマップとして返します。

レスポンスビルダー

関数	シグネチャ	説明
response	(status: int, headers: Map<str, str>, body: str) -> HttpResponse	指定したステータスコード、ヘッダー、ボディで HTTP レスポンスを作成します。

使用例

基本的な HTTP サーバー

```
from http import listen, method, path, header, body, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    m = method(req)
    p = path(req)
    if p == "/hello":
        return response(200, {"Content-Type": "text/plain"}, "Hello, World!")
    if p == "/echo":
        b = body(req)
        return response(200, {"Content-Type": "text/plain"}, b)
    return response(404, {"Content-Type": "text/plain"}, "Not Found")
)
```

async function によるノンブロッキングサーバー

```
from http import listen, path, response

async function start_server(port: int) -> str:
    listen("127.0.0.1", port, (req: HttpRequest) -> HttpResponse:
        p = path(req)
        if p == "/api/health":
            return response(200, {"Content-Type": "application/json"}, "{\"status\": \"ok\"}")
        return response(404, {"Content-Type": "text/plain"}, "Not Found")
    )
    return "done"
```

```
t = start_server(8080)
# サーバーはバックグラウンドタスクとして実行される
```

リクエスト制限付きサーバー (max_requests)

```
from http import listen, path, response, http_get, status, body

port_holder = [0]
function on_port(p: int) -> Unit:
    port_holder[0] = p

async function start_server() -> str:
```

```

listen("127.0.0.1", 0, (req: HttpRequest) -> HttpResponse:
    return response(200, {"Content-Type": "text/plain"}, "Hello!")
, 1, on_port) # 1 リクエスト後に停止; on_port にバインドされたポートを通知
return "done"

t = start_server()
sleep(100) # サーバーの起動を待機
port = port_holder[0]

case http_get("http://127.0.0.1:" + to_str(port) + "/"):
    Ok(resp):
        print(body(resp)) # "Hello!"
    Err(e):
        print("error")

result = block_on(t) # サーバーは 1 リクエスト後に終了; block_on が完了する

```

クエリパラメータの読み取り

```

from http import listen, path, query, query_all, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    p = path(req)
    if p == "/search":
        case query(req, "q"):
            Some(q):
                return response(200, {"Content-Type": "text/plain"}, "Search: " + q)
            None:
                return response(400, {"Content-Type": "text/plain"}, "Missing query parameter: q")
    return response(404, {"Content-Type": "text/plain"}, "Not Found")
)

```

ヘッダーの読み取り

```

from http import listen, header, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    case header(req, "Authorization"):
        Some(token):
            return response(200, {"Content-Type": "text/plain"}, "Authenticated: " + token)
        None:
            return response(401, {"Content-Type": "text/plain"}, "Unauthorized")
)

```

フォーム送信の処理

```

from http import listen, form_field, form_file, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    case form_field(req, "username"):
        Some(name):
            case form_file(req, "avatar"):
                Some(file_info):
                    filename = file_info["filename"]
                    return response(200, {"Content-Type": "text/plain"}, "Hello " + name + ", file: " +

```

```

        None:
            return response(400, {"Content-Type": "text/plain"}, "No file uploaded")
    None:
        return response(400, {"Content-Type": "text/plain"}, "Missing username")
)

```

Cookie の読み取り

```

from http import listen, cookie, cookies, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    case cookie(req, "session_id"):
        Some(sid):
            return response(200, {"Content-Type": "text/plain"}, "Session: " + sid)
        None:
            return response(401, {"Content-Type": "text/plain"}, "No session")
)

```

動作仕様

- `listen()` はアドレスにバインドし、リスニングを開始して `accept` ループに入ります。
- 引数 3 つで呼び出した場合、`accept` ループは無期限に実行されます。
- 引数 4 つで呼び出した場合 (`max_requests`)、指定した数のリクエストを処理した後にサーバーが停止します。`max_requests` は正の整数でなければなりません。これにより `async function + block_on()` によるライフサイクル管理が可能になります。不正なリクエスト (サイレントにスキップされる) は制限のカウントに含まれません。
- 引数 5 つで呼び出した場合 (`max_requests`, `port_callback`)、`bind + listen` 成功後に `port_callback` が実際にバインドされたポートで同期的に呼び出されます。並列テストでのポート競合を避けるため、ポート 0 (OS が割り当てるエフェメラルポート) と組み合わせて使用できます。
- サーバーはデフォルトで HTTP/1.1 キープアライブをサポートします。単一の接続で複数のリクエストを処理できます。サーバーは各リクエストの `Connection` ヘッダーを確認します: `Connection: close` が送信された場合、レスポンス後に接続が閉じられます。それ以外の場合、接続は後続のリクエストのために維持されます。アイドル接続は 5 秒のタイムアウト後に閉じられます。
- `Content-Length` がヘッダーマップに指定されていない場合、レスポンスに自動的に追加されます。
- サーバーは `Content-Length` ベースのボディ読み取りと `Transfer-Encoding: chunked` デコードを含む HTTP/1.1 をサポートします。
- リクエストに `Transfer-Encoding: chunked` が存在する場合、ボディは自動的にデコード・結合されます。Ry コードは完全なボディを透過的に受け取ります。
- `Transfer-Encoding: chunked` と `Content-Length` の両方が存在する場合、リクエストは不正として拒否されます (RFC 9112 準拠)。
- チャンクレスポンスを送信するには、`response()` に渡すヘッダーマップに `"Transfer-Encoding": "chunked"` を含めます。ボディは自動的にチャンク形式でエンコードされます。
- `header()` によるヘッダー検索は大文字小文字を区別しません。
- `path()` はクエリ文字列を除いたパスを返します。クエリパラメータは `query()` または `query_all()` で別途アクセスします。
- クエリパラメータの値は自動的に URL デコードされます (%20 → スペース、+ → スペース)。
- 重複するクエリパラメータキーの場合、最初の値が返されます。
- `cookie()` と `cookies()` は `Cookie` ヘッダーを; で分割し、各ペアを最初の = で分割してパースします。名前と値の先頭と末尾の空白はトリムされます。
- 重複する `Cookie` 名の場合、最初の値が返されます。
- `Cookie` の値には = 文字を含めることができます (最初の= のみが名前と値の区切りとなります)。
- `form_field()`、`form_file()`、`form_fields()` は `multipart/form-data` リクエストボディをパースします。パースは遅延実行され、最初の呼び出し時にパースされてキャッシュされます。
- `boundary` パラメータは `Content-Type` ヘッダーから抽出され、クォート付きとクォートなしの両方の値をサポートします。

- Content-Disposition に filename を持つパートはファイルアップロードとして扱われ、持たないパートはテキストフィールドとして扱われます。
- 重複するフィールド/ファイル名の場合、最初の値が返されます。
- form_file() は"filename"、"content_type"、"data" のキーを持つ Some(map) を返すか、フィールドが見つからない場合は None を返します。パートに Content-Type が指定されていない場合、デフォルトで"application/octet-stream" になります。
- マルチパートでないリクエストの場合、フォーム関数は None (form_field、form_file) または空のマップ (form_fields) を返します。

サポートされるステータスコード

以下のステータスコードには標準の理由フレーズがあります (RFC 9110) :

コード	理由
100	Continue
101	Switching Protocols
200	OK
201	Created
202	Accepted
203	Non-Authoritative Information
204	No Content
205	Reset Content
206	Partial Content
300	Multiple Choices
301	Moved Permanently
302	Found
303	See Other
304	Not Modified
307	Temporary Redirect
308	Permanent Redirect
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
405	Method Not Allowed
406	Not Acceptable
408	Request Timeout
409	Conflict
410	Gone
411	Length Required
413	Content Too Large
414	URI Too Long
415	Unsupported Media Type
416	Range Not Satisfiable
417	Expectation Failed
422	Unprocessable Content
426	Upgrade Required
429	Too Many Requests
500	Internal Server Error
501	Not Implemented
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout
505	HTTP Version Not Supported

コード	理由
-----	----

その他のステータスコードは理由フレーズとして "Unknown" を使用します。

HTTP クライアント

クライアント関数

関数	シグネチャ	説明
<code>http_get</code>	<code>(url: str) -> Result<HttpClientResponse, Error></code>	指定した URL に HTTP GET リクエストを送信します。
<code>http_post</code>	<code>(url: str, body: str, headers: Map<str, str>) -> Result<HttpClientResponse, Error></code>	ボディとヘッダーを指定して HTTP POST リクエストを送信します。
<code>http_request</code>	<code>(method: str, url: str, headers: Map<str, str>, body: str) -> Result<HttpClientResponse, Error></code>	カスタムメソッドで HTTP リクエストを送信します。

レスポンスアクセサ

関数	シグネチャ	説明
<code>status</code>	<code>(resp: HttpClientResponse) -> int</code>	HTTP ステータスコードを返します。
<code>body</code>	<code>(resp: HttpClientResponse) -> str</code>	レスポンスボディを文字列として返します。最初の NUL バイトで切り詰められます。バイナリデータには <code>body_bytes</code> を使用してください。
<code>body_bytes</code>	<code>(resp: HttpClientResponse) -> List<u8></code>	レスポンスボディをバイトリストとして返します。バイナリセーフで、NUL を含むすべてのバイトを保持します。
<code>header</code>	<code>(resp: HttpClientResponse, key: str) -> Option<str></code>	レスポンスヘッダーの値を返します（大文字小文字を区別しない検索）。見つからない場合は <code>None</code> を返します。
<code>http_client_response_free</code>	<code>(resp: HttpClientResponse) -> Unit</code>	レスポンスと関連メモリを解放します。レスポンスの使用が終わったら呼び出してください。

クライアントの使用例

```
from http import http_get, http_post, status, body, header
```

```
# シンプルな GET リクエスト
```

```
case http_get("http://example.com/api/data"):
    Ok(resp):
        s = status(resp)
        b = body(resp)
```

```

        print(to_str(s) + ": " + b)
Err(e):
    print("Request failed")

# ボディとヘッダー付きの POST リクエスト
headers: Map<str, str> = {"Content-Type": "application/json"}
case http_post("http://example.com/api/data", {"key": "value"}, headers):
    Ok(resp):
        print(body(resp))
    Err(e):
        print("Request failed")

```

クライアントの動作仕様

- `http://` と `https://` の両方の URL をサポートします。HTTPS はシステム CA バンドル証明書検証による TLS を使用します。
- Host ヘッダーは URL に基づいて自動的に追加されます。
- Connection: close が常に送信されます。各リクエストは個別の TCP 接続を使用します。
- Content-Length は常に自動的に追加されます (空のボディには 0 を含む)。ユーザーが指定した Content-Length ヘッダーは正しい値で上書きされます。
- レスポンスボディの読み取りは Content-Length、Transfer-Encoding: chunked、または接続クローズまでの読み取りをサポートします。
- レスポンスヘッダーの検索は大文字小文字を区別しません。
- 接続タイムアウトは 5 秒、受信タイムアウトは 30 秒です。
- 接続失敗、無効な URL、不正なレスポンスの場合は Err を返します。
- `HttpClientResponse` は割り当てられたメモリ (ヘッダー、ボディ) を所有しています。メモリリークを避けるため、使用後に `http_client_response_free()` を呼び出してください。

リダイレクトの動作

HTTP クライアント関数はリダイレクトレスポンス (Location ヘッダー付きの 3xx) を自動的にフォローします。

- サポートされるステータスコード: 301、302、303、307、308
- 最大リダイレクト数: 10 (超過した場合は Err を返す)
- メソッド変換 (RFC 9110 準拠):
 - 301、302: POST は GET に変更される (ボディは破棄)。その他のメソッドは維持
 - 303: メソッドは常に GET に変更される (ボディは破棄)
 - 307、308: メソッドとボディは維持
- URL 解決: Location ヘッダーの絶対 URL、プロトコル相対 (`//...`)、絶対パス (`/...`)、相対パスをサポート
- ユーザー指定のヘッダーは各リダイレクトホップで再送信されますが、機密ヘッダー (`Authorization`、`Proxy-Authorization`、`Cookie`) はクロスオリジンリダイレクト (異なるホストまたはポート) ではストリップされます
- Location ヘッダーが欠落しているか空の場合、リダイレクトレスポンスがそのまま返されます
- 呼び出し元はすべてのリダイレクトをフォローした後の最終レスポンスのみを受け取ります

エラー処理

- `listen()` は `bind()` が失敗した場合 (例: ポートが既に使用中) にランタイムエラーを発生させます。
- 不正なリクエストやキープアライブ接続のアイドルタイムアウトにより接続が閉じられます。その後サーバーは新しい接続の受け入れを再開します。
- ハンドラ関数は常に `HttpResponse` を返す必要があります。デフォルトのレスポンスはありません。
- クライアント関数は `Result<HttpClientResponse, Error>` を返します。成功と失敗を処理するには `case` を使用してください。

I/O 関数リファレンス

標準入出力とファイル操作の関数一覧です。すべての関数は `io` からの明示的なインポートが必要です。

```
from io import read_text, write_text, exists
```

関数一覧

標準入力

関数	シグネチャ	説明
<code>read_line</code>	<code>() -> str</code>	<code>stdin</code> から 1 行読み取り（末尾改行除去）
<code>read_all</code>	<code>() -> str</code>	<code>stdin</code> を EOF まで全読み

ファイル I/O

関数	シグネチャ	説明
<code>read_text</code>	<code>(str) -> Result<str, Error></code>	ファイル全体を文字列として読み取り
<code>write_text</code>	<code>(str, str) -> Result<Unit, Error></code>	ファイルに文字列を書き込み（上書き）
<code>append_text</code>	<code>(str, str) -> Result<Unit, Error></code>	ファイル末尾に文字列を追記
<code>exists</code>	<code>(str) -> bool</code>	ファイル存在チェック
<code>delete_file</code>	<code>(str) -> Result<Unit, Error></code>	ファイル削除
<code>read_bytes</code>	<code>(str) -> Result<List<u8>, Error></code>	ファイルをバイト列として読み取り
<code>write_bytes</code>	<code>(str, List<u8>) -> Result<Unit, Error></code>	バイト列をファイルに書き込み

バイト列変換

関数	シグネチャ	説明
<code>to_bytes</code>	<code>(str) -> List<u8></code>	文字列を UTF-8 バイト列に変換
<code>bytes_to_str</code>	<code>(List<u8>) -> Result<str, Error></code>	バイト列を文字列に変換

使用例

ファイルの読み書き

```
from io import read_text, write_text, append_text, exists, delete_file

case write_text("hello.txt", "Hello, World!"):
    Ok(_):
        case read_text("hello.txt"):
            Ok(content):
                print(content)    # Hello, World!
```

```

        Err(e):
            print(e.message)
    Err(e):
        print(e.message)

print(exists("hello.txt"))    # true

case delete_file("hello.txt"):
    Ok(_):
        print(exists("hello.txt"))    # false
    Err(e):
        print(e.message)

```

バイト操作

```

from io import to_bytes, bytes_to_str, write_bytes, read_bytes

bs = to_bytes("ABC")
print(length(bs))    # 3

case write_bytes("data.bin", bs):
    Ok(_):
        case read_bytes("data.bin"):
            Ok(rb):
                case bytes_to_str(rb):
                    Ok(s):
                        print(s)    # ABC
                    Err(e):
                        print(e.message)
            Err(e):
                print(e.message)
    Err(e):
        print(e.message)

```

標準入力からの読み取り

```

from io import read_line

name = read_line()
print(f"Hello, {name}!")

```

エラーハンドリング

ファイル操作は失敗時に終了するのではなく、`Result<T, Error>` を返します。case で `Ok/Err` パターンを使ってエラーを処理してください:

```

case read_text("missing.txt"):
    Ok(content):
        print(content)
    Err(e):
        print(e.message)    # cannot open file 'missing.txt' for reading

```

操作	エラー条件
<code>read_text / read_bytes</code>	ファイルが存在しない、または開けない

操作	エラー条件
<code>write_text</code> / <code>write_bytes</code> / <code>append_text</code>	ファイルを書き込み用につけていない
<code>delete_file</code>	ファイルを削除できない
<code>bytes_to_str</code>	入力が NUL バイトが含まれている

備考

- `List<u8>` をバッファ型として使用します。標準的なリスト操作 (`length()`、`append()`、`slice()`、インデックスアクセス) がすべてバイトリストで使えます。
- ファイルパスは絶対パスを指定しない限り、カレントディレクトリからの相対パスとなります。
- `write_text` と `write_bytes` は既存ファイルを上書きします。既存ファイルに追記するには `append_text` を使用してください。

[English](#) | [日本語](#) | [繁體中文](#)

JSON 関数リファレンス

JSON のパースとシリアライズを行う関数一覧です。すべての関数は `json` からの明示的なインポートが必要です。

```
from json import parse, stringify, kind, get, at, to_str, to_int, to_float, to_bool, length, keys, json
```

概要

`json` パッケージは、JSON テキストをオペーク（不透明）な `JsonValue` 型にパースし、アクセサ関数で内容にアクセスし、テキストに再シリアライズする機能を提供します。JSON の値は異種型を含みうるため（オブジェクトには文字列、数値、真偽値、配列、ネストされたオブジェクトが含まれる）、オペークポインタ型と型付きアクセサ関数を使用します。

関数一覧

パース / シリアライズ

関数	シグネチャ	説明
<code>parse</code>	<code>(str) -> Result<JsonValue, Error></code>	JSON 文字列を <code>JsonValue</code> にパース
<code>stringify</code>	<code>(JsonValue) -> str</code>	<code>JsonValue</code> をコンパクトな JSON テキストに変換
<code>stringify</code>	<code>(JsonValue, int) -> str</code>	インデント付きで整形出力 (引数はスペース数)

型クエリ

関数	シグネチャ	説明
<code>kind</code>	<code>(JsonValue) -> str</code>	JSON の型を返す: "object", "array", "string", "number", "boolean", "null"

オブジェクト / 配列アクセス

関数	シグネチャ	説明
get	(JsonValue, str) -> Result<JsonValue, Error>	オブジェクトからキーでフィールドを取得
at	(JsonValue, int) -> Result<JsonValue, Error>	配列からインデックスで要素を取得

値の取り出し

関数	シグネチャ	説明
to_str	(JsonValue) -> Result<str, Error>	文字列値を取り出す
to_int	(JsonValue) -> Result<int, Error>	整数値を取り出す
to_float	(JsonValue) -> Result<float, Error>	浮動小数点値を取り出す
to_bool	(JsonValue) -> Result<bool, Error>	真偽値を取り出す

コレクション情報

関数	シグネチャ	説明
length	(JsonValue) -> int	配列の長さまたはオブジェクトのキー数を返す
keys	(JsonValue) -> Result<List<str>, Error>	オブジェクトのキー一覧を返す。値がオブジェクトでない場合はエラー

メモリ管理

関数	シグネチャ	説明
json_free	(JsonValue) -> Unit	JsonValue とその子要素をすべて解放

Result<JsonValue, Error> のアンラップ

parse, get, at は Result<JsonValue, Error> を返します。内側の値を別の json 関数に渡す前に Result をアンラップする必要があります。Result を直接渡すとコンパイル時に拒否されます:

```
case parse(text):
  Ok(doc):
    # ✖ エラー: kind() は JsonValue 引数を要求
    # kind(get(doc, "name"))
    # ✔ 先にアンラップする
    case get(doc, "name"):
      Ok(name_val):
        print(kind(name_val))
      Err(e):
        print("no name")
  Err(e):
    print("parse error")
```

汎用的な文字列化 (to_str(result)、print(result)、f-string 補間) は Result に対してもそのまま動作し、他の Result 値と同様に Ok(...) / Err(...) としてフォーマットされます。

使用例

フィールドのパーズとアクセス

```
from json import parse, get, to_str, to_int, json_free

case parse("{\"name\": \"Alice\", \"age\": 30}"):
    Ok(data):
        case get(data, "name"):
            Ok(val):
                case to_str(val):
                    Ok(name):
                        print(name)    # "Alice"
                    Err(e):
                        print("error")
            Err(e):
                print("error")
        json_free(data)
    Err(e):
        print("parse error: " + e.message)
```

配列の操作

```
from json import parse, at, to_int, length, json_free

case parse("[10, 20, 30]"):
    Ok(data):
        print(to_str(length(data)))    # 3
        case at(data, 0):
            Ok(elem):
                case to_int(elem):
                    Ok(n):
                        print(to_str(n))    # 10
                    Err(e):
                        print("error")
            Err(e):
                print("error")
        json_free(data)
    Err(e):
        print("parse error")
```

整形出力付きシリアライズ

```
from json import parse, stringify, json_free

case parse("{\"key\":\"value\", \"count\":42}"):
    Ok(data):
        print(stringify(data, 2))
        # {
        #   "key": "value",
        #   "count": 42
        # }
        json_free(data)
    Err(e):
        print("error")
```

注意事項

- `to_int` は整数と整数値の浮動小数点数（例: 42.0 → 42）の両方を受け付けます
- `to_float` は浮動小数点数と整数（例: 42 → 42.0）の両方を受け付けます
- `get` と `at` はパースツリー内の子要素への参照を返します。子要素に対して `json_free` を呼ばず、`parse` で返されたルート値に対してのみ呼んでください
- `kind` は整数と浮動小数点数の両方に対して `"number"` を返します

[English](#) | [日本語](#) | [繁體中文](#)

数学関数 (math)

概要

`math` パッケージは数学定数と関数を提供します。`std` パッケージと異なり、自動インポートされません。明示的なインポートで使います。

```
from math import sqrt, PI, sin
```

定数

定数	型	説明
PI	float	円周率 (3.141592653589793)
E	float	ネイピア数 (2.718281828459045)
Inf	float	正の無限大
NaN	float	非数 (Not a Number)

```
from math import PI, E, Inf, NaN
```

```
circumference = 2.0 * PI * radius
```

絶対値

関数	シグネチャ	説明
<code>abs(x)</code>	<code>(int) -> int</code>	整数の絶対値
<code>abs(x)</code>	<code>(float) -> float</code>	浮動小数点数の絶対値

```
from math import abs
```

```
abs(-5)      # 5
abs(-3.14)   # 3.14
```

丸め

関数	シグネチャ	説明
<code>floor(x)</code>	<code>(float) -> int</code>	切り捨て
<code>floor(x, digits)</code>	<code>(float, int) -> float</code>	指定の小数桁数で切り捨て

関数	シグネチャ	説明
<code>ceil(x)</code>	<code>(float) -> int</code>	切り上げ
<code>ceil(x, digits)</code>	<code>(float, int) -> float</code>	指定の小数桁数で切り上げ
<code>round(x)</code>	<code>(float) -> int</code>	四捨五入 (0 から遠い側へ)
<code>round(x, digits)</code>	<code>(float, int) -> float</code>	指定の小数桁数で四捨五入 (0 から遠い側へ)

```
from math import floor, ceil, round
```

```
floor(3.7)      # 3
ceil(3.2)       # 4
round(3.5)      # 4
```

```
round(3.14159, 2) # 3.14
floor(3.789, 1)   # 3.7
ceil(3.123, 1)    # 3.2
```

2 引数形式は負の `digits` を受け付け、10 のべき乗で丸めることができます:

```
round(1234.5, -2) # 1200.0
round(1750.0, -3) # 2000.0
```

丸めは C99 の half-away-from-zero セマンティクス (`round(x * 10digits) / 10digits` として実装) を使い、1 引数形式と一致します。これは Python の banker's rounding とは異なります – 例えば `round(2.675, 2) == 2.68` で、2.67 ではありません。NaN と `±Inf` はそのまま通過します。

冪乗・平方根

関数	シグネチャ	説明
<code>sqrt(x)</code>	<code>(float) -> float</code>	平方根
<code>pow(x, y)</code>	<code>(float, float) -> float</code>	x の y 乗
<code>pow(x, y)</code>	<code>(int, int) -> int</code>	高速累乗を使った整数累乗

```
from math import sqrt, pow
```

```
sqrt(9.0)      # 3.0
pow(2.0, 3.0)   # 8.0
pow(2, 10)      # 1024
pow(-2, 3)      # -8
```

整数オーバーロードは指数が負の場合にランタイムエラーを発生させます (`pow(2, -1)` は `pow() integer exponent must be non-negative` で中断)。オーバーフローは暗黙的にラップされ、Ry 既存の整数演算モデルと一致します。

対数

関数	シグネチャ	説明
<code>log(x)</code>	<code>(float) -> float</code>	自然対数 (底 e)
<code>log(x, base)</code>	<code>(float, float) -> float</code>	任意の底の対数
<code>log2(x)</code>	<code>(float) -> float</code>	底 2 の対数

関数	シグネチャ	説明
<code>log10(x)</code>	<code>(float) -> float</code>	常用対数 (底 10)

```
from math import log
```

```
log(8.0, 2.0)      # 3.0
log(100.0, 10.0)   # 2.0
```

`log(x, base)` は $\log(x) / \log(\text{base})$ として計算されるため、いずれかの引数の定義域エラーは NaN または -Inf として伝播します。

指数

関数	シグネチャ	説明
<code>exp(x)</code>	<code>(float) -> float</code>	e の x 乗

三角関数

関数	シグネチャ	説明
<code>sin(x)</code>	<code>(float) -> float</code>	正弦 (x はラジアン)
<code>cos(x)</code>	<code>(float) -> float</code>	余弦 (x はラジアン)
<code>tan(x)</code>	<code>(float) -> float</code>	正接 (x はラジアン)

逆三角関数

関数	シグネチャ	説明
<code>asin(x)</code>	<code>(float) -> float</code>	逆正弦 (結果はラジアン)
<code>acos(x)</code>	<code>(float) -> float</code>	逆余弦 (結果はラジアン)
<code>atan(x)</code>	<code>(float) -> float</code>	逆正接 (結果はラジアン)

2 引数関数

関数	シグネチャ	説明
<code>atan2(y, x)</code>	<code>(float, float) -> float</code>	y/x の逆正接 (結果はラジアン)
<code>hypot(x, y)</code>	<code>(float, float) -> float</code>	斜辺の長さ: $\sqrt{x^2 + y^2}$

判定関数

関数	シグネチャ	説明
<code>is_nan(x)</code>	<code>(float) -> bool</code>	<code>x</code> が NaN なら <code>true</code>
<code>is_inf(x)</code>	<code>(float) -> bool</code>	<code>x</code> が正または負の無限大なら <code>true</code>

```
from math import is_nan, is_inf, NaN, Inf
```

```
is_nan(NaN)    # true
is_inf(Inf)    # true
is_nan(1.0)    # false
```

[English](#)

ネットワーク (TCP) リファレンス

型

型	説明
<code>TcpListener</code>	TCP サーバソケットの不透明ハンドル
<code>TcpStream</code>	TCP 接続の不透明ハンドル
<code>TlsStream</code>	TLS 暗号化された TCP 接続の不透明ハンドル

すべての型は ARC (自動参照カウント) で管理される不透明ポインタです。直接構築することはできません。`TcpListener/TcpStream` を取得するには `bind()` または `connect()` を使用し、`TlsStream` には `tls_connect()` を使用します。参照がなくなるとリソースは自動的に閉じられます。即座にクリーンアップしたい場合は明示的な `close()` 呼び出しも可能ですが、オプションです。

関数 (net パッケージ)

これらの関数は明示的なインポートが必要です:

```
from net import bind, listen, accept, connect, listener_port, shutdown, set_timeout, set_receive_timeout
```

関数	シグネチャ	説明
<code>bind</code>	<code>(host: str, port: int) -> Result<TcpListener, Error></code>	指定アドレスにバインドされた TCP サーバソケットを作成します。失敗時は <code>Err</code> を返します。動的割り当てにはポート 0 を使用します。
<code>listen</code>	<code>(listener: TcpListener, backlog: int) -> Result<Unit, Error></code>	受信接続のリスニングを開始します。失敗時は <code>Err</code> を返します。
<code>accept</code>	<code>(listener: TcpListener) -> Result<TcpStream, Error></code>	新しい接続を受け入れます。クライアントの接続を最大 1 秒待ちます。タイムアウトまたは失敗時は <code>Err</code> を返します。
<code>connect</code>	<code>(host: str, port: int) -> Result<TcpStream, Error></code>	リモート TCP サーバに接続します。5 秒後にタイムアウトします。タイムアウトまたは失敗時は <code>Err</code> を返します。

関数	シグネチャ	説明
<code>listener_port</code>	<code>(listener: TcpListener) -> int</code>	リスナーが実際にバインドされているポート番号を返します。ポート 0 (OS が割り当てるポート) でバインドした場合に便利です。
<code>shutdown</code>	<code>(listener: TcpListener) -> Unit</code>	リスナーに接続受け入れの停止を通知します。保留中の <code>accept()</code> は最大 1 秒以内に返されます。
<code>tls_connect</code>	<code>(host: str, port: int) -> Result<TlsStream, Error></code>	TLS 暗号化でリモートサーバーに接続します。サーバー証明書をシステム CA バンドルに対して検証します。接続またはハンドシェイク失敗時は <code>Err</code> を返します。
<code>set_timeout</code>	<code>(stream: TcpStream\ TlsStream, ms: int) -> Unit</code>	受信と送信の両方のタイムアウトをミリ秒単位で設定します。
<code>set_receive_timeout</code>	<code>(stream: TcpStream\ TlsStream, ms: int) -> Unit</code>	受信タイムアウトをミリ秒単位で設定します。この時間内にデータが届かない場合、 <code>receive()</code> は <code>Err</code> を返します。
<code>set_send_timeout</code>	<code>(stream: TcpStream\ TlsStream, ms: int) -> Unit</code>	送信タイムアウトをミリ秒単位で設定します。

組み込みオーバーロード関数

これらの関数は組み込みで、TCP ソケット型で動作します。インポートは不要です。

関数	シグネチャ	説明
<code>send</code>	<code>(stream: TcpStream\ TlsStream, data: List<u8>) -> Result<int, Error></code>	TCP または TLS 接続を通じてバイトを送信します。成功時は送信バイト数を含む <code>Ok</code> を返し、失敗時は <code>Err</code> を返します。
<code>receive</code>	<code>(stream: TcpStream\ TlsStream, max: int) -> Result<List<u8>, Error></code>	最大 <code>max</code> バイトを受信します。接続クローズ時は空のリストを含む <code>Ok</code> を返し、エラー時は <code>Err</code> を返します。
<code>close</code>	<code>(handle: TcpStream\ TlsStream) -> Unit</code>	TCP または TLS ストリームを閉じます。
<code>close</code>	<code>(handle: TcpListener) -> Unit</code>	TCP リスナーを閉じます。

使用例

エコーサーバー

```

from net import bind, listen, accept, connect
from io import to_bytes, bytes_to_str

# サーバー
case bind("127.0.0.1", 8080):
    Ok(server):

```

```

    case listen(server, 128):
        Ok(_):
            case accept(server):
                Ok(conn):
                    case receive(conn, 4096):
                        Ok(data):
                            case send(conn, data):
                                Ok(_):
                                    ...
                                Err(e):
                                    print(e.message)
                            Err(e):
                                print(e.message)
                        close(conn)
                    Err(e):
                        ...
                Err(e):
                    print("listen failed")
            close(server)
        Err(e):
            print("bind failed")

```

クライアント

```

case connect("127.0.0.1", 8080):
    Ok(conn):
        case send(conn, to_bytes("hello")):
            Ok(_):
                ...
            Err(e):
                print(e.message)
        case receive(conn, 4096):
            Ok(resp):
                case bytes_to_str(resp):
                    Ok(s):
                        print(s)
                    Err(e):
                        print(e.message)
            Err(e):
                print(e.message)
        close(conn)
    Err(e):
        print("connect failed")

```

async function による並行エコーサーバー

```

from net import bind, listen, accept, connect, listener_port
from io import to_bytes, bytes_to_str

```

```

async function echo_server(server: TcpListener) -> str:
    case accept(server):
        Ok(conn):
            case receive(conn, 4096):
                Ok(data):
                    case send(conn, data):

```

```

        Ok(_):
            ...
        Err(e):
            ...

    Err(e):
        ...
    close(conn)
    Err(e):
        ...
    close(server)
    return "done"

case bind("127.0.0.1", 0):
    Ok(server):
        case listen(server, 1):
            Ok(_):
                port = listener_port(server)
                t = echo_server(server)
                # ... port を使用するクライアントコード ...
                block_on(t)
            Err(e):
                ...
        Err(e):
            ...

```

タイムアウト設定

デフォルトでは、カスタムタイムアウトが設定されていない場合、`receive()` は 30 秒のタイムアウトを使用します。デフォルトを上書きするには `set_timeout()`、`set_receive_timeout()`、または `set_send_timeout()` を使用します:

```

from net import connect, set_receive_timeout

case connect("127.0.0.1", 8080):
    Ok(conn):
        set_receive_timeout(conn, 5000) # 5 秒のタイムアウト
        case receive(conn, 4096):
            Ok(data):
                ...
            Err(e):
                print("timeout or error")
        close(conn)
    Err(e):
        print("connect failed")

```

タイムアウトを無効にする（無期限に待機する）には 0 を渡します。

エラー処理

- `close()` を除くすべての TCP 関数は `Result<T, Error>` を返します。失敗を処理するには `Ok/Err` で `case` を使用してください。
- `receive()` はピアによって接続が閉じられた場合は空の `List<u8>` を含む `Ok` を返し、実際のエラー（タイムアウト、ソケットエラー）の場合は `Err` を返します。
- `close()` はソケットを閉じてハンドルを解放します。閉じた後のハンドルの使用は未定義動作です。

バイト変換

TCP 操作は `List<u8>` を使用します。文字列とバイトリストの変換には `io` パッケージの `to_bytes()` と `bytes_to_str()` を使用してください。

[English](#) | [日本語](#) | [繁體中文](#)

演算子リファレンス

優先順位テーブル

優先順位は数字が小さいほど高い（先に評価される）。

優先順位	演算子	説明	結合性
0	? !!	エラー伝播（後置）	左
1	()	グループ化	一
2	+x -x ~x	単項正・負、ビット NOT	右
3	**	累乗	右
3.5	as	型キャスト	左
4	* / % //	乗算・除算・剰余・整数除算	左
5	+ -	加算・減算	左
6	<< >> >>>	ビットシフト	左
7	&	ビット AND	左
8	^	ビット XOR	左
9	\	ビット OR	左
10	== != < <= > >= in not in	比較・所属	左
11	not	論理 NOT	右
12	and	論理 AND	左
13	or	論理 OR	左
13.5	??	null 合体	左

算術演算子

演算子	説明	例
+	加算 / 文字列結合	1 + 2 -> 3、"a" + "b" -> "ab"、"x" + 1 -> "x1"
-	減算	5 - 3 -> 2
*	乗算 / 文字列繰り返し	4 * 3 -> 12、"ab" * 3 -> "ababab"
/	除算（常に float）	7 / 2 -> 3.5
//	整数除算（-∞ 方向への切り捨て）	7 // 2 -> 3、-7 // 2 -> -4
%	剰余	7 % 3 -> 1
**	累乗（常に float）	2 ** 10 -> 1024.0
-x	単項マイナス	-5, -3.14
+x	単項プラス	+5（符号変更なし）

```
a = 10 // 3    # 3 (int)
b = 10 / 3     # 3.3333... (float)
c = 2 ** 8     # 256.0 (float)
s = "foo" + "bar" # "foobar"
t = "val=" + 42  # "val=42"
u = 3.14 + "!"   # "3.14!"
```

+ の一方のオペランドが `str` で他方が `int`、`float`、`bool` の場合、非 `str` オペランドは自動的にその文字列表現に変換されて結合されます。

比較演算子

すべて `bool` を返す。

演算子	説明
<code>==</code>	等しい
<code>!=</code>	等しくない
<code><</code>	より小さい
<code><=</code>	以下
<code>></code>	より大きい
<code>>=</code>	以上

- 数値型 (`int` / `float`) と `bool` に対して使用可能。
- `str` 同士は辞書順 (バイト順) で比較。
- レコード型はフィールドごとの自動比較で `==` と `!=` をサポート ([構造体リファレンス](#)参照)。
- `in` 演算子はセット、リスト、マップに対する所属チェックに使用 (`x in s`)。
- `not in` 演算子は `in` の否定 (`x not in s`)。
- マップの場合、`in` はキーの存在を確認します。

```
x = 3 < 5          # true
y = "abc" < "abd"  # true (辞書順)
s = {1, 2, 3}
z = 2 in s         # true
w = 4 not in s     # true
xs = [1, 2, 3]
a = 2 in xs        # true (リスト線形探索)
m = {"a": 1}
b = "a" in m       # true (マップキー検索)
```

論理演算子

演算子	説明	型
<code>and</code>	論理 AND	<code>bool x bool -> bool</code>
<code>or</code>	論理 OR	<code>bool x bool -> bool</code>
<code>not</code>	論理 NOT	<code>bool -> bool</code>

```
a = true and false # false
b = true or false  # true
c = not true       # false
```

ビット演算子

`int` 型のみ使用可能。`float` や `bool` に適用するとコンパイルエラー。

演算子	説明	例
<code>&</code>	ビット AND	<code>0b1100 & 0b1010 -> 0b1000</code>
<code>\ </code>	ビット OR	<code>0b1100 \ 0b1010 -> 0b1110</code>
<code>^</code>	ビット XOR	<code>0b1100 ^ 0b1010 -> 0b0110</code>
<code>~</code>	ビット NOT (単項)	<code>~0 -> -1</code>

演算子	説明	例
<<	左シフト	1 << 4 -> 16
>>	算術右シフト	16 >> 2 -> 4
>>>	論理右シフト	-1 >>> 1 -> 9223372036854775807

```
flags = 0b0001 | 0b0010 # 3
masked = flags & 0b0011 # 3
shifted = 1 << 8 # 256
```

エラー伝播演算子 (? / !!)

後置 ? 演算子は happy path で Result や Option 値をアンラップし、unhappy path では短絡します。!! 演算子は ? のエイリアスで、同一のセマンティクスを持ちます。

オペランド	happy path	unhappy path
Result<T, E>	Ok の内側の値 v に評価される	外側の関数から Err(e) を返す
Option<T>	Some の内側の値 v に評価される	外側の関数から None を返す

関数内部で使う場合、オペランドの型は外側の関数の戻り値型に一致する必要があります:

- Result 値に対する ? は、外側の関数が Result を返す必要があります。
- Option 値に対する ? は、外側の関数が Option を返す必要があります。

```
function safe_divide(a: int, b: int) -> Result<int, Error>:
  if b == 0:
    return Err(Error("division by zero"))
  return Ok(a // b)

function compute(a: int, b: int, c: int) -> Result<int, Error>:
  x = safe_divide(a, b)? # b == 0 の場合は Err を早期リターン
  y = safe_divide(x, c)!!
  return Ok(y + 1)

function safe_get(xs: List<int>, i: int) -> Option<int>:
  if i < 0 or i >= xs.length():
    return none
  return Some(xs[i])

function first_plus_second(xs: List<int>) -> Option<int>:
  a = safe_get(xs, 0)? # 範囲外なら None を早期リターン
  b = safe_get(xs, 1)?
  return Some(a + b)
```

トップレベルでの使用

? と !! はスクリプトのトップレベルでも直接使えます。トップレベルでは、Err(e) と None は致命的エラーとして扱われ、エラーメッセージが stderr に書き出され、プロセスがステータス 1 で終了します。

```
function mk() -> Result<int, Error>:
  return Err(Error("something broke"))

v = mk()? # "error: something broke" を stderr に出力してステータス 1 で終了
```

```
x: int? = none
y = x?      # "error: unexpected None" を stderr に出力してステータス 1 で終了
```

トップレベルでは、Result の Err 型は Error でなければなりません (message フィールドを出力できるようにするため)。

case: 条件式

```
x = case:
  condition => true_value
  _ => false_value
```

上から順に条件を評価し、最初に真になったアームの式を返します。すべての結果式は同じ型でなければなりません。_ => ワイルドカードアームは必須なので、式は常に値を生成します。

```
x = case:
  3 > 2 => 10
  _ => 20      # 10
```

```
s = case:
  false => "yes"
  _ => "no"   # "no"
```

ネストされた三項演算は複数のアームにフラット化される

```
score = 85
y = case:
  score >= 90 => 3
  score >= 80 => 2
  _ => 1        # 2
```

範囲演算子

.. 演算子は両端を含む整数の範囲を生成します。

```
xs = 1 .. 5      # [1, 2, 3, 4, 5]
```

```
for i in 1 .. 3:
  print(i)      # 1 2 3
```

結果は左オペランドから右オペランドまで (両端含む) のすべての整数を含む List<int> です。

null 合体演算子 (??)

```
x = optional_val ?? default_val
```

?? 演算子は、左辺に Option<T> または Result<T, E> を受け付けます:

左辺	結果
Some(v)	v
None	default_val
Ok(v)	v
Err(_)	default_val (エラー値は破棄される)

右辺のオペランドは `Option` の内部型（または `Result` の `Ok` 型）と同じ型でなければなりません。

```
a: int? = Some(10)
b: int? = none

print(a ?? 0)    # 10
print(b ?? 0)    # 0

function parse_int(s: str) -> Result<int, Error>:
    # ...

i = parse_int("42") ?? -1    # 成功なら 42, Err なら -1
j = parse_int("nope") ?? -1  # -1 -- Err 値は破棄される
```

複合代入演算子

変数を更新するショートハンド。 `x op= y` は `x = x op y` と等価。

演算子	等価な式
<code>x += y</code>	<code>x = x + y</code>
<code>x -= y</code>	<code>x = x - y</code>
<code>x *= y</code>	<code>x = x * y</code>
<code>x /= y</code>	<code>x = x / y</code>
<code>x %= y</code>	<code>x = x % y</code>
<code>x //= y</code>	<code>x = x // y</code>
<code>x **= y</code>	<code>x = x ** y</code>
<code>x &= y</code>	<code>x = x & y</code>
<code>x \ = y</code>	<code>x = x \ y</code>
<code>x ^= y</code>	<code>x = x ^ y</code>
<code>x <<= y</code>	<code>x = x << y</code>
<code>x >>= y</code>	<code>x = x >> y</code>

```
x = 10
x += 5    # x = 15
x -= 3    # x = 12
x *= 2    # x = 24
x /= 3    # x = 8
x //= 3   # x = 2
x &= 6    # x = 0
```

複合代入は任意の lvalue に対して許可されています – 通常の変数、リストやマップの要素、record のフィールド、任意のネストしたチェーンに使えます:

```
xs = [1, 2, 3]
xs[0] += 10    # リスト要素

record Point:
    x: int
    y: int
p = Point(1, 2)
p.x *= 5       # record フィールド

pts = [Point(1, 2), Point(3, 4)]
pts[0].x -= 1  # チェーン: リスト中の record フィールド
```

チェーン LHS 上の各インデックス式はちょうど 1 回だけ評価されます。存在しないマップキーに対する複合代入 (`m["absent"] += 1`) はランタイムエラーになります。

インクリメント・デクリメント演算子

変数を 1 増減させるポストフィックス演算子。ステートメントとしてのみ使用可能。内部的にはそれぞれ `x = x + 1`、`x = x - 1` にデシュガーされる。

演算子	等価な式
<code>x++</code>	<code>x = x + 1</code>
<code>x--</code>	<code>x = x - 1</code>

```
count = 0
count++      # count = 1
count++      # count = 2
count--      # count = 1

f = 1.5
f++          # f = 2.5 (int 1 が float に型昇格)
```

注意: `++` / `--` はステートメントとしてのみ使用でき、式の中では使えません。`@const` 変数にはインクリメント・デクリメントを適用できません（不変性が強制されます）。

演算の型規則

演算	左辺型	右辺型	結果型
<code>+</code> <code>-</code> <code>*</code>	int	int	int
<code>+</code> <code>-</code> <code>*</code>	float	int / float	float
<code>+</code> <code>-</code> <code>*</code>	int	float	float
<code>/</code>	任意の数値	任意の数値	float
<code>//</code>	int	int	int
<code>//</code>	float または int (片方 float)	-	float
<code>**</code>	任意の数値	任意の数値	float
<code>%</code>	int	int	int
<code>%</code>	float または int (片方 float)	-	float
<code>+</code>	str	str	str
<code>+</code>	str	int / float / bool	str
<code>+</code>	int / float / bool	str	str
<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>	数値 / bool / str	同型	bool
<code>*</code>	str	int	str
<code>in</code>	任意	Set / List / Map<K, V>	bool
<code>not in</code>	任意	Set / List / Map<K, V>	bool
<code>&</code> <code> </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code> <code>>>></code>	int	int	int
<code>and</code> <code>or</code> <code>not</code>	bool	bool	bool

演算子オーバーロード

ユーザー定義型に対して演算子の動作を定義できます。

構文

```
# 二項演算子 (引数 2 個)
function operator+(a: MyType, b: MyType) -> MyType:
    ...

# 単項演算子 (引数 1 個)
function operator-(a: MyType) -> MyType:
    ...
```

オーバーロード可能な演算子一覧

種別	演算子
算術 (二項)	+ - * / % ** //
比較 (二項)	== != < <= > >=
ビット (二項)	& \ ^ << >> >>>
論理 (二項)	and or
所属	in
添字	[] (読み取り)、[] = (書き込み)
呼び出し	()
キャスト	as
単項	- ~ not
複合代入	+= -= *= /= %= // = **= &= \ = ^= <<= >>=

戻り値型の制約

比較演算子と論理演算子は bool を返す必要があります:

種別	演算子	必須戻り値型
比較	== != < <= > >=	bool
論理	and or not	bool
所属	in	bool
キャスト	as	必須 (ターゲット型)

```
# OK
function operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y

# エラー: comparison operator '==' must return 'bool', but returns 'int'
function operator==(a: Vec2, b: Vec2) -> int:
    return 42
```

算術演算子・ビット演算子には戻り値型の制約はありません。

二項 / 単項の区別

引数の個数で区別します。

```
# 二項 -
function operator-(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x - b.x, a.y - b.y)

# 単項 -
function operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

複合代入演算子のオーバーロード

複合代入演算子（+=、-= 等）は個別にオーバーロードできます。大きなデータ構造のインプレース最適化を可能にします。

```
record Matrix:
    data: List
    rows: int
    cols: int

function operator+=(a: Matrix, b: Matrix) -> Matrix:
    for i in range(len(a.data)):
        a.data[i] = a.data[i] + b.data[i]
    return a
```

解決優先順位 x += y が評価される際:

1. operator+= がその型に定義されている場合 → 直接呼び出す
2. operator+= が未定義で operator+ がある場合 → $x = x + y$ にフォールバック
3. どちらも未定義の場合（組み込み型以外）→ コンパイルエラー

```
record Vec2:
    x: float
    y: float

function operator+=(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)
```

```
v = Vec2(1.0, 2.0)
v += Vec2(3.0, 4.0) # operator+= を直接呼び出す
# v.x == 4.0, v.y == 6.0
```

複合代入演算子は引数が正確に 2 つ必要で、戻り値型の制約はありません。

添字演算子のオーバーロード

[]（読み取り）と []=（書き込み）演算子でユーザー定義型にカスタムの添字アクセスを定義できます。複数インデックスアクセス（例: `m[row, col]`）もサポートされています。

```
record Grid:
    a: int
    b: int
    c: int
    d: int

# 読み取り: 2個以上のパラメータ（オブジェクト + インデックス）が必要
function operator[](g: Grid, row: int, col: int) -> int:
    if row == 0 and col == 0:
        return g.a
    if row == 0 and col == 1:
        return g.b
    if row == 1 and col == 0:
        return g.c
    return g.d
```

```
# 書き込み: 3個以上のパラメータ（オブジェクト + インデックス + 値）が必要
function operator[]=(g: Grid, row: int, col: int, value: int):
```

...

```
g = Grid(1, 2, 3, 4)
print(g[0, 1])    # 2
g[1, 0] = 99
```

ユーザー定義の添字演算子が最初に試行され、一致しない場合は組み込みの添字動作（リスト、マップ、配列）がフォールバックとして使用されます。

所属演算子のオーバーロード

`in` 演算子をオーバーロードしてカスタムの所属チェックを定義できます。`bool` を返す必要があります。

```
record Range:
  lo: int
  hi: int

function operator in(value: int, r: Range) -> bool:
  return value >= r.lo and value < r.hi
```

```
r = Range(1, 10)
print(5 in r)      # true
print(15 not in r) # true
```

ユーザー定義の `in` 演算子が最初に試行され、一致しない場合は組み込みの動作（セット、マップ、リスト）がフォールバックとして使用されます。`in` が定義されていれば `not in` は自動的にサポートされます。

呼び出し演算子のオーバーロード

() 演算子でレコードを呼び出し可能なオブジェクトとして動作させることができます。2 個以上のパラメータ（オブジェクト+ 引数）が必要です。

```
record Adder:
  base: int

function operator()(a: Adder, x: int) -> int:
  return a.base + x
```

```
add5 = Adder(5)
print(add5(10))    # 15
```

レコード値を保持する変数を関数のように呼び出すと、コンパイラはまず `operator()` のオーバーロードを試みます。一致しない場合は、他の呼び出し解決戦略（関数、コンストラクタ、ラムダ）が優先されます。

キャスト演算子のオーバーロード

`as` 演算子をオーバーロードしてカスタムの型変換を定義できます。パラメータは正確に 1 つ（ソース値）で、戻り値型（ターゲット型）の指定が必要です。ソース型と戻り値型でディスパッチされます。

```
record Celsius:
  value: int

record Fahrenheit:
  value: int

function operator as(c: Celsius) -> Fahrenheit:
  return Fahrenheit(c.value * 9 // 5 + 32)
```

```
c = Celsius(100)
f = c as Fahrenheit    # Fahrenheit(212)
```

ターゲット型はコンパイラが解決できる任意の型で、ジェネリック型を含みます:

```
record Temperature:
  value: int

function operator as(t: Temperature) -> int?:
  return Some(t.value)
```

```
t = Temperature(42)
result: int? = t as int?    # Some(42)
```

ユーザー定義の `as` 演算子が最初に試行され、一致しない場合は組み込みのキャスト (`int`、`float`、`bool`、`str` 等) がフォールバックとして使用されます。

[English](#) | [日本語](#) | [繁體中文](#)

パッケージリファレンス

概要

Ry はパッケージシステムでコードを管理します。パッケージは単一の `.ry` ファイル、またはディレクトリ (複数の `.ry` ファイルを含む) のいずれかです。 `from` 文でパッケージをインポートします。

`std` パッケージ (標準ライブラリ) はすべてのプログラムに自動的にインポートされます。

インポート構文

全定義インポート

```
from math
```

パッケージ内のすべての関数・型をインポートします。

選択インポート

```
from math import sqrt
```

指定した定義のみをインポートします。

複数選択インポート

```
from math import sqrt, PI
```

カンマ区切りで複数の定義を選択インポートします。

相対インポート

```
from .helper import greet
```

現在のファイルのディレクトリからの相対パスでモジュールをインポートします。 `.` プレフィックスにより、解決は現在のディレクトリだけに制限されます (標準ライブラリやその他の検索パスは検索されません)。

サブディレクトリからの相対インポート

```
from .utils import helper_fn
from .utils.calc import add
```

現在のファイルのディレクトリからの相対パスでサブディレクトリからインポートします。

カレントディレクトリからの全相対インポート

```
from . import add, sub
```

カレントディレクトリパッケージ（ディレクトリ内のすべての .ry ファイル、_ プレフィックスと .test.ry ファイルを除く）から特定のシンボルをインポートします。

パッケージ解決

ドット区切りのパッケージ名は以下のように解決されます:

インポート文	解決先
from math	math/ ディレクトリ（パッケージ）または math.ry ファイル
from utils.math	utils/math/ ディレクトリまたは utils/math.ry ファイル
from str	str/ ディレクトリまたは str.ry ファイル

解決順序

各検索パスに対して: 1. ディレクトリ ({path}/) — 存在すればディレクトリ内の全 .ry ファイルを読み込む (パッケージ) 2. ファイル ({path}.ry) — 単一ファイル (後方互換)

ディレクトリパッケージ

パッケージがディレクトリに解決された場合: - ディレクトリ内のすべての .ry ファイルが自動的に読み込まれる - _ で始まるファイルは除外される - テストファイル (.test.ry) は除外される - 特別なエントリファイル (__init__.py のようなもの) は不要 - ディレクトリ内のファイルで定義されたすべての関数・型がエクスポートされる

プライベートシンボル

名前が _ (アンダースコア) で始まる定義はパッケージ内部のプライベートシンボルとして扱われ、インポートできません:

- ワイルドカードインポート (from pkg) では _ プレフィックスのシンボルが自動的に除外される
- 名前指定インポート (from pkg import _helper) はコンパイルエラーになる

```
# mylib/internal.ry
function _helper() -> int:      # プライベート — インポート不可
    return 42
function public_api() -> int:   # パブリック — インポート可能
    return _helper()

mypackage/
  calc.ry      # function add(), function sub()
  string.ry    # function concat()
```

```
from mypackage          # add, sub, concat をインポート
from mypackage import add # add のみインポート
```

標準ライブラリ (std)

std パッケージはすべてのプログラムに自動的にインポートされます。提供する機能: - 組み込み関数 (print, length, range など) - 文字列関数 (contains, find, replace など) - 型変換関数 (to_int, to_float, to_str) - コレクション関数 (map, filter, sort など)

サブパッケージ

以下のサブパッケージは明示的なインポートが必要です:

パッケージ	説明
<code>math</code>	数学定数・関数
<code>io</code>	ファイル I/O・標準入力・バイト変換
<code>path</code>	ファイルパス操作 (join, basename, dirname 等)

```
from math import sqrt, PI, sin
```

標準ライブラリのパッケージから特定の定義を明示的にインポートすることもできます:

```
from str import contains
```

RY_HOME

標準ライブラリは \$RY_HOME/share/std/ にインストールされます。RY_HOME のデフォルト値は ~/.ry です。

```
export RY_HOME="$HOME/.ry" # デフォルト
```

RY_ENV

RY_ENV 環境変数でランタイム環境モードを制御します。--env=<value> CLI フラグでも指定可能です。

値	エイリアス	.env 読み込み	lib 探索
prod	production	無効	リポジトリビルド用プロジェクトオーバーライド → \$RY_HOME/share (フォールバック: lib) → exe/./share (フォールバック: lib) → exe/share (フォールバック: lib)
dev	development	.env.dev → .env	prod と同じ
test	—	.env.test → .env	prod と同じ
staging	—	.env.staging → .env	prod と同じ
internal	—	.env.internal → .env	リポジトリビルド用プロジェクトオーバーライド → exe/./share (フォールバック: lib) → exe/share (フォールバック: lib) (\$RY_HOME スキップ)

値	エイリアス	.env 読み込み	lib 探索
(未設定) (デフォルト)	—	.env のみ	prod と同じ

エイリアスは自動的に正規形に解決されます。例えば `RY_ENV=production` は `prod` に正規化されます。

`prod` モードではセキュリティのため `.env` ファイルを読み込みません。本番環境の秘密情報はインフラレベルの環境変数管理 (CI/CD、シークレットマネージャー等) で管理してください。

その他のモードでは `.env.<環境名>` を先に読み込み (存在する場合)、次に `.env` を読み込みます。既存の環境変数は上書きされないため、環境別の値が優先されます。

短縮形 (推奨)

```
RY_ENV=dev ./build/ry app.ry
```

フルネーム (後方互換)

```
RY_ENV=development ./build/ry app.ry
```

CLI フラグ

```
./build/ry --env=dev test
```

`prod` モード: `.env` は読み込まれない

```
RY_ENV=prod ./build/ry app.ry
```

`Ry` 自体の開発時の追加 *isolation*

```
RY_ENV=internal ./build/ry test
```

`Ry` のソースツリー内でビルドされた `ry` 実行バイナリは、プロジェクトの `package.toml` からリポジトリローカルの `stdlib` オーバーライドを使用できます。これにより、`~/.ry/share/std` が古い場合でも、チェックアウトされた `share/std` とリポジトリビルドの整合性が保たれます。インストール済みの `ry` バイナリはこのオーバーライドを無視し、`$RY_HOME/share/std` を引き続き使用します。

検索パスの優先順位

1. インポート元ファイルのディレクトリ
2. 現在の `Ry` チェックアウトからのリポジトリローカル `stdlib` オーバーライド (リポジトリビルドの `ry` 使用時)
3. `$RY_HOME/share` (標準ライブラリの場所、レガシーインストールでは `$RY_HOME/lib` にフォールバック)
4. 実行ファイル相対の `share/` ディレクトリ (レガシーレイアウトでは `lib/` にフォールバック)
5. `RY_PATH` 環境変数に含まれるパス (コロン区切り)

RY_PATH 環境変数

`RY_PATH` にコロン区切りでディレクトリを指定すると、パッケージ検索パスに追加されます。

```
export RY_PATH="/usr/local/ry/lib:/home/user/ry-packages"
```

制約

制約	詳細
使用可能な位置	トップレベルのみ（関数・ブロック内は不可）
二重インポート	自動でスキップ（エラーにならない）
循環インポート	コンパイルエラー
相対インポート	<code>from .</code> と <code>from .pkg</code> は現在のファイルのディレクトリのみに対して解決される
親ディレクトリインポート	<code>from ..</code> は未対応
パッケージ名	アルファベット、数字、アンダースコアのみ使用可能（ハイフンは不可）

```
# エラー例：ブロック内でのインポート
function main():
    from math    # エラー：トップレベル以外ではインポート不可

# OK: 同じパッケージを複数回インポートしてもエラーにならない
from math
from math    # スキップされる
```

パッケージファイルの作成

単一ファイルパッケージ

```
# calc.ry
function add(a: int, b: int) -> int:
    return a + b

function sub(a: int, b: int) -> int:
    return a - b

# main.ry
from calc import add, sub

print(add(1, 2))    # 3
print(sub(5, 3))    # 2
```

ディレクトリパッケージ

```
mylib/
  calc.ry
  string.ry

# main.ry
from mylib import add, concat
```

Native 関数の命名規約

C ランタイム関数として実装される stdlib パッケージの関数は、`__ry_<package>_<function_name>` 規約に従います。

注意: この規約は stdlib パッケージ関数 (例: `base64`, `filesystem`, `path`) に適用されます。組み込み関数 (例: `print`, `length`) や `math` 関数は実装がさまざま (インライン LLVM IR、libc 呼び出し等) で、この命名パターンには従いません。

フォーマット

`__ry_<package>_<function_name>`

ルール

1. **プレフィックス:** `__ry_`
2. **パッケージ:** パッケージ名 (例: `from base64 import encode` なら `base64`)
3. **関数名:** Ry で宣言されている `snake_case` の関数名
4. **オーバーロード:** 関数がアリティの異なる複数のオーバーロードを持つ場合、引数数をサフィックスとして付加する (例: `__ry_path_join2`, `__ry_path_join3`)
5. **エラーゲッター:** `Result` 型を返す各パッケージは `__ry_<pkg>_get_last_error` を提供する (手書きせず `include/ry/runtime_error.hpp` の `DEFINE_LAST_ERROR(pkg)` マクロで生成することが多い)

例

Ry 宣言	C ランタイム関数名
<code>base64::encode(data: str) -> str</code>	<code>__ry_base64_encode</code>
<code>filesystem::list_dir(path: str) -> Result<List<str>, Error></code>	<code>__ry_filesystem_list_dir</code>
<code>path::join(a: str, b: str) -> str</code>	<code>__ry_path_join2</code>
<code>path::join(a: str, b: str, c: str) -> str</code>	<code>__ry_path_join3</code>

[English](#) | [日本語](#) | [繁體中文](#)

パス関数リファレンス

ファイルパスの操作を行います。すべての関数は `path` からの明示的なインポートが必要です。

```
from path import join, basename, dirname, extension, resolve, is_absolute
```

関数一覧

関数	シグネチャ	説明
<code>join</code>	<code>(str, str) -> str</code>	2つのパスセグメントを結合
<code>join</code>	<code>(str, str, str) -> str</code>	3つのパスセグメントを結合
<code>join</code>	<code>(str, str, str, str) -> str</code>	4つのパスセグメントを結合
<code>basename</code>	<code>(str) -> str</code>	ファイル名コンポーネントを抽出
<code>dirname</code>	<code>(str) -> str</code>	ディレクトリコンポーネントを抽出
<code>extension</code>	<code>(str) -> str</code>	ファイル拡張子を抽出 (ドットを含む)
<code>resolve</code>	<code>(str) -> Result<str, Error></code>	パスを絶対正規形に解決
<code>is_absolute</code>	<code>(str) -> bool</code>	パスが絶対パスかどうかを返す

使用例

パスの結合

```
from path import join

p = join("/tmp", "data", "file.txt")
print(p)  # /tmp/data/file.txt

# 第2引数が絶対パスの場合、第1引数を置き換える
print(join("/tmp", "/usr"))  # /usr
```

パスコンポーネントの抽出

```
from path import basename, dirname, extension

p = "/home/user/docs/report.pdf"

print(basename(p))  # report.pdf
print(dirname(p))   # /home/user/docs
print(extension(p)) # .pdf
```

拡張子のエッジケース

```
from path import extension

print(extension("archive.tar.gz"))  # .gz
print(extension(".gitignore"))      # (空文字列 — 拡張子のない隠しファイル)
print(extension(".config.json"))    # .json
print(extension("Makefile"))        # (空文字列)
```

絶対パスの判定

```
from path import is_absolute

print(is_absolute("/usr/local"))  # true
print(is_absolute("src/main.py")) # false
```

パスの解決

```
from path import resolve

case resolve("/tmp"):
    Ok(p):
        print(p)  # /private/tmp (on macOS) or /tmp
    Err(e):
        print(e.message)

case resolve("/nonexistent"):
    Ok(p):
        print(p)
    Err(e):
        print(e.message)  # cannot resolve path '/nonexistent': No such file or directory
```

[English](#) | [日本語](#) | [繁體中文](#)

プロジェクト管理

CLI 概要

```
ry <file.ry> [args...]           # Ry スクリプトを実行
echo '<code>' | ry -c             # 標準入力からコードを実行
ry test [options] [<file> | <dir>] # テストを実行
ry init                          # プロジェクトを初期化
ry new <project-name>            # 新規プロジェクトを作成
ry run [<script-name>]           # プロジェクトスクリプトを実行
ry fmt [options] [<file> | <dir>] # ソースファイルをフォーマット
ry self-update [options]         # ry 自身を更新
```

グローバルオプション

オプション	説明
-c	標準入力からコードを読み取って実行
-h, --help	ヘルプを表示
-v, --version	バージョンを表示
--env=<env>	環境を設定 (production development internal)。 RY_ENV 環境変数を上書きする。

エントリポイント実行

ファイル引数が指定されていない場合、ry はカレントディレクトリ（または親ディレクトリ）の package.toml を検索し、entry フィールドで指定されたファイルを実行します:

```
ry                               # エントリファイルを実行 (例: src/main.ry)
ry -- arg1 arg2                  # 引数付きでエントリファイルを実行
```

package.toml が見つからない場合、または entry フィールドが設定されていない場合、ry はヘルプを表示して終了します。

ベースファイル名

最初の引数が **単一パスコンポーネント** で、名前が .ry で終わる場合 (例: main.ry)、かつカレントディレクトリにその名前のファイルが存在しない場合、ry は最も近い package.toml プロジェクトを次の順序で検索します: **最初に** プロジェクトルートディレクトリ (package.toml を含むディレクトリ)、**次に** [paths] に列挙された各ディレクトリ (キーはアルファベット順、_ で始まるキーはこの検索では予約済みで無視されますが、_dev_stdlib は別に扱われます)。最初に存在する通常ファイルが採用されます (例: src/foo.ry と foo.ry の両方が存在する場合、package.toml と同じディレクトリの foo.ry が選ばれます)。どれも一致しない場合、ry はファイルが存在しないことと試したパスの一覧を報告します。.ry サフィックスを持たないトークンはこの方法で解決されません (つまり、スペルミスしたサブコマンドは引き続き “unknown command” と表示されます)。

引数が **2 つ以上のコンポーネントを持つパス** (例: src/foo.ry や ./foo.ry) で、そのパスが存在しない場合、ry は未知のサブコマンドとして扱うのではなく **no such file** を報告します。

同じルールは、<file.ry> がベース名で、カレントディレクトリ相対では存在しない場合の ry test <file.ry> にも適用されます。

標準入力からの実行

-c フラグを使用して標準入力からコードを読み取って実行します:

```
echo 'print("hello")' | ry -c
```

ry init - プロジェクト初期化

カレントディレクトリを Ry プロジェクトとして初期化します。

```
ry init
```

生成されるファイル・ディレクトリ

```
my-project/  
  package.toml      # プロジェクト設定ファイル  
  src/  
    main.ry         # エントリポイント (サンプルコード)
```

動作

1. package.toml が既に存在する場合はエラー終了する
 2. src/ ディレクトリを作成 (存在しなければ)
 3. package.toml を生成 (name はカレントディレクトリ名)
 4. src/main.ry を生成 (既に存在する場合はスキップ)
-

ry new - 新規プロジェクト作成

新しいディレクトリを作成し、Ry プロジェクトとして初期化します。

```
ry new my-project
```

生成されるファイル・ディレクトリ

```
my-project/  
  package.toml      # プロジェクト設定ファイル  
  src/  
    main.ry         # エントリポイント (サンプルコード)
```

動作

1. プロジェクト名が指定されていない場合はエラー終了する
 2. 同名のディレクトリが既に存在する場合はエラー終了する
 3. <project-name>/ ディレクトリを作成
 4. その中に src/ ディレクトリを作成
 5. package.toml を生成 (name は指定されたプロジェクト名)
 6. src/main.ry を生成
-

ry run - プロジェクトスクリプトの実行

package.toml の [scripts] セクションに定義されたスクリプトを実行します。

```
ry run          # 利用可能なスクリプトを一覧表示  
ry run build    # "build" スクリプトを実行  
ry run test     # "test" スクリプトを実行
```

動作

1. カレントディレクトリから上方向に package.toml を検索
2. 引数なしの場合、利用可能なスクリプトとそのコマンドを一覧表示
3. スクリプト名が指定された場合、対応するシェルコマンドを /bin/sh -c で実行

4. 実行したコマンドの終了コードが伝播される
5. スクリプト名が見つからない場合、利用可能なスクリプト一覧とともにエラーを表示

注意事項

- LLVM の初期化は不要（高速起動）
 - コマンドはカレントワーキングディレクトリで実行される
 - コマンドはシェル経由で実行されるため、シェル機能（パイプ、リダイレクト等）が使用可能
-

ry fmt - コードフォーマッター

.ry ソースファイルを一貫した 2 スペースインデントと正規スタイルでフォーマットします。

```
ry fmt                # プロジェクト内の全 .ry ファイルをフォーマット (package.toml 必要)
ry fmt src/main.ry    # 単一ファイルをフォーマット
ry fmt src/           # ディレクトリ内の全 .ry ファイルを再帰的にフォーマット
ry fmt --check         # フォーマット済みか確認 (未フォーマットなら exit 1)
ry fmt --check src/    # 特定ディレクトリを確認
```

フォーマットルール

- ブロックレベルごとに 2 スペースインデント
- 二項演算子の前後にスペース (a + b、a+b ではない)
- カンマの後にスペース (f(a, b)、f(a,b) ではない)
- トップレベル定義間（関数、レコード、列挙型）に空行
- コメントは保持される

動作

1. ソースファイルを読み取り、AST にパースし、正規フォーマットで再出力する
2. フォーマット結果をファイルに書き戻す（インプレース）
3. --check 指定時は未フォーマットファイルを報告し、存在する場合はコード 1 で終了（CI 向け）
4. 再帰フォーマット時は .git/、build/、node_modules/ ディレクトリをスキップ

注意事項

- LLVM の初期化は不要（高速起動）
 - 複合代入演算子（+=、-= 等）はパーサーがデシュガーするため、フォーマット後は展開形 (x = x + expr) になる
 - 16 進数 (0xFF) ・ 2 進数 (0b1010) リテラルは 10 進数表記に変換される
-

ry test - テスト実行

テストファイル (*.test.ry) を探索して実行します。テスト構文の詳細は[テスト](#)を参照。

```
ry test                # *.test.ry ファイルを自動検出して全テスト実行
ry test tests/spec     # ディレクトリ内の全テストを実行
ry test test_file.ry   # 特定のテストファイルを実行
ry test -p             # テストを並列実行
ry test -w             # ウォッチモード: ファイル変更時に再実行
ry test --coverage     # 行カバレッジ情報を収集
```

オプション

オプション	説明
<code>-p, --parallel</code>	テストを並列実行
<code>-w, --watch</code>	ファイル変更を監視して再実行
<code>--coverage, --cov</code>	カバレッジ情報を収集
<code>-h, --help</code>	ヘルプを表示

動作

1. 引数なしの場合、`package.toml` を検索してプロジェクトルートを見つけ、`*.test.ry` ファイルを再帰的に探索する（`.git`、`build`、`node_modules` はスキップ）
2. 全テスト成功なら終了コード 0、失敗があれば 1
3. `--coverage` と `--parallel` を同時に指定すると順次実行にフォールバック

ry self-update - セルフアップデート

ry 自身を最新バージョンに更新します。GitHub Releases からバイナリをダウンロードし、現在の実行ファイルを置換します。

```
ry self-update          # 最新安定版に更新
ry self-update --nightly # 最新 nightly プレリリースに更新
ry self-update v0.0.1   # 指定バージョンに更新
```

動作

1. 現在のバージョンを表示
2. 引数に応じて更新対象のバージョンを解決
 - 引数なし: GitHub の最新安定リリース（`/releases/latest`）
 - `--nightly`: 最新のプレリリース
 - バージョン指定: 指定されたタグのリリース
3. 現在のバージョンと同じ場合は "Already up to date." で終了
4. バイナリをダウンロードして現在の実行ファイルを置換

セキュリティ

リリースアーカイブは 2 段階で検証されます:

1. **真正性**: `checksums.txt.sig` ファイルを組み込みの Ed25519 公開鍵で検証します。
 - 署名ファイルが**存在しない**場合、`RY_SKIP_SIGNATURE=1` が設定されていない場合はアップデートは中止されます。
 - 署名ファイルが**存在するが無効**な場合、`RY_SKIP_SIGNATURE` の設定に関係なくアップデートは中止されます。
2. **完全性**: アーカイブの SHA-256 ハッシュを `checksums.txt` と照合します。

署名ファイルが存在しない場合に限りアップデートを続行するには（非推奨）、`RY_SKIP_SIGNATURE=1` を設定してください。署名ファイルが存在し検証に失敗した場合は、この環境変数に関係なくアップデートは中止されます。

注意事項

- 実行には `curl` および `tar` コマンドが必要です
- パーミッション不足でバイナリの置換に失敗した場合、`sudo` の使用を促すメッセージが表示されます（`sudo` は自動的に実行されません）

- ダウンロードはテンポラリディレクトリに対して行われますが、ファイルシステムをまたぐ cp フォールバックが中断された場合、バイナリが不完全な状態になる可能性があります

package.toml 設定ファイル

プロジェクトのメタデータとパス設定を TOML 形式で記述します。

```
[project]
name = "my-project"
version = "0.1.0"
entry = "src/main.ry"

[paths]
src = "src"

[scripts]
build = "cmake --preset default && cmake --build build"
test = "./build/ry_tests"
clean = "rm -rf build"
```

[project] セクション

キー	説明
name	プロジェクト名（初期化時はディレクトリ名）
version	バージョン文字列
entry	エントリポイントとなるソースファイル

[paths] セクション

キー	説明
src (その他のキー)	ソースコードディレクトリ プロジェクト相対の追加ディレクトリ。値は絶対パスであってはならず、.. を含めることはできません。 src と合わせて、これらのディレクトリは ry <file> と ry test <file> のベースファイル名を解決するために使われます（上記「ベースファイル名」を参照）。
_dev_stdlib	任意。標準ライブラリ位置の開発オーバーライド（ツーリングドキュメントを参照）。ベースファイル名の解決には使われません。

[scripts] セクション

ry run <name> で実行できる名前付きスクリプトを定義します。各キーがスクリプト名で、値はシェルコマンド文字列です。

キー	説明
<name>	実行するシェルコマンド（ry run <name> で実行）

TOML サブセット仕様

package.toml は以下の TOML サブセットをサポートします。

- セクションヘッダ: [section]
- キー・値ペア: key = "value" (文字列値のみ)
- コメント: # から行末まで
- 空行は無視される

[English](#) | [日本語](#) | [繁體中文](#)

正規表現リファレンス

正規表現リテラル構文

正規表現リテラルは /pattern/ 構文を使用し、Regex 型の値を生成します:

```
from regex import is_match, split, replace

# 正規表現リテラルにより型ベースのオーバーロードが有効になる
"hello".is_match(/[a-z]+)/          # true
"a1b2c".split(/[0-9]/)              # ["a", "b", "c"]
"abc123".replace(/[0-9]+/, "X")    # "abcX"
```

正規表現リテラルは変数に格納できます:

```
pat = /[a-z]+/
"hello".is_match(pat)  # true
```

正規表現リテラル内の / は \ / でエスケープできます:

```
"a/b".is_match(/a\b/)  # true
```

除算と正規表現の区別

レクサーはコンテキストを使用して正規表現リテラルと除算を区別します:

- 値を生成するトークン (識別子、数値、文字列リテラル、) または] の後では、/ は除算として解析される
- 演算子、キーワード、または式を期待する区切り文字 ((, [, ,, =) の後では、/ は正規表現リテラルの開始として扱われる

```
x = 10 / 2          # 除算: 5
y = is_match("a", /a/) # 正規表現リテラル
```

関数一覧

正規表現リテラル関数 (テキスト優先、UFCS 互換)

これらの関数は Regex 型のパターンを取り、UFCS 用にテキスト優先の引数順序を使用します:

関数	シグネチャ	説明
is_match	(str, Regex) -> bool	テキスト全体がパターンにマッチするかを返す
search	(str, Regex) -> int	最初のマッチの開始位置を返す (見つからない場合 -1)

関数	シグネチャ	説明
<code>replace</code>	<code>(str, Regex, str) -> str</code>	マッチした部分を置換文字列で置換
<code>split</code>	<code>(str, Regex) -> List<str></code>	パターンマッチでテキストを分割
<code>find_all</code>	<code>(str, Regex) -> List<str></code>	重複しないすべてのマッチを返す

```
from regex import is_match, search, replace, split, find_all
```

```
# 直接呼び出し
print(is_match("hello", /[a-z]+/))      # true

# UFCS (text.function(pattern))
print("abc123".search(/[0-9]+/))        # 3
print("abc123".replace(/[0-9]+/, "X"))   # abcX
parts = "hello world".split(/\s+/)
nums = "a1b2c3".find_all(/[0-9]/)
```

レガシー関数 (テキスト優先)

元の `regex_*` 関数は後方互換性のために引き続き利用可能です。正規表現リテラルではなくパターン文字列を受け取り、正規表現リテラル API と一貫したテキスト優先の引数順序を使用します:

関数	シグネチャ	説明
<code>regex_match</code>	<code>(text: str, pattern: str) -> bool</code>	テキスト全体がパターンにマッチするかを返す
<code>regex_search</code>	<code>(text: str, pattern: str) -> int</code>	最初のマッチの開始位置を返す (見つからない場合 -1)
<code>regex_replace</code>	<code>(text: str, pattern: str, replacement: str) -> str</code>	マッチした部分を置換文字列で置換
<code>regex_split</code>	<code>(text: str, pattern: str) -> List<str></code>	パターンマッチでテキストを分割
<code>regex_find_all</code>	<code>(text: str, pattern: str) -> List<str></code>	重複しないすべてのマッチを返す

```
print(regex_match("hello", "[a-z]+"))    # true
pos = regex_search("abc123", "[0-9]+")   # 3
```

サポートするパターン構文

構文	説明	例
<code>abc</code>	リテラル文字	"hello"
<code>.</code>	任意の 1 文字 (改行を除く)	"a.c" は "abc", "aXc" にマッチ
<code> </code>	選択	"cat dog" は "cat" か "dog" にマッチ
<code>*</code>	0 回以上の繰り返し	"a*" は "", "a", "aaa" にマッチ

構文	説明	例
+	1 回以上の繰り返し	"a+" は "a", "aaa" にマッチ
?	0 回または 1 回	"a?" は "" か "a" にマッチ
{n}	ちょうど n 回	"a{3}" は "aaa" にマッチ
{n,m}	n 回以上 m 回以下	"a{2,4}" は "aa" ~ "aaaa" にマッチ
{n,}	n 回以上	"a{2,}" は "aa", "aaa", ... にマッチ
?	0 回以上 (非貪欲)	".?" は最短マッチ
+?	1 回以上 (非貪欲)	".+?" は最短マッチ
??	0 回または 1 回 (非貪欲)	"a??" は 0 回を優先
{n,m}?	範囲 (非貪欲)	"a{2,4}?" は n 回を優先
(...)	グループ	"(ab)+" は "abab" にマッチ
[abc]	文字クラス	"[aeiou]" は母音にマッチ
[a-z]	文字範囲	"[a-z]+" は小文字の単語にマッチ
[^...]	否定文字クラス	"[^0-9]" は数字以外にマッチ
^	文字列先頭アンカー	"^hello"
\$	文字列末尾アンカー	"world\$"
\d	数字 [0-9]	"\d+" は数値にマッチ
\D	数字以外 [^0-9]	
\w	単語文字 [a-zA-Z0-9_]	"\w+" は識別子にマッチ
\W	単語文字以外	
\s	空白文字	"\s+" はスペースやタブにマッチ
\S	空白文字以外	
\b	単語境界	"\bword\b" は単語全体にマッチ
\B	非単語境界	"\Bword" は単語の内部にマッチ
(?i)	大文字小文字無視フラグ	"(?i)hello" は "HELLO" にマッチ
\.	エスケープされた特殊文字	"\." はリテラルの . にマッチ

使用例

範囲量指定子

```
print(regex_match("123-4567", "\\d{3}-\\d{4}")) # true
print(regex_match("aaa", "a{2,4}"))           # true
print(regex_match("ababab", "(ab){2,}"))      # true
```

非貪欲 (最短) マッチ

```
# 貪欲: 最長マッチ
g = regex_replace("\"a\" and \"b\"", "\".*\"", "X")
print(g) # X
```

```
# 非貪欲: 最短マッチ
l = regex_replace("\"a\" and \"b\"", "\".*?\"", "X")
print(l) # X and X
```

```
# 個別の HTML タグを取得
```

```
tags = regex_find_all("<a> <bb> <ccc>", "<.*?>")
print(length(tags)) # 3
```

注意: 非貪欲マッチはマッチ全体の長さを制御します。括弧で囲んだグループの抽出がサポートされていないため、greedy/lazy の混在パターンは PCRE エンジンと異なる動作をする場合があります。

単語境界

```
# 単語全体にマッチ
pos = regex_search("hello world", "\\bworld\\b")
print(pos) # 6

# すべての単語を取得
words = regex_find_all("hello world foo", "\\b\\w+\\b")
print(length(words)) # 3
```

大文字小文字を無視したマッチ

```
# (?i) をパターンの先頭に置くと大文字小文字を無視
print(regex_match("HELLO", "(?i)hello")) # true
print(regex_match("Hello", "(?i)hello")) # true
```

注意: (?i) はパターンの先頭に記述する必要があり、パターン全体に適用されます。部分的な大文字小文字無視 (例: (?i:sub)pattern) はサポートされていません。

[English](#) | [日本語](#) | [繁體中文](#)

構造体リファレンス

概要

構造体はスタック上の値型です。record キーワードで定義します。構造体には invariant 節で契約による設計の不変条件を定義できます。[契約による設計](#)を参照。

命名規則: 構造体名は PascalCase (例: Point、Rectangle) を使用する必要があります。フィールド名は snake_case を使用します。コンパイラがこれらの規則を強制します。

定義構文

```
record TypeName:
  field_name: type
  field_name: type
```

例

```
record Point:
  x: int
  y: int

record Rectangle:
  width: float
  height: float
```

コンストラクタ

フィールド定義順に引数を渡します。名前付き引数はサポートされていません。

```
p = Point(10, 20)
r = Rectangle(3.0, 4.5)
```

フィールドアクセス

ドット記法でフィールドを読み取ります。

```
p = Point(10, 20)
print(p.x)    # 10
print(p.y)    # 20
```

フィールド代入

変数宣言	フィールド代入
可変 (@const なし)	可能
@const	コンパイルエラー

```
p = Point(10, 20)
p.x = 100    # OK: 可変変数

@const
q = Point(10, 20)
q.x = 100    # エラー: @const 変数のフィールドは変更不可
```

チェーン / 深いフィールド代入

フィールド代入は、ネストしたレコード、コレクション要素、複合演算子をまたいで合成できます。左辺は可変変数を根とする任意の後置チェーンを使えます。

```
record Inner:
  val: int

record Outer:
  inner: Inner
  tag: str

o = Outer(Inner(1), "t")
o.inner.val = 42          # 深いフィールド書き込み
o.inner.val += 1          # 複合代入形式
print(o.inner.val)        # 43

pts = [Point(1, 2), Point(3, 4)]
pts[0].x = 99             # リスト中の record フィールド更新
pts[0].x *= 2
print(pts[0].x)           # 198
```

完全なマトリクスと、record 内のネストしたコレクションに対するエイリアシングの注意点については [collections](#) → [インデックスとフィールド代入のチェーン](#) を参照してください。

関数引数・戻り値としての使用

```
function distance(p: Point) -> float:
    return (p.x * p.x + p.y * p.y) as float

function make_point(x: int, y: int) -> Point:
    return Point(x, y)
```

ネスト構造体

構造体を別の構造体のフィールドとして使用できます。

```
record Point:
    x: int
    y: int

record Circle:
    center: Point
    radius: float

c = Circle(Point(0, 0), 1.0)
print(c.center.x)    # 0
```

比較 (== / !=)

レコード型は == と != 演算子を自動的にサポートします。フィールドごとの比較（構造的等価性）が行われます。

```
record Point:
    x: int
    y: int

p1 = Point(10, 20)
p2 = Point(10, 20)
p3 = Point(30, 40)

print(p1 == p2)    # true
print(p1 != p3)    # true
```

- すべてのフィールドが順番に比較されます。== ではすべてのフィールドが等しい必要があり、!= では少なくとも1つのフィールドが異なる必要があります。
 - ネストされたレコードは再帰的に比較されます。
 - ユーザー定義の operator== または operator!= がある場合、自動生成版より優先されます。
-

レコードのサブタイプ化（継承）

レコードは < 構文を使った単一継承をサポートします。子レコードは親のすべてのフィールドを継承します。

構文

```
record ChildName < ParentName:
    child_field: type
```

例

```
record HttpError < Error:
  status: int
  url: str
```

フィールドの継承

- 子レコードはレイアウトの先頭に親のすべてのフィールドを継承します。
- コンストラクタは親のフィールドを先に、次に子固有のフィールドを取ります。

```
err = HttpError("not found", 404, 404, "/api")
print(err.message) # "not found" (Error から継承)
print(err.status)  # 404 (固有フィールド)
```

サブタイプの型強制

子の値は親の型が期待される場所に渡すことができます。子は自動的にスライスされ、親のプレフィックスフィールドが抽出されます（値型スライシング）。

```
function handle(e: Error) -> str:
  return e.message

err = HttpError("fail", 500, 500, "/api")
handle(err) # OK — HttpError が Error に型強制される
```

深い継承

レコードは継承チェーンを形成できます。各レベルはすべての祖先フィールドを継承します。

```
record DetailedHttpError < HttpError:
  detail: str

# コンストラクタ: Error フィールド + HttpError フィールド + 固有フィールド
derr = DetailedHttpError("fail", 500, 500, "/x", "server crash")
handle(derr) # OK — Error (祖父母) に型強制される
```

ルール

ルール	詳細
単一継承のみ	record A < B: — 親は1つのみ
深い継承	record C < B: where record B < A: — 許可
名前の衝突	子のフィールドが親のフィールドと同名 → コンパイルエラー
自動 == / to_str	継承したフィールドもすべて含む
不変条件の継承	親の invariant: 節は子レコードの構築・変更時にもチェックされる
サブタイプの型強制	適用先: 関数引数、return、Err()、フィールド代入、? 演算子
ジェネリックバウンド	<T: RecordName> で型パラメータをレコードのサブタイプに制約
@const	継承したフィールドも含むすべてのフィールドに適用

制約とエラー

制約	詳細
同一フィールド名の重複	コンパイルエラー
@const 変数のフィールド代入	コンパイルエラー

エラー例: 同一フィールド名の重複

```
record Bad:
    x: int
    x: int    # エラー
```

列挙型 (enum)

概要

列挙型は名前付き定数の集合です。デフォルトでは i64 整数 (0, 1, 2, ...) の連番として表現されます。明示的な整数値を割り当てることもできます。

定義構文

```
enum TypeName:
    VariantName
    VariantName
    ...
```

例

```
enum Color:
    Red
    Green
    Blue
```

バリエーションアクセス

:: 演算子でバリエーションにアクセスします。

```
c = Color::Red
print(c)    # Red
```

比較

enum 値は整数なので == / != でそのまま比較できます。

```
print(Color::Red == Color::Red)    # true
print(Color::Red != Color::Green)  # true
```

if 文での使用

```
c = Color::Green
case:
    c == Color::Red:
        print("red")
    c == Color::Green:
        print("green")
```

```
_:
    print("blue")
```

関数引数

型名として enum 名を使用します。

```
function is_red(c: Color) -> bool:
    return c == Color::Red

print(is_red(Color::Red))    # true
print(is_red(Color::Green))  # false
```

print

print() でバリエーション名が出力されます。

```
c = Color::Blue
print(c)    # Blue
```

明示的な値の割り当て

simple enum のバリエーションに明示的な整数値を割り当てることができます。HTTP ステータスコードやビットマスクパターンなどに有用です。

```
enum HttpStatus:
    Ok = 200
    NotFound = 404
    InternalError = 500

s = HttpStatus::NotFound
print(s)                # NotFound
print(s == HttpStatus::NotFound)  # true

enum Permission:
    Read = 1
    Write = 2
    Execute = 4
```

ルール: - simple enum (関連データを持たない ADT バリエーションなし) のみ対応- 値は整数リテラルのみ (負の値も可) - いずれかのバリエーションに明示的な値がある場合、全バリエーションに必要 (混在不可) - 重複値はコンパイラエラー - print() はバリエーション名を表示 (整数値ではない)

ADT バリエーションの名前付きフィールド

ADT バリエーションのフィールドにはドキュメント目的で名前をオプションで付けることができます。名前付きフィールドは定義を自己説明的にしますが、構築やパターンマッチングのセマンティクスは変わりません。

```
enum Shape:
    Circle(radius: float)
    Rect(width: float, height: float)
    Point
```

- 構築は常に位置ベース: Shape::Circle(3.14) であり、Shape::Circle(radius: 3.14) ではない。
- パターンマッチングはユーザーが選んだ変数名で束縛: case Shape::Circle(r):。
- フィールド名は snake_case でなければならない。単一バリエーション内での名前付きと名前なしの混在は不可。
- 名前なし構文 (Circle(float)) は引き続き有効。

制約とエラー

制約	詳細
バリエーションアクセスは <code>EnumName::VariantName</code> バリエーション値	<code>::</code> 演算子が必須 デフォルトは自動割り当て (0, 1, 2, ...)、 <code>=</code> 値で明示的に指定可能
比較は整数比較 名前付きフィールド名	<code>==</code> , <code>!=</code> が使用可能 <code>snake_case</code> でなければならない; バリエーション内で重複不可; 名前付きと名前なしの混在不可

[English](#) | [日本語](#) | [繁體中文](#)

テスト機能

Ry は RSpec 風のテスト構文を内蔵しています。ry test サブコマンドでテストファイルを実行します。

実行方法

```
ry test                # プロジェクト内の *.test.ry を自動検出して実行
ry test tests/spec     # 指定ディレクトリ以下の *.test.ry を再帰的に実行
ry test test_file.ry   # 特定のテストファイルを実行
ry test -p             # 全テストを並列実行 (-p または --parallel)
ry test -p tests/      # 指定ディレクトリのテストを並列実行
ry test -w             # ウォッチモード: ファイル変更時にテストを自動再実行 (-w または --watch)
ry test -w -p          # ウォッチモード + 並列実行
ry test -w tests/      # 指定ディレクトリをウォッチ
ry test --coverage     # 全テストをラインカバレッジ付きで実行
ry test --cov          # --coverage の短縮形
ry test --outline      # テストを実行せずに describe/it 構造を表示
```

終了コードは全テスト成功時に 0、失敗がある場合は 1 です。

自動検出モード

ry test を引数なしで実行すると:

1. package.toml を探してプロジェクトルートを特定
2. プロジェクトルート以下の *.test.ry ファイルを再帰的に検出 (.git、build、node_modules はスキップ)
3. 各ファイルを実行し、結果を集計

構文

関数ベース構文 (推奨)

@it と @describe ディレクティブを使って、テストケースを通常の名前付き関数として定義します:

```
@it("test case name")
function test_add():
  expect(1 + 2).to_eq(3)
```

関連するテストは @describe でグループ化します:

```
@describe("Arithmetic")
function arithmetic_tests():
  @it("should add integers")
  function test_add():
    expect(1 + 2).to_eq(3)
```

```
  @it("should subtract integers")
  function test_sub():
    expect(5 - 3).to_eq(2)
```

- 関数名はコードナビゲーションとシンボル同一性に使われる
- 説明文字列（ディレクティブ内）はテスト出力と報告に使われる
- @it 関数は @each または @property と組み合わせない限り、パラメータを持つてはならない
- @it と @describe は ry test でのみ使用可能

共有セットアップ @describe 関数ボディで宣言された変数は、内側の @it 関数に自動的にキャプチャされます:

```
@describe("User validation")
function user_validation_tests():
  min_length = 8
  max_length = 64

  @it("should reject short passwords")
  function test_short():
    expect(min_length).to_be_greater_than(0)

  @it("should accept passwords within length limits")
  function test_range():
    expect(max_length).to_be_greater_than(min_length)
```

ネストした@describe @describe 関数はネストさせて多層のグループ化を作成できます。出力はネストの深さを反映してインデントされます:

```
@describe("API")
function api_tests():
  @describe("GET /users")
  function get_users_tests():
    @it("should return 200 OK")
    function test_ok():
      expect(true).to_be_true()
```

出力:

```
API
  GET /users
    + should return 200 OK
```

ラムダ構文（非推奨）

非推奨: describe() と it() のラムダ呼び出し構文は非推奨です。代わりに @describe と @it ディレクティブを名前付き関数に使用してください。ラムダ構文は将来のリリースで削除されます。

移行:

ラムダ構文	ディレクティブ構文
<code>it("name", (): ...)</code>	<code>@it("name") function name(): ...</code>
<code>describe("name", (): ...)</code>	<code>@describe("name") function name(): ...</code>

```
describe("説明文", ():
    it("テストケース名", ():
        # テスト本体
        expect(実際の値).to_eq(期待値)
    )
)
```

- `describe` と `it` は説明文字列とラムダ引数 `()` を第二引数に取る
- `describe` ブロック内には `it` ブロックやその他の文（変数宣言など）を記述可能
- 各 `it` ブロックは独立したテストケース
- `describe` / `expect` は `ry test` でのみ使用可能（通常の `ry` 実行ではコンパイルエラー）

トレイリングブロック構文

任意の関数呼び出し（`describe/it/mock` を除く）にトレイリングブロック構文が使えます。`()` の後に `:` を付けると、インデントブロックが引数なしラムダとして最後の引数に渡されます:

```
# 以下は等価:
foo("arg"):
    bar()
```

```
foo("arg", ():
    bar()
)
```

`expect` / マッチャー

マッチャー	説明	対応型
<code>to_eq(expected)</code>	等値比較	<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code>
<code>to_not_eq(expected)</code>	等しくないこと	<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code>
<code>to_be_true()</code>	<code>true</code> であること	<code>bool</code>
<code>to_be_false()</code>	<code>false</code> であること	<code>bool</code>
<code>to_be_none()</code>	<code>None</code> であること	<code>Option</code>
<code>to_be_some()</code>	<code>Option</code> が <code>Some</code> であること	<code>Option</code>
<code>to_be_ok()</code>	<code>Result</code> が <code>Ok</code> であること	<code>Result</code>
<code>to_be_err()</code>	<code>Result</code> が <code>Err</code> であること	<code>Result</code>
<code>to_contain(val)</code>	コンテナが値を含むこと	<code>List</code> , <code>Set</code> , <code>Map</code> , <code>str</code>
<code>to_not_contain(val)</code>	コンテナが値を含まないこと	<code>List</code> , <code>Set</code> , <code>Map</code> , <code>str</code>
<code>to_be_greater_than(v)</code>	<code>actual > v</code> であること	<code>int</code> , <code>float</code>
<code>to_be_less_than(v)</code>	<code>actual < v</code> であること	<code>int</code> , <code>float</code>
<code>to_be_greater_than_or_eq(v)</code>	<code>actual >= v</code> であること	<code>int</code> , <code>float</code>
<code>to_be_less_than_or_eq(v)</code>	<code>actual <= v</code> であること	<code>int</code> , <code>float</code>
<code>to_have_length(n)</code>	長さが <code>n</code> であること	<code>List</code> , <code>Set</code> , <code>Map</code> , <code>str</code>
<code>to_be_empty()</code>	長さが <code>0</code> であること	<code>List</code> , <code>Set</code> , <code>Map</code> , <code>str</code>
<code>to_start_with(prefix)</code>	文字列が <code>prefix</code> で始まること	<code>str</code>
<code>to_end_with(suffix)</code>	文字列が <code>suffix</code> で終わること	<code>str</code>

`fail`

現在のテストを即座に失敗としてマークします。

```
it("should not reach here", ():
  fail("unexpected error")
)
```

- fail() — 汎用メッセージでテストを失敗にする
 - fail(msg) — カスタムメッセージでテストを失敗にする
 - fail() 後も実行は継続される（テストを中断はしない）
 - ry test モードでのみ使用可能
-

出力形式

```
Calculator
+ should add numbers
+ should subtract
- should fail (赤色)
  line 10: expected 3, got 2
```

2 passed, 1 failed

- + は成功（緑色）、- は失敗（赤色）
 - 失敗時は行番号と期待値/実際の値を表示
-

例

```
describe("Arithmetic", ():
  it("should add integers", ():
    expect(1 + 2).to_eq(3)

  )
  it("should compare strings", ():
    expect("hello").to_eq("hello")

  )
  it("should check booleans", ():
    expect(3 > 1).to_be_true()

  )
)
describe("Booleans", ():
  it("should return false", ():
    expect(1 > 2).to_be_false()

  )
)
```

モック

mock(fn_name, replacement)

現在の it ブロック内で関数をモック実装に差し替えます。it ブロック終了時にモックは自動的にクリアされます。

```
function fetch_data() -> str:
```

```

    return "real data"

describe("mocking", ():
    it("should replace function", ():
        mock(fetch_data, () => "fake")
        expect(fetch_data()).to_eq("fake")

    )
    it("should auto-restore after it block", ():
        expect(fetch_data()).to_eq("real data")
    )
)

```

- 第1引数は関数名（識別子、文字列ではない）
- 第2引数は差し替え用ラムダ
- 差し替え関数は元の関数と同じ引数型・戻り値型である必要がある
- 元の関数の `require` と `ensure` 契約はモック呼び出し時にも強制される
- `it` ブロック終了時にモックは自動復元される

verify(fn_name)

モック済み関数の呼び出し回数を `int` で返します。

```

describe("verify", ():
    it("should count calls", ():
        mock(fetch_data, () => "fake")
        fetch_data()
        fetch_data()
        expect(verify(fetch_data)).to_eq(2)
    )
)

```

モックの制限事項

- オーバーロードされた関数のモックは非対応
- キャプチャ付きクロージャでのモックは非対応（プレーンラムダのみ）
- `@native function` のモックは非対応

パラメタライズドテスト (@each)

`@each` は同じテストを複数のパラメータセットで実行します。

関数ベース構文（推奨）：

```

@each([
    (1, 2, 3),
    (0, 0, 0),
    (-1, 1, 0)
])
@it("should add {0} + {1} = {2}")
function test_add(a: int, b: int, expected: int):
    expect(a + b).to_eq(expected)

```

ラムダ構文：

```
@each([
    (1, 2, 3),
    (0, 0, 0),
    (-1, 1, 0)
])
it("should add {0} + {1} = {2}", (a: int, b: int, expected: int):
    expect(a + b).to_eq(expected)
)
```

- リストにはパラメータ数と同じアリティのタプルを含める
- 説明文の {0}, {1}, ... はパラメータ値で置換される
- 各タプルは独立したテストケースとして実行される
- 対応するパラメータ型: int, float, bool, str

プロパティベーステスト (@property)

@property はランダムな入力を生成し、テストを複数回実行します。

関数ベース構文 (推奨):

```
@property(count=100)
@it("should verify addition is commutative")
function test_commutative(a: int, b: int):
    expect(a + b).to_eq(b + a)
```

ラムダ構文:

```
@property(count=100)
it("should verify addition is commutative", (a: int, b: int):
    expect(a + b).to_eq(b + a)
)
```

- count=N でランダム試行回数を指定 (デフォルト: 100)
- 失敗時は反例 (失敗した入力値) が表示される
- 最初の失敗でテストを停止
- 対応するパラメータ型: int ([-1000, 1000]), float ([-1000.0, 1000.0]), bool, str (ランダム ASCII, 0-20 文字)

テストカバレッジ

--coverage (または--cov) フラグでラインカバレッジを計測できます:

```
ry test --coverage          # 全テスト + カバレッジサマリー
ry test --cov tests/spec/math.test.ry # 単一ファイル
ry test --coverage tests/spec/      # ディレクトリ
```

出力例

```
Test Coverage Summary:
tests/spec/math.test.ry    100.0% (74/74 lines)
tests/spec/strings.test.ry 92.3% (24/26 lines)
-----
Total                      95.1% (98/100 lines)
```

- 標準ライブラリのファイルは除外され、ユーザーコードのみが対象
- --coverage と --parallel を同時に指定した場合、逐次実行にフォールバック

テストアウトライン

--outline を使用すると、テスト本体を実行せずにテストファイルの describe/it 構造を表示できます:

```
ry test --outline tests/spec/mock.test.ry
```

出力:

```
describe mock
  it should replace function
  it should auto-restore after it block
  it should mock with arguments
describe verify
  it should count calls
  it should count zero calls
```

- 個別ファイル、ディレクトリ、-p (全テストファイル) で使用可能
- @each パラメタライズドテストはフォーマットテンプレートに (@each) サフィックスを付けて表示
- @property テストはラベルに (@property) サフィックスを付けて表示

テスト説明のスタイル

it の説明は should で始めることが推奨されます。テスト出力で完全な文として自然に読めます:

```
it should add integers
it should reject invalid input
it should return error when file is missing
```

推奨:

説明	備考
"should add integers"	動詞は原形
"should reject short passwords"	動詞は原形
"should return error for missing file"	動詞は原形
"should add {0} + {1} = {2}"	パラメタライズド: 動詞は原形
"should verify addition is commutative"	プロパティベース

避けるべき:

説明	理由
"adds integers"	三人称動詞、"it adds" として読むときこちない
"integer addition"	名詞句、文ではない
"handles error"	三人称動詞

describe ブロックは名詞またはトピック句を使います (例: "Arithmetic", "List", "GET /users")。should は不要です。

制限事項

- describe のネスト（ラムダ構文）は未対応。ネストしたグループ化には関数ベースの@describe ディレクティブ構文を使用してください
- before_each / after_each は未対応

[English](#) | [日本語](#) | [☒体中文](#)

スレッドリファレンス

thread パッケージは、CPU バウンドの並列ワークロードやきめ細かい並行制御のための OS レベルのスレッドプリミティブを提供します。

型

型	説明
Thread	OS スレッドの不透明ハンドル
Lock	ミューテックス（相互排他ロック）の不透明ハンドル
RWLock	読み書きロックの不透明ハンドル
Semaphore	カウンティングセマフォの不透明ハンドル
Barrier	スレッドバリアの不透明ハンドル
AtomicInt	アトミック 64 ビット整数の不透明ハンドル
AtomicBool	アトミック真偽値の不透明ハンドル

すべての型は ARC（自動参照カウント）で管理される不透明ポインタです。インスタンスの作成には対応する*_new() 関数を使用します。リソースは参照がなくなると自動的にクリーンアップされます。即座にクリーンアップするための*_free() 呼び出しはオプションですがサポートされています。

インポート

```
from thread import thread_spawn, thread_join, lock_new, lock_acquire, lock_release
```

Thread

関数	シグネチャ	説明
thread_spawn	(body: function() -> T) -> Thread	body を実行する新しい OS スレッドを作成・開始する。キャプチャされた変数は値によりコピーされる。T は Unit、int、float、bool のいずれか。下の制限事項を参照。
thread_join	(thread: Thread) -> Result<T, Error>	スレッドの終了を待ち、ワーカーの値を Ok(value) で返す。既に join 済みの Thread を join すると Err("thread already joined") を返す。

使用例 — 副作用ワーカー (Unit)

```
from thread import thread_spawn, thread_join, atomic_int_new, atomic_int_load, atomic_int_add

counter = atomic_int_new(0)
t = thread_spawn():
    atomic_int_add(counter, 1)
)
thread_join(t)
print(atomic_int_load(counter)) # 1
```

使用例 — 値を返すワーカー (int / float / bool)

```
from thread import thread_spawn, thread_join

t = thread_spawn() => 42
case thread_join(t):
    Ok(v):
        print(v) # 42
    Err(e):
        print(e.message)
```

キャプチャは値を返すワーカーでも動作する:

```
x = 10
t2 = thread_spawn() => x * x
case thread_join(t2):
    Ok(v):
        print(v) # 100
    Err(_):
        print("error")
```

注意: キャプチャされた変数はスレッドの環境にコピーされます。プリミティブ型 (int、float、bool、str) の場合、独立したコピーが作成されます。不透明ハンドル型 (Lock、AtomicInt など) の場合、ポインタがコピーされ基盤となるリソースを共有します。これは同期プリミティブの意図された動作です。

制限事項 (MVP)

- **戻り値型。** ワーカーは Unit、int、float、bool を返せます。ARC 管理の型 (str、List、Map、Set、record) と直和型 (Option、Result、enum) はまだサポートされていません。そのようなワーカーを渡すと、フォローアップ issue を示す codegen エラーになります。
- **ラムダボディの形式。** 値を返せるのは式ボディのラムダ (() => <expr>) のみです。ブロックボディのラムダは Unit ワーカーには引き続き使えますが、Unit 以外の戻り値は運べません。
- **変数参照ワーカー。** thread_spawn(my_fn) は引き続き my_fn を Unit ワーカーとして扱います。戻り値を読み取るには、現状ではインラインラムダ形式が必要です。
- **パニック。** ワーカー内のランタイムパニック (例: 0 除算、配列範囲外、契約違反) はプロセス全体を終了させます。現時点では thread_join から Err として表面化しません - これには Ry のパニック機構の別途リファクタが必要で、フォローアップ issue として追跡されています。

Lock (ミューテックス)

関数	シグネチャ	説明
lock_new	() -> Lock	新しいミューテックスを作成する。

関数	シグネチャ	説明
lock_acquire	(lock: Lock) -> Result<Unit, Error>	ロックを取得する。利用可能になるまでブロックする。
lock_release	(lock: Lock) -> Result<Unit, Error>	ロックを解放する。
lock_free	(lock: Lock) -> Unit	ロックを即座に解放する。 オプション — ARC が自動的にクリーンアップする。

RWLock（読み書きロック）

関数	シグネチャ	説明
rwlock_new	() -> RWLock	新しい読み書きロックを作成する。
rwlock_read_lock	(rwlock: RWLock) -> Result<Unit, Error>	共有読み取りロックを取得する。複数の読み取りが許可される。
rwlock_write_lock	(rwlock: RWLock) -> Result<Unit, Error>	排他書き込みロックを取得する。すべての読み取り・書き込みが解放されるまでブロックする。
rwlock_unlock	(rwlock: RWLock) -> Result<Unit, Error>	ロック（共有または排他）を解放する。
rwlock_free	(rwlock: RWLock) -> Unit	読み書きロックを即座に解放する。オプション — ARC が自動的にクリーンアップする。

Semaphore

関数	シグネチャ	説明
semaphore_new	(count: int) -> Result<Semaphore, Error>	指定した初期カウントでセマフォを作成する。カウントが負の場合は Err を返す。
semaphore_acquire	(sem: Semaphore) -> Result<Unit, Error>	セマフォをデクリメントする。カウントがゼロの場合はブロックする。
semaphore_release	(sem: Semaphore) -> Result<Unit, Error>	セマフォをインクリメントし、待機中のスレッドを起こす。
semaphore_free	(sem: Semaphore) -> Unit	セマフォを即座に解放する。オプション — ARC が自動的にクリーンアップする。

Barrier

関数	シグネチャ	説明
<code>barrier_new</code>	<code>(count: int) -> Result<Barrier, Error></code>	<code>count</code> 個のスレッドを同期するバリアを作成する。カウントが正でない場合は <code>Err</code> を返す。
<code>barrier_wait</code>	<code>(barrier: Barrier) -> Result<Unit, Error></code>	すべての <code>count</code> 個のスレッドが <code>barrier_wait</code> を呼び出すまでブロックする。
<code>barrier_free</code>	<code>(barrier: Barrier) -> Unit</code>	バリアを即座に解放する。オプション — ARC が自動的にクリーンアップする。

AtomicInt

関数	シグネチャ	説明
<code>atomic_int_new</code>	<code>(value: int) -> AtomicInt</code>	指定した初期値でアトミック整数を作成する。
<code>atomic_int_load</code>	<code>(atomic: AtomicInt) -> int</code>	値をアトミックに読み取る。
<code>atomic_int_store</code>	<code>(atomic: AtomicInt, value: int) -> Unit</code>	値をアトミックに書き込む。
<code>atomic_int_add</code>	<code>(atomic: AtomicInt, delta: int) -> int</code>	<code>delta</code> をアトミックに加算し、 変更前 の値を返す。
<code>atomic_int_sub</code>	<code>(atomic: AtomicInt, delta: int) -> int</code>	<code>delta</code> をアトミックに減算し、 変更前 の値を返す。
<code>atomic_int_cas</code>	<code>(atomic: AtomicInt, expected: int, desired: int) -> bool</code>	Compare-and-swap: 現在の値が <code>expected</code> と等しい場合、 <code>desired</code> に設定して <code>true</code> を返す。そうでなければ <code>false</code> を返す。
<code>atomic_int_free</code>	<code>(atomic: AtomicInt) -> Unit</code>	アトミック整数を即座に解放する。オプション — ARC が自動的にクリーンアップする。

AtomicBool

関数	シグネチャ	説明
<code>atomic_bool_new</code>	<code>(value: bool) -> AtomicBool</code>	指定した初期値でアトミック真偽値を作成する。
<code>atomic_bool_load</code>	<code>(atomic: AtomicBool) -> bool</code>	値をアトミックに読み取る。
<code>atomic_bool_store</code>	<code>(atomic: AtomicBool, value: bool) -> Unit</code>	値をアトミックに書き込む。
<code>atomic_bool_free</code>	<code>(atomic: AtomicBool) -> Unit</code>	アトミック真偽値を即座に解放する。オプション — ARC が自動的にクリーンアップする。

async/await との比較

特徴	async/await	thread パッケージ
実行モデル	ワークスティーリングスレッドプール	専用 OS スレッド
適した用途	I/O バウンドタスク、多数の軽量タスク	CPU バウンドタスク、きめ細かい制御
同期	await による自動同期	Lock、Semaphore 等による手動同期
オーバーヘッド	低い（タスクスケジューリング）	高い（OS スレッド作成）

[English](#) | [日本語](#) | [繁體中文](#)

型リファレンス

型一覧

型	内部表現	リテラル例	説明
int	i64	42, -7, 0xFF, 0b1010, 100_000	64 ビット符号付き整数
u8	i8	(専用リテラルなし)	符号なし 8 ビット整数 (0-255)。型アノテーション b: u8 = 42 で使用
float	f64	3.14, 0.5, 3.14_159, 1e10, 1.5e-3, 2.5E+2	64 ビット浮動小数点数 (科学的記法をサポート)
bool	i1	true, false	真偽値
str	ptr	"hello", "", "a\nb"	文字列 (ヒープ上の不変バイト列)
Unit	void	(戻り値なし)	戻り値のない関数の戻り値型。-> Unit で明示的に指定する必要がある
Option<T>	{ i1, T }	Some(42), None	値が存在するかもしれない型
(T1, T2, ...)	LLVM StructType (literal)	(1, 3.14)	タプル型
List<T>	ptr (ヒープ)	[1, 2, 3]	動的配列
Map<K, V>	ptr (ヒープ)	{"a": 1}	ハッシュマップ
Set<T>	ptr (ヒープ)	{1, 2, 3}	重複なしの集合
function(T1, T2) -> R	ptr (関数ポインタ)	(x: int) => x * 2	関数型
ユーザー定義型	LLVM StructType (named)	record Point: ...	record キーワードで定義する構造体
enum	i64 / タグ付きユニオン	Color::Red, Shape::Circle(3.14)	enum キーワードで定義する列挙型 (関連データをサポート)
Error	{ ptr, i64 }	Error("msg"), Error("msg", 404)	組み込みエラー型
Type	{ i64, ptr }	type_of(42)	type_of が返すコンパイル時の型 identity。Type を参照
any	{ i64, [8 x i8] }	x: any = 42	任意のプリミティブ値を保持できるタグ付きユニオン

型	内部表現	リテラル例	説明
T1 \ T2	{ i64, [N x i8] }	int \ str	union 型（複数の型のいずれかを保持）
int リテラル	i64	42, 0 \ 1	int リテラル型（値の制約）
str リテラル	ptr	"N" \ "S"	str リテラル型（値の制約）
範囲	i64	1..12, -10..10	範囲型（整数の範囲制約）
i8	i8	x: i8 = 42, x = 42i8	8 ビット符号付き整数（低レベル、暗黙の変換なし）
i16	i16	x: i16 = 100, x = 100i16	16 ビット符号付き整数（低レベル、暗黙の変換なし）
i32	i32	x: i32 = 42, x = 42i32	32 ビット符号付き整数（低レベル、暗黙の変換なし）
i64	i64	x: i64 = 100, x = 100i64	64 ビット符号付き整数（低レベル、暗黙の変換なし）
u8	i8	x: u8 = 200, x = 200u8	8 ビット符号なし整数（低レベル、暗黙の変換なし）
u16	i16	x: u16 = 60000, x = 60000u16	16 ビット符号なし整数（低レベル、暗黙の変換なし）
u32	i32	x: u32 = 4294967295, x = 100u32	32 ビット符号なし整数（低レベル、暗黙の変換なし）
u64	i64	x: u64 = 18446744073709551615, x = 0xFFFFFFFFFFFFFFFFFu64	2 ⁶⁴ - 1 までの 64 ビット符号なし整数（低レベル、暗黙の変換なし）
f32	float	x: f32 = 3.14, x = 1e10f32	32 ビット浮動小数点数（低レベル、暗黙の変換なし）
weak T	ptr (header)	weak s	ARC 管理された値への弱参照（解放を妨げない）
Regex	ptr	/[a-z]+/, /\d{3}/	正規表現パターン（正規表現リテラル構文で作成）
Result<T, E>	{ i1, T/E }	Ok(42), Err(Error("fail"))	成功 (Ok) または失敗 (Err) を表す型
Task<T>	ptr	(async 関数が返す)	非同期タスクハンドル (await と block_on で使用)
Iterator<T>	ptr	(iter() で作成)	逐次要素アクセス用の遅延イテレータ
T[N]	[N x T]	buf: i32[8]	固定長配列。低レベル型 T の N 要素（スタック割り当て、連続メモリ）

型アノテーション構文

変数宣言時に型を明示できます。型が推論可能な場合は省略可能です。

```
x: int = 42
b: u8 = 255
f: float = 3.14
s: str = "hello"
b: bool = true
opt: Option<int> = Some(10)
t: (int, float) = (1, 3.14)
xs: List<int> = [1, 2, 3]
m: Map<str, int> = {"a": 1}
s: Set<int> = {1, 2, 3}
fn_val: function(int) -> int = (x: int) => x * 2
rx: Regex = /[0-9]+/
u: int | str = 42
a: any = 42
```

使用可能な型名一覧

型名	備考
int	組み込みスカラー型
u8	組み込みスカラー型（符号なし 0-255）
float	組み込みスカラー型
bool	組み込みスカラー型
str	組み込み文字列型
Unit	戻り値なし関数の戻り値型
Option<T>	ジェネリック型（T は任意の型）
(T1, T2, ...)	タプル型（要素数・型の組み合わせは任意）
List<T>	ジェネリック動的配列型
Map<K, V>	ジェネリックハッシュマップ型
Set<T>	ジェネリック集合型
function(T1, ...) -> R	関数型
Error	組み込みエラー型（message: str、code: int）
any	任意のプリミティブ値（int, float, bool, str）または Unit を保持できる組み込み型。暗黙の変換をサポート: 具体型の値は any への代入時に自動的にラップされ、any の値は具体型への代入時にランタイム型チェック付きで自動アンラップされる。any(int) → float の自動昇格に対応。詳細は any 型 を参照
T1 \ T2 \ ...	union 型（\ で区切った複数の型のいずれか）
i8	低レベル 8 ビット符号付き整数（暗黙の変換なし）
i16	低レベル 16 ビット符号付き整数（暗黙の変換なし）
i32	低レベル 32 ビット符号付き整数（暗黙の変換なし）
i64	低レベル 64 ビット符号付き整数（暗黙の変換なし）
u8	低レベル 8 ビット符号なし整数（暗黙の変換なし）
u16	低レベル 16 ビット符号なし整数（暗黙の変換なし）
u32	低レベル 32 ビット符号なし整数（暗黙の変換なし）
u64	低レベル 64 ビット符号なし整数（暗黙の変換なし）
f32	低レベル 32 ビット浮動小数点数（暗黙の変換なし）
T[N]	低レベル型 T の N 要素の固定長配列。スタック割り当て、連続メモリ。インデックスの読み書きと length() をサポート

型名	備考
ユーザー定義型名	record または enum キーワードで宣言した型

型エイリアス

type キーワードで既存の型に新しい名前を付けます。エイリアスは元の型と完全に互換性があります。

```
type Meters = float
type StringList = List<str>
```

```
d: Meters = 3.14
names: StringList = ["Alice", "Bob"]
```

命名規則: 型エイリアス名は PascalCase (例: Meters、StringList) を使用する必要があります。コンパイラがこの規則を強制します。

型エイリアスは関数型、リテラル型、範囲型にも使用できます:

```
type Callback = function(int, int) -> int

add: Callback = function(a: int, b: int) => a + b
print(add(3, 4))    # 7

type Month = 1..12
type Direction = "N" | "S" | "E" | "W"
type Digit = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

m: Month = 6
d: Direction = "N"
n: Digit = 5
```

型エイリアスはユニオン型 (プリミティブ型やユーザー定義型を含む) も指せます。エイリアスはインライン展開したユニオンと同一に振る舞います:

```
type Simple = int | str | bool

x: Simple = 42
y: Simple = "hello"
z: Simple = true

function describe(v: Simple) -> str:
    return to_str(v)
```

ユニオンの構成要素がまたエイリアスであるネストされたエイリアスは透過的にフラット化され、重複メンバーは除去されます。以下の 3 つの形式は等価です:

```
type A = int | str
type B = A | bool           # `int / str / bool` と同じ
type C = B | int            # `int / str / bool` と同じ (int は重複除去)

x: B = 42
y: B = "hello"
z: B = true
```

数値リテラル

整数リテラル

10 進、16 進 (0x/0X)、2 進 (0b/0B) 形式を受け付けます。桁の間にアンダースコアを視覚的な区切り文字として使えます (1_000_000, 0xFFFF_FFFF)。

受け付けられる値の大きさはターゲット型で決まります:

ターゲット	範囲
裸の int / i64	-9_223_372_036_854_775_808 .. 9_223_372_036_854_775_807 (i64)
i8 / i16 / i32	対応する符号付き範囲
u8 / u16 / u32	0 .. $2^N - 1$
u64	0 .. 18_446_744_073_709_551_615 ($2^{64} - 1$)

大きな符号なしリテラルには、サフィックス (18446744073709551615u64) または受け取り側の変数への型アノテーション (x: u64 = 18446744073709551615) が必要です。負リテラルは非負の大きさに対する単項マイナスとして到達するため、-1i8 は許容されますが -1u8 は拒否されます。

```
max_u64: u64 = 18446744073709551615    # 264 - 1
mask:    u64 = 0xFFFF_FFFF_FFFF_FFFF    # 同じ値を 16 進で
word:    u32 = 4294967295                 # 232 - 1
```

浮動小数点リテラル

```
FloatLiteral := DecDigits '.' DecDigits Exponent? FloatSuffix?
               | DecDigits Exponent FloatSuffix?
Exponent      := ('e' | 'E') ('+' | '-')? DecDigits
FloatSuffix   := 'f32' | 'f64'
```

float が期待される場所ならどこでも科学的記法が使えます:

```
avogadro = 6.022e23
planck   = 6.626e-34
light_spd = 2.998E8
big       = 1e10f32
```

指数がオーバーフローする場合は +Inf / -Inf が生成されます (コンパイルエラーではありません)。注意: ランタイムの to_float() コンバータはより厳格で、オーバーフロー時に +Inf ではなく Err(Error) を返します。

リテラル型

リテラル型は、変数の値を特定の定数値に制限します。定数値の場合はコンパイル時に、動的な値の場合は実行時に制約がチェックされます。

int リテラル型

```
x: 42 = 42           # 単一リテラル型
y: 0 | 1 = 0         # int リテラルの union
z: 0 | 1 = 0
z = 1                # OK
# z = 2              # コンパイルエラー (定数) または実行時エラー (動的値)
```

str リテラル型

```
dir: "N" | "S" | "E" | "W" = "N"
# @const bad: "N" | "S" = "X"    # コンパイルエラー
```

制約チェック

- **コンパイル時:** 代入値が定数 (ConstantInt や文字列リテラル) の場合、コンパイル時にチェックされ違反時はコンパイルエラー
 - **実行時:** 値が動的 (関数の戻り値など) の場合、実行時にチェックされ違反時はプログラムがエラー終了
-

範囲型

範囲型は、整数変数の値を連続した範囲 (両端を含む) に制限します。

```
month: 1..12 = 6          # OK
# @const bad: 1..12 = 0    # コンパイルエラー: 範囲外
# @const bad: 1..12 = 13   # コンパイルエラー: 範囲外

t: -10..10 = -5          # 負の範囲もサポート
```

可変変数での再代入 (実行時チェック)

```
x: 1..12 = 6
x = 12          # OK
# x = dynamic_value()  # 実行時チェック: 範囲外ならエラー終了
```

関数パラメータでの使用

```
function set_month(m: 1..12) -> int:
    return m

set_month(6)          # OK
# set_month(13)       # コンパイルエラー (定数引数)
```

none キーワードと Option 型の省略記法

none キーワードは Option 型の値が存在しないことを表し、None と同等です。

T? 構文は Option<T> の省略記法です。

```
x: int? = 42          # Option<int> と同等
y: int? = none        # None と同等

function find(xs: List<int>, val: int) -> int?:
    for x in xs:
        if x == val:
            return Some(x)
    return none
```

弱参照 (weak T)

weak 参照は、ARC 管理された値への非所有参照です。強参照とは異なり、弱参照は強参照カウントをインクリメントしません。最後の強参照が解放されると、参照先オブジェクトは解放され、残存する弱参照は自動的に None になります。

弱参照は参照サイクルを解消するためのユーザー向けメカニズムです。

弱参照の作成

型アノテーションと式の両方で weak キーワードを使用します:

```
s = "hello"
w: weak str = weak s
```

weak T 型は新しい型コンストラクタで、T は ARC 管理された型 (現在は str、List<T>、Map<K, V>、Set<T>) でなければなりません。

弱参照のアクセス (アップグレード)

弱変数のアクセスは自動的にアップグレードを行います。これは強参照カウントのアトミックなチェックとインクリメントです。結果は常に Option<T> です:

- 参照先がまだ存在する場合 (強参照カウント > 0) は Some(value)
- 参照先が解放済みの場合 (強参照カウント == 0) は None

```
s = "alive"
w: weak str = weak s
case w:
  Some(v):
    print(v)           # "alive"
  None:
    print("deallocated")
```

合体演算子 (??) も弱参照で使用できます:

```
w: weak str = weak s
val = w ?? "default"
```

再代入

弱参照は再代入可能です。古い弱参照は解放され、新しいものが保持されます:

```
a = "first"
b = "second"
w: weak str = weak a
w = weak b
```

スレッドセーフティ

アップグレード操作は内部的に compare-and-swap (CAS) ループを使用するため、スレッド間で安全に使用できます。これは強参照が並行して解放される可能性があるため重要です。

スコープのクリーンアップ

弱参照はスコープから外れると自動的に解放されます。強参照カウントと弱参照カウントの両方がゼロになると、ARC ヘッダーが解放されます。

F-String (文字列補間)

f"..." 構文による文字列補間です。{} 内の式が評価され、文字列に変換されます。

```
name = "world"
print(f"Hello {name}")      # Hello world

a = 1
b = 2
print(f"{a} + {b} = {a + b}")  # 1 + 2 = 3
```

補間で使用可能な型

{} 内には int、float、bool、str、record 型、タプル、またはコレクション型 (List、Map、Set) に評価される任意の式を使用できます。

```
xs = [1, 2, 3]
print(f"items: {xs}")      # items: [1, 2, 3]

t = (1, "hello")
print(f"tuple: {t}")      # tuple: (1, hello)
```

エスケープシーケンス

シーケンス	出力
{	{ (リテラルの波括弧)
}	} (リテラルの波括弧)
\n \r \t \\ \"	通常の文字列と同じ

```
print(f"{{braces}}")      # {braces}
```

型キャスト (as)

as キーワードによる明示的な型変換です。

```
x = 42 as float      # 42.0
y = 3.14 as int      # 3
z = 1 as bool        # true
s = 42 as str        # "42"
b = 255 as u8        # u8 値 255
```

サポートされるキャスト

変換元	変換先	動作
int	float	SIToFP
float	int	切り捨て (FPToSI)
int	bool	0 -> false、非 0 -> true
bool	int	false -> 0、true -> 1
int / float / bool	str	文字列表現
int	u8	切り捨て (下位 8 ビット)
u8	int	ゼロ拡張

int | i8 / i16 / i32 / i64 | 切り捨て (i64 の場合は恒等) |
i8 / i16 / i32 / i64 | int | 符号拡張 (SExt) |

```

int | u8 / u16 / u32 / u64 | 切り捨て (u64 の場合は恒等) |
u8 / u16 / u32 / u64 | int | ゼロ拡張 (ZExt) |
符号付き | 符号付き (より広い) | 符号拡張 (SExt) |
符号付き | 符号付き (より狭い) | 切り捨て |
符号なし | 符号なし/符号付き (より広い) | ゼロ拡張 (ZExt) |
符号なし | 符号なし/符号付き (より狭い) | 切り捨て |
符号付き / 符号なし整数 | float | SIToFP / UIToFP → f64 |
float | 符号付き / 符号なし整数 | FPToSI / FPToUI |
float | f32 | FPTrunc |
f32 | float | FPEExt |
符号付き整数 | f32 | SIToFP |
符号なし整数 | f32 | UIToFP |
f32 | 符号付き / 符号なし整数 | FPToSI / FPToUI |

```

as のターゲット型はジェネリック型を含む完全な型構文をサポートします:

```

x = value as Option<int>
y = data as Map<str, int>

```

as キャスト (ジェネリクスを含む) は、組み込みキャストまたは対応するユーザー定義の operator as が必要です。それ以外はコンパイルエラーになります。文字列から数値への変換には to_int() / to_float() を使用してください。

関連データを持つ enum (ADT)

バリエーション名の後ろに括弧で型を指定することで、enum バリエーションに関連データを持たせることができます。括弧なしのバリエーションは従来通りの単純なタグとして機能します。

```

enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point

```

名前付きフィールド

バリエーションにはドキュメント明確化のために名前付きフィールドをオプションで使用できます。名前付きフィールドはバリエーション定義を自己説明的にしますが、ランタイム動作は変わりません。構築とパターンマッチングは位置ベースのままです。

```

enum Shape:
    Circle(radius: float)
    Rectangle(width: float, height: float)
    Point

```

ルール: - フィールド名は snake_case でなければなりません。- 単一バリエーション内では、すべてのフィールドが名前付きか名前なしのいずれかでなければなりません (混在不可)。- バリエーション内のフィールド名の重複はコンパイルエラーです。

コンストラクタ

EnumName::Variant(value) の構文でデータ付きバリエーションを構築します。フィールドが名前付きであっても引数は常に位置ベースです。

```

c = Shape::Circle(3.14)
r = Shape::Rectangle(4.0, 5.0)
p = Shape::Point

```

バインディング付きパターンマッチング

`case EnumName::Variant(binding):` の形式で関連データを取り出せます。バインディングにはフィールド名ではなくユーザーが選択した変数名を使用します。

```
case c:
  Shape::Circle(r):
    print(r)           # 3.14
  Shape::Rectangle(w, h):
    print(w)
    print(h)
  Shape::Point:
    print("point")
```

内部表現

ADT `enum` はタグ付きユニオンとして格納されます: `{ i64 tag, [N x i8] data }`。N は最大バリエーションのペイロードに合わせたサイズです。

ジェネリック `enum`

`enum` は `<T>` の形式で型パラメータを持てます。これにより、同じ `enum` 構造で異なる型のペイロードを保持できます。

```
enum MyOption<T>:
  MySome(T)
  MyNone
```

使用法

コンパイラが型を推論できない場合は、具体的な型引数を指定してインスタンス化します。

```
a = MyOption<int>::MySome(42)
b = MyOption<int>::MyNone
```

```
case a:
  MyOption::MySome(v):
    print(v)           # 42
  MyOption::MyNone:
    print("none")
```

Error 型

エラーハンドリング用の組み込み型です。Error は `message (str)` と `code (int)` の2つのフィールドを持ちます。

```
e = Error("something went wrong")           # code のデフォルトは 0
e2 = Error("not found", 404)                 # 明示的な code

print(e.message)   # something went wrong
print(e2.code)     # 404
print(e2)          # Error: not found (code: 404)
```

Result を使ったエラーハンドリング

失敗する可能性のある関数は `Result<V, E>` を返します:

```
function divide(a: int, b: int) -> Result<int, Error>:
    if b == 0:
        return Err(Error("division by zero"))
    return Ok(a // b)

case divide(10, 2):
    Ok(v):
        print(v)                # 5
    Err(e):
        print(e.message)
```

戻り値が意味を持たない場合は `Result<Unit, Error>` を使用します:

```
function save(path: str, data: str) -> Result<Unit, Error>:
    return Ok(0 as u8)          # Unit プレースホルダー

case save("/tmp/test.txt", "hello"):
    Ok(_):
        print("saved")
    Err(e):
        print(e.message)
```

Result 型

`Result<V, E>` は 2 つのコンストラクタを持つ組み込みパラメータ化型です:

- `Ok(value)` - 成功バリエント
- `Err(error)` - エラーバリエント

`case` を使った網羅的なエラーハンドリングに使用します。 `Ok` と `Err` の両方のケースをカバーするか、 `_` ワイルドカードを使用する必要があります。

等価性: `Result<T, E>` は `==` と `!=` をサポートします。バリエントが一致し (`Ok/Ok` または `Err/Err`)、内部値が等しい場合に 2 つの結果は等しくなります。

```
function make_ok(v: int) -> Result<int, Error>: return Ok(v)
make_ok(42) == make_ok(42)    # true
make_ok(1)  == make_ok(2)    # false
make_ok(1)  != Err(Error("e")) # true
```

テストマッチャー: - `expect(x).to_be_ok()` - 結果が `Ok` であることをアサート - `expect(x).to_be_err()` - 結果が `Err` であることをアサート

内部表現

`Error` は `{ ptr message, i64 code }` として表現されます。 `Result<V, E>` は `{ i1 isOk, V okValue, E errValue }` として表現されます。

Type

`Type` は組み込みの `type_of` 関数が返す値です。型のコンパイル時 `identity` を表現し、実行時に反射的な比較ができます。

```
print(to_str(type_of(42)))    # int
print(to_str(type_of([1, 2, 3]))) # List
```



```
print(type_of(42) == type_of(100)) # true
print(type_of(42) == type_of(3.14)) # false
```

主な性質:

- 型定義（プリミティブ、コレクション、record、enum、Option、Result、function、Type 自身など）ごとにコンパイル時に一意な identity が与えられる。
- Type 値に対する == / != は表示名ではなく identity を比較する。2 つの異なる record（または同名の record と enum）は常に区別可能。
- print と to_str は人間が読める型名を表示する（例: "int", "List", "Point", "i32"）。
- 低レベルの数値型（i8, i16, ..., f32）は int / float と区別される。
- コレクションのジェネリクスはベース名に畳まれる。type_of([1, 2]) は "List<int>" ではなく "List" を返す。
- Type は反射的。type_of(type_of(x)) は Type 自身を表す Type 値を返す。

内部表現

Type は { i64 id, ptr name } として表現されます。id フィールドは等価性比較に、name フィールドは表示に使われます。両フィールドはコンパイル時に type_of によって格納されます。

union 型

| を使って複数の型を持ちうる変数を宣言できます。

```
x: int | str = 42
x = "hello"      # 再代入可能 (union のいずれかの型)
print(x)         # hello
```

関数引数・戻り値での使用

```
function show(x: int | str) -> int:
    print(x)
    return 0

function get_val(flag: bool) -> int | str:
    if flag:
        return 42
    return "hello"
```

内部表現

union 型は { i64 tag, [N x i8] data } として表現されます。tag は各コンポーネント型のインデックス（アルファベット順ソート後）を示し、data は最大コンポーネントサイズ分のバイト配列です。

等価性

ユニオン型は現在、プリミティブバリエント（int, float, str, bool）について == と != をサポートします。2 つのユニオン値は、同じバリエント（同じタグ）を持ち、内部値が等しい場合に等しくなります。

```
x: int | str = 42
y: int | str = 42
x == y      # true

z: int | str = "42"
x == z      # false (異なるタグ: int と str)
```

制約

- union に含まれない型を代入するとコンパイルエラー
- `int | str` と `str | int` は同じ型（正規化される）
- `print()` で union 値を出力すると、実行時のタグに基づいて適切な型で表示される
- `==` と `!=` はプリミティブバリエーション（`int`, `float`, `str`, `bool`）をサポート。クロージャバリエーションは非対応

any 型

any 型は、任意のプリミティブ値を保持できる組み込みの動的型です。Python の柔軟な型付けアプローチに倣い、静的な型の保証が不要な場面で、ジェネリクスや union 型を使わずに複数の型を扱えるようにします。

保持可能な型

any は以下の型を保持できます:

型	タグ	説明
int	0	64 ビット符号付き整数
float	1	64 ビット浮動小数点数
bool	2	真偽値
str	3	文字列
Unit	4	Unit 値（戻り値なし関数用）

any にはコレクション型（`List`, `Map`, `Set`）、リソース型（`TcpListener`, `TcpStream` 等）、関数ポインタ、ユーザー定義型（`record`, `enum`）は**保持できません**。

内部表現

any はタグ付きユニオンとして実装されています:

```
{ i64 tag, [8 x i8] data } // 合計 16 バイト
```

tag フィールドが格納されている型を識別し、data フィールドに値（最大 8 バイト）を保持します。

ラッピングとアンラッピング

具体型の値は any への代入時に自動的にラップされ、any の値は具体型への代入時に自動的にアンラップされます。

```
# ラッピング: 具体型 → any
x: any = 42          # int が any にラップされる
x = "hello"          # 異なる型への再代入が可能

# アンラッピング: any → 具体型
function get_value() -> any:
    return 42
n: int = get_value() # any(int) が int にアンラップされる

# アンラップ時の int → float 自動昇格
f: float = get_value() # any(int) がアンラップされ float に昇格
```

実行時の型がターゲット型と一致しない場合（例: `any(str)` を `int` 変数にアンラップ）、ランタイムエラーが発生します。

再代入

any 変数は、保持可能な任意の型の値に再代入できます:

```
x: any = 42
x = 3.14      # OK: float を保持
x = "hello"   # OK: str を保持
x = true      # OK: bool を保持
```

算術演算

両方のオペランドが any の場合、実行時の型に基づいて演算がディスパッチされます:

演算	型	結果
+	int + int	int
+	float + float	float
+	int + float	float
+	str + str	str (連結)
-	数値	int または float
*	数値	int または float
*	str * int / int * str	str (繰り返し)
/	数値	float (常に)
//	int // int	int
//	float を含む場合	float
%	数値	int または float
**	数値	float (常に)
単項 -	int	int
単項 -	float	float

一方が any で他方が具体型の場合、具体型の値は演算前に自動的にラップされます。

```
x: any = 10
y: any = x + 20      # 20 が自動ラップされる; 結果は any(int) = 30
```

互換性のない型の組み合わせ (例: str - int) はランタイムエラーになります。

比較演算

演算	動作
==, !=	同じ型同士で動作; int/float の混合比較が可能
<, <=, >, >=	数値 (int/float 混合可) と文字列 (辞書順)

```
x: any = 3
y: any = 3.0
print(x == y)      # true (int/float 比較)
```

比較時の型不一致 (例: int < str) はランタイムエラーになります。

文字列変換

any の値は print() と f-string 補間をサポートします:

```
x: any = 42
print(x)          # 42
print(f"value: {x}") # value: 42
```

変換ルール: `int` → 10 進文字列、`float` → `%g` 形式、`bool` → `"true"/"false"`、`str` → そのまま、`Unit` → `"Unit"`。

型付き関数への `any` の受け渡し

`any` の値を具体的な引数型を持つ関数に渡せます。値は実行時の型チェック付きで自動的にアンラップされます:

```
function add_one(x: int) -> int:
    return x + 1

v: any = 42
result = add_one(v)  # any(int) が int にアンラップされる; 結果は 43
```

型規則 (演算時の型変換)

演算	左辺	右辺	結果型	備考
<code>+</code> <code>-</code> <code>*</code>	<code>int</code>	<code>int</code>	<code>int</code>	
<code>+</code> <code>-</code> <code>*</code>	<code>u8</code>	<code>u8</code>	<code>u8</code>	低レベル型: native 幅の <code>unsigned</code> 演算、暗黙の昇格なし
<code>+</code> <code>-</code> <code>*</code>	<code>float</code> または <code>int</code>	<code>float</code> または <code>int</code> (片方が <code>float</code>)	<code>float</code>	暗黙の <code>float</code> 昇格
<code>/</code>	任意の数値	任意の数値	<code>float</code>	常に <code>float</code>
<code>//</code>	任意の数値	任意の数値	<code>int</code> または <code>float</code>	切り捨て除算 ($-\infty$ 方向); <code>int</code> オペランド同士なら <code>int</code> 、片方が <code>float</code> なら <code>float</code>
<code>**</code>	任意の数値	任意の数値	<code>float</code>	<code>libm pow</code> 使用
<code>%</code>	<code>int</code>	<code>int</code>	<code>int</code>	
<code>%</code>	<code>float</code> または <code>int</code>	<code>float</code> または <code>int</code> (片方が <code>float</code>)	<code>float</code>	
<code>+</code>	<code>str</code>	<code>str</code>	<code>str</code>	文字列結合
<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>	<code>str</code>	<code>str</code>	<code>bool</code>	辞書順比較
<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>	数値または <code>bool</code>	数値または <code>bool</code>	<code>bool</code>	
<code>in</code>	任意	<code>Set</code>	<code>bool</code>	要素がセットに含まれるか
<code>&</code> <code>\ </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code>	<code>int</code>	<code>int</code>	<code>int</code>	<code>float</code> にはエラー
<code>+</code> <code>-</code> <code>*</code>	<code>i32</code>	<code>i32</code>	<code>i32</code>	低レベル型: 暗黙の変換なし、同じ型が必要
<code>/</code> <code>//</code>	<code>i32</code>	<code>i32</code>	<code>i32</code>	符号付き整数除算 (<code>SDiv</code>)
<code>/</code> <code>//</code>	<code>u32</code>	<code>u32</code>	<code>u32</code>	符号なし整数除算 (<code>UDiv</code>)
<code>%</code>	<code>i32</code>	<code>i32</code>	<code>i32</code>	符号付き剰余 (<code>SRem</code>)
<code>%</code>	<code>u32</code>	<code>u32</code>	<code>u32</code>	符号なし剰余 (<code>URem</code>)
<code>+</code> <code>-</code> <code>*</code> <code>/</code>	<code>f32</code>	<code>f32</code>	<code>f32</code>	
<code>==</code> <code>!=</code>	<code>i32/u32</code>	<code>i32/u32</code>	<code>bool</code>	符号非依存の等値比較

演算	左辺	右辺	結果型	備考
< <= > >=	i32	i32	bool	符号付き比較 (ICMP_SLT 等)
< <= > >=	u32	u32	bool	符号なし比較 (ICMP_ULT 等)
>>	i32	i32	i32	算術右シフト (符号保持)
>>	u32	u32	u32	論理右シフト (ゼロ埋め)
**	低レベル	任意	エラー	低レベル型には累乗演算子は未対応
混合	低レベル	異なる型	エラー	低レベル型と高レベル型の混合はコンパイルエラー

エスケープシーケンス (str リテラル内)

シーケンス	意味
\n	改行
\r	復帰
\t	タブ
\\	バックスラッシュ
\"	ダブルクォート
\0	ヌル文字

型安全性の制約

- **暗黙の拡大変換** –関数呼び出しでは安全な拡大変換がサポートされます: `u8 → int`、`u8 → float`、`int → float`。二項演算子では `int` と `float` の混合で `float` 昇格が発生します。`u8` は native 幅の unsigned 演算を行う低レベル型であり、二項演算子での `u8` と `int` の混合はコンパイルエラーです。縮小変換 (例: `float → int`) は暗黙には許可されません。`int` リテラルから `u8` への縮小変換は型アノテーション `b: u8 = 42` でのみ許可されます。
- **変数の型は宣言時に固定される** –一度 `int` として宣言した変数に `float` を再代入することはできない。
- **ビット演算は `int` のみ** –`float` や `bool` に対してビット演算を適用するとコンパイルエラー。
- **`bool` 以外の型も条件式に使える** –`if` の条件式には `int` (`0 = false`、非 `0 = true`) など `bool` 以外も使用可能。
- **数値リテラルセパレータ** –アンダースコアは数値リテラルの視覚的な区切りとして使用可能: `100_000`、`0xFF_FF`、`0b1010_0101`、`3.14_159`。アンダースコアは桁の間に配置する必要があります (先頭、末尾、連続は不可)。
- **数値リテラルサフィックス** –低レベル型はリテラルサフィックスで指定可能: `42i32`、`255u8`、`3.14f32`、`.5f32`、`0xFFu8`、`0b1010u8`。`float` サフィックスを付けた整数リテラル (`42f32`) は `float` 値を生成します。整数サフィックスを付けた浮動小数点リテラル (`3.14i32`) はコンパイルエラーです。範囲外の値 (例: `256u8`、`129i8`) もコンパイルエラーです。
- **低レベル数値型 (`i8`、`i16`、`i32`、`i64`、`u8`、`u16`、`u32`、`u64`、`f32`) は暗黙の変換なし**–低レベル型同士または高レベル型 (`int`、`float`) との混合はコンパイルエラーです。明示的な `as` キャストを使用してください。低レベル整数の `/` 演算子は (Rust と同様に) `float` 除算ではなく整数除算を行います。符号付き型は `SDiv/SRem`、符号なし型は `UDiv/URem` を使用します。
- **符号付き vs 符号なし** –符号付き型 (`i8`、`i16`、`i32`、`i64`) は符号付き比較 (ICMP_SLT 等) と算術右シフト (`AShr`) を使用します。符号なし型 (`u8`、`u16`、`u32`、`u64`) は符号なし比較 (ICMP_ULT 等) と論理右シフト (`LShr`) を使用します。`>>>` 演算子は符号に関わらず常に論理シフトを行います。
- **`int` 算術オーバーフローはランタイムエラー** –高レベル `int` 型の算術演算 (`+`、`-`、`*`、単項`-`) はオーバーフロー時にランタイムエラーを発生させます。Swift のデフォルト動作と同様です。これにより、2 の補

数ラッピングによるサイレントなデータ破損を防ぎます。オーバーフローする定数式はコンパイル時に検出されます。

- **低レベル整数のオーバーフローはラップアラウンド** –低レベル整数型の算術演算はオーバーフロー時に Ry 定義の 2 の補数ラップ（符号付き）またはモジュラー演算（符号なし）を使用します。例えば、`2147483647i32 + 1i32` は `-2147483648` にラップします。明示的なオーバーフロー制御には `checked_add/sub/mul` (`Result<T, Error>` を返す)、`saturating_add/sub/mul` (型の境界値にクランプ)、`wrapping_add/sub/mul` (自己文書化的なラッピング) を使用できます。[関数リファレンス](#)を参照。